Dokumentasi Resmi & Sintaks Swift (Swift 5.10/Swift 6 Up-to-Dat

Bahasa: Swift | Total Chunk/Dokumen: 152 | Disiapkan untuk RAG System

[swift_001] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Overview

Sumber: ht

The Basics
Work with common kinds of data and write basic syntax.
Swift provides many fundamental data types,
including `Int` for integers,
`Double` for floating-point values,
`Bool` for Boolean values,
and `String` for text.
Swift also provides powerful versions of the three primary collection types,
`Array`, `Set`, and `Dictionary`,
as described in .
Swift uses variables to store and refer to values by an identifying name.
Swift also makes extensive use of variables whose values can't be changed.
These are known as constants, and are used throughout Swift to make code safer and clearer in intent
when you work with values that don't need to change.
In addition to familiar types,
Swift introduces advanced types such as tuples.
Tuples enable you to create and pass around groupings of values.
You can use a tuple to return multiple values from a function as a single compound value.
Swift handles the absence of a value
using optional types.
Optionals say either "there *is* a value, which is *x*"
or "there *isn't* a value at all".
Optionals ensure that code always
checks whether a value is missing before using the value,
and non-optional values are guaranteed to never be missing.
Swift is a safe language,
which means it makes it easier for you to find and fix several categories of bugs
as early as possible during the development process,
and lets you guarantee that certain kinds of bugs can't happen.
Type safety enables you to be clear about
the types of values your code works with.
If part of your code requires a `String`,
type safety prevents you from passing it an `Int` by mistake.
Memory safety ensures that you work with valid data only,
not uninitialized memory or deinitialized objects,
and ensures that you work with that data in safe ways ---
even in programs that ru

[swift_002] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Constants and Variables

Sumber: ht

Constants and Variables
Constants and variables associate a name
(such as `maximumNumberOfLoginAttempts` or `welcomeMessage`)
with a value of a particular type
(such as the number `10` or the string `"Hello"`).
The value of a *constant* can't be changed once it's set,
whereas a *variable* can be set to a different value in the future.

[swift_003] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Declaring Constants and Variables

Sumber: ht

Declaring Constants and Variables
Constants and variables must be declared before they're used.
You declare constants with the `let` keyword
and variables with the `var` keyword.
Here's an example of how constants and variables can be used
to track the number of login attempts a user has made:
This code can be read as:
"Declare a new constant called `maximumNumberOfLoginAttempts`,
and give it a value of `10`.
Then, declare a new variable called `currentLoginAttempt`,
and give it an initial value of `0`."
In this example,
the maximum number of allowed login attempts is declared as a constant,
because the maximum value never changes.
The current login attempt counter is declared as a variable,
because this value must be incremented after each failed login attempt.
If a stored value in your code won't change,
always declare it as a constant with the `let` keyword.
Use variables only for storing values that change.
When you declare a constant or a variable,
you can give it a value as part of that declaration,
like the examples above.
Alternatively,
you can assign its initial value later in the program,
as long as it's guaranteed to have a value
before the first time you read from it.
In this example,
the maximum number of login attempts is constant,
and its value depends on the environment.
In the development environment,
it has a value of 100;
in any other environment, its value is 10.
Both branches of the `if` statement
initialize `maximumNumberOfLoginAttempts` with some value,

guaranteeing that the constant always gets a value.
For information about how Swift checks your code
when you set an initial value this way,
see .
You can declare multiple constants or multiple variables on a single line,
separated by commas:

Contoh Kode / Perintah CLI:

```
let maximumNumberOfLoginAttempts = 10
var currentLoginAttempt = 0

// ----------------------------------------

-->

This code can be read as:

"Declare a new constant called `maximumNumberOfLoginAttempts`,
and give it a value of `10`.
Then, declare a new variable called `currentLoginAttempt`,
and give it an initial value of `0`."

In this example,
the maximum number of allowed login attempts is declared as a constant,
because the maximum value never changes.
The current login attempt counter is declared as a variable,
because this value must be incremented after each failed login attempt.

If a stored value in your code won't change,
always declare it as a constant with the `let` keyword.
Use variables only for storing values that change.

When you declare a constant or a variable,
you can give it a value as part of that declaration,
like the examples above.
Alternatively,
you can assign its initial value later in the program,
as long as it's guaranteed to have a value
before the first time you read from it.

// ----------------------------------------

<!--
  - test: `constantsWithDeferredInitialization`
```

[swift_004] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Type Annotations

Sumber: ht

Type Annotations
You can provide a *type annotation* when you declare a constant or variable,
to be clear about the kind of values the constant or variable can store.
Write a type annotation by placing a colon after the constant or variable name,
followed by a space, followed by the name of the type to use.
This example provides a type annotation for a variable called `welcomeMessage`,
to indicate that the variable can store `String` values:
The colon in the declaration means "...of type...,"

so the code above can be read as:
"Declare a variable called `welcomeMessage` that's of type `String`."
The phrase "of type `String`" means "can store any `String` value."
Think of it as meaning "the type of thing" (or "the kind of thing") that can be stored.
The `welcomeMessage` variable can now be set to any string value without error:
You can define multiple related variables of the same type on a single line,
separated by commas, with a single type annotation after the final variable name:
> Note: It's rare that you need to write type annotations in practice.
> If you provide an initial value for a constant or variable at the point that it's defined,
> Swift can almost always infer the type to be used for that constant or variable,
> as described in .
> In the `welcomeMessage` example above, no initial value is provided,
> and so the type of the `welcomeMessage` variable is specified with a type annotation
> rather than being inferred from an initial value.

Contoh Kode / Perintah CLI:

```
var welcomeMessage: String

// ----------------------------------------

-->

The colon in the declaration means "...of type...,"
so the code above can be read as:

"Declare a variable called `welcomeMessage` that's of type `String`."

The phrase "of type `String`" means "can store any `String` value."
Think of it as meaning "the type of thing" (or "the kind of thing") that can be stored.

The `welcomeMessage` variable can now be set to any string value without error:

// ----------------------------------------

<!--
  - test: `typeAnnotations`
```

[swift_005] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Naming Constants and Variables

Sumber: ht

Naming Constants and Variables
Constant and variable names can contain almost any character,
including Unicode characters:
Constant and variable names can't contain
whitespace characters, mathematical symbols, arrows, private-use Unicode scalar values,
or line- and box-drawing characters.
Nor can they begin with a number,
although numbers may be included elsewhere within the name.
Once you've declared a constant or variable of a certain type,
you can't declare it again with the same name,
or change it to store values of a different type.
Nor can you change a constant into a variable
or a variable into a constant.
> Note: If you need to give a constant or variable the same name as a reserved Swift keyword,

> surround the keyword with backticks (`` ` ``) when using it as a name.
> However, avoid using keywords as names unless you have absolutely no choice.
You can change the value of an existing variable to another value of a compatible type.
In this example, the value of `friendlyWelcome` is changed from
`"Hello!"` to `"Bonjour!"`:
Unlike a variable, the value of a constant can't be changed after it's set.
Attempting to do so is reported as an error when your code is compiled:

Contoh Kode / Perintah CLI:

```
let ? = 3.14159
let ?? = "????"
let ?? = "dogcow"

// ----------------------------------------

-->

Constant and variable names can't contain
whitespace characters, mathematical symbols, arrows, private-use Unicode scalar values,
or line- and box-drawing characters.
Nor can they begin with a number,
although numbers may be included elsewhere within the name.

Once you've declared a constant or variable of a certain type,
you can't declare it again with the same name,
or change it to store values of a different type.
Nor can you change a constant into a variable
or a variable into a constant.

> Note: If you need to give a constant or variable the same name as a reserved Swift keyword,
> surround the keyword with backticks (`` ` ``) when using it as a name.
> However, avoid using keywords as names unless you have absolutely no choice.

You can change the value of an existing variable to another value of a compatible type.
In this example, the value of `friendlyWelcome` is changed from
`"Hello!"` to `"Bonjour!"`:

// ----------------------------------------

<!--
  - test: `constantsAndVariables`
```

[swift_006] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Printing Constants and Variables

Sumber: ht

Printing Constants and Variables
You can print the current value of a constant or variable with the `print(_:separator:terminator:)` function:
The `print(_:separator:terminator:)` function
is a global function that prints one or more values
to an appropriate output.
In Xcode, for example,
the `print(_:separator:terminator:)` function prints its output in Xcode's "console" pane.
The `separator` and `terminator` parameter have default values,
so you can omit them when you call this function.
By default, the function terminates the line it prints by adding a line break.
To print a value without a line break after it,

pass an empty string as the terminator --- for example,
`print(someValue, terminator: "")`.
For information about parameters with default values,
see .
Swift uses *string interpolation* to include the name of a constant or variable
as a placeholder in a longer string,
and to prompt Swift to replace it with the current value of that constant or variable.
Wrap the name in parentheses and escape it with a backslash before the opening parenthesis:
> Note: All options you can use with string interpolation
> are described in .

Contoh Kode / Perintah CLI:

```
print(friendlyWelcome)
// Prints "Bonjour!"

// ----------------------------------------

-->

The `print(_:separator:terminator:)` function
is a global function that prints one or more values
to an appropriate output.
In Xcode, for example,
the `print(_:separator:terminator:)` function prints its output in Xcode's "console" pane.
The `separator` and `terminator` parameter have default values,
so you can omit them when you call this function.
By default, the function terminates the line it prints by adding a line break.
To print a value without a line break after it,
pass an empty string as the terminator --- for example,
`print(someValue, terminator: "")`.
For information about parameters with default values,
see <doc:Functions#Default-Parameter-Values>.

<!--
  - test: `printingWithoutNewline`

// ----------------------------------------

-->

<!--
  QUESTION: have I referred to Xcode's console correctly here?
  Should I mention other output streams, such as the REPL / playgrounds?
-->

<!--
  NOTE: this is a deliberately simplistic description of what you can do with print().
  It will be expanded later on.
-->

Swift uses *string interpolation* to include the name of a constant or variable
as a placeholder in a longer string,
and to prompt Swift to replace it with the current value of that constant or variable.
Wrap the name in parentheses and escape it with a backslash before the opening parenthesis:
```

[swift_007] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Comments

Sumber: ht

Comments
Use comments to include nonexecutable text in your code,
as a note or reminder to yourself.
Comments are ignored by the Swift compiler when your code is compiled.
Comments in Swift are very similar to comments in C.
Single-line comments begin with two forward-slashes (`//`):
Multiline comments start with a forward-slash followed by an asterisk (`/*`)
and end with an asterisk followed by a forward-slash (`*/`):
Unlike multiline comments in C,
multiline comments in Swift can be nested inside other multiline comments.
You write nested comments by starting a multiline comment block
and then starting a second multiline comment within the first block.
The second block is then closed, followed by the first block:
Nested multiline comments enable you to comment out large blocks of code quickly and easily,
even if the code already contains multiline comments.

Contoh Kode / Perintah CLI:

```
// This is a comment.

// ----------------------------------------

-->

Multiline comments start with a forward-slash followed by an asterisk (`/*`)
and end with an asterisk followed by a forward-slash (`*/`):

// ----------------------------------------

<!--
  - test: `comments`
```

[swift_008] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Semicolons

Sumber: ht

Semicolons
Unlike many other languages,
Swift doesn't require you to write a semicolon (`;`) after each statement in your code,
although you can do so if you wish.
However, semicolons *are* required
if you want to write multiple separate statements on a single line:

Contoh Kode / Perintah CLI:

```
let cat = "?"; print(cat)
// Prints "?".
```

[swift_009] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Integers

Sumber: ht

Integers
Integers are whole numbers with no fractional component,
such as `42` and `-23`.
Integers are either *signed* (positive, zero, or negative)
or *unsigned* (positive or zero),
and their maximum and minimum value depends on their *size*
(the number of bits used to store values).
The integer types include their size and sign in their names ---
for example, an 8-bit unsigned integer is of type `UInt8`,
and a 32-bit signed integer is of type `Int32`.
Like all types in Swift, these integer types have capitalized names.
In most cases,
when you don't need to specify the exact integer size,
you use the `Int` type described below.
Integer types behave like most arithmetic you do by hand;
integer math produces results without approximating.
These characteristics make integers suitable for counting
and other calculations that represent exact amounts ---
for example,
finding the longest line in a text file,
applying a score multiplier in a game,
or totaling prices on a receipt.
Although integers don't have a fractional component,
you can use integers to represent quantities with fractions
by counting a fractional part.
For example,
you can represent $1.23 by storing the number `123`
in an integer that counts cents.
This approach is known as *fixed-point math*
because the decimal point is at a fixed position in the number.
In the example above,
the number `123` is understood to have a decimal point
before the last two digits.
> Note:
> For calculations in a regulated area like finance or construction,
> or in a domain that has an expectation of high-precision results,
> you might need a special-purpose numeric type
> that implements behaviors such as rounding and truncation,
> according to that area's requirements.

[swift_010] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Integer Bounds

Sumber: ht

Integer Bounds
You can access the minimum and maximum values of each integer type
with its `min` and `max` properties:
The values of these properties are of the appropriate-sized number type
(such as `UInt8` in the example above)
and can therefore be used in expressions alongside other values of the same type.
Calculations that produce out-of-bounds results,
like a number larger than the `max` property,
stop the program's execution instead of storing an invalid result.
You can explicitly make the operation overflow instead,
as described in .

Contoh Kode / Perintah CLI:

```
let minValue = UInt8.min  // minValue is equal to 0, and is of type UInt8
let maxValue = UInt8.max  // maxValue is equal to 255, and is of type UInt8
```

[swift_011] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Int

Sumber: ht

Int
In most cases, you don't need to pick a specific size of integer to use in your code.
Swift provides an additional integer type, `Int`,
which has the same size as the current platform's native word size:
- On a 32-bit platform, `Int` is the same size as `Int32`.
- On a 64-bit platform, `Int` is the same size as `Int64`.
Unless you need to work with a specific size of integer,
always use `Int` for integer values in your code.
This aids code consistency and interoperability.
Even on 32-bit platforms, `Int` can store any value between `-2,147,483,648` and `2,147,483,647`,
and is large enough for many integer ranges.

[swift_012] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): UInt

Sumber: ht

UInt
Swift also provides an unsigned integer type, `UInt`,
which has the same size as the current platform's native word size:
- On a 32-bit platform, `UInt` is the same size as `UInt32`.
- On a 64-bit platform, `UInt` is the same size as `UInt64`.
> Note: Use `UInt` only when you specifically need
> an unsigned integer type with the same size as the platform's native word size.
> If this isn't the case, `Int` is preferred,
> even when the values to be stored are known to be nonnegative.
> A consistent use of `Int` for integer values aids code interoperability,

> avoids the need to convert between different number types,
> and matches integer type inference, as described in .

[swift_013] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Floating-Point Numbers

Sumber: ht

Floating-Point Numbers
Floating-point numbers have a fractional component,
such as `3.14159`, `0.1`, and `-273.15`.
Swift provides a variety of floating-point types
that support different sizes of numbers,
just like it has different sizes of integers.
If you don't need to specify an exact size, use `Double`.
Otherwise,
use the type that includes the needed size in its name,
such as `Float16` or `Float80`.
Following common terminology for floating-point math,
`Float` uses 32 bits and `Double` uses 64 bits.
You can also write these types as `Float32` or `Float64`.
For example,
graphics code often uses `Float` to match the GPU's fastest data type.
Some floating-point types are supported only by certain platforms,
but `Float` and `Double` are available on all platforms.
Floating-point numbers let you work with
very small and very large numbers,
but can't represent every possible value in that range.
Unlike integer calculations,
which always produce an exact result,
floating-point math rounds results to the nearest representable number.
For example,
when storing the number 10,000 as a `Float`,
the next largest number you can represent is 10,000.001 ----
values between these two numbers round to one or the other.
The space between numbers is also variable;
there are larger spaces between large numbers
than between small numbers.
For example,
the next `Float` value after 0.001 is 0.0010000002,
which is smaller than the spacing after 10,000.
Floating-point numbers have values for
negative zero, infinity, and negative infinity,
which represent overflow and underflow in calculations.
They also have not-a-number (NaN) values
to represent an invalid or undefined result,
such as dividing zero by zero.
This behavior is different from integers,
which stop the program if they can'

[swift_014] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Type Safety and Type Inference

Sumber: ht

Type Safety and Type Inference
Every value in a Swift program has a type.
Every place you store a value ---
including constants, variables, and properties ---
also has a type.
You might write the type explicitly using a type annotation,
or Swift might infer the type from an initial value.
Every place in your code where you provide a value,
that value's type must match the place you use it.
For example,
if part of your code requires a `String`,
you can't pass it an `Int` by mistake.
This kind of checking makes Swift a *type-safe* language.
A type safe language encourages you to be clear about
the types of values your code works with.
Values of one type are never implicitly converted to another type.
However, some types can be explicitly converted.
When building code,
Swift checks the code for type safety
and flags any mismatched types as errors.
Type checking helps you avoid errors when you're working with different types of values.
However, this doesn't mean that you have to specify the type of
every constant and variable that you declare.
If you don't specify the type of value you need,
Swift uses *type inference* to work out the appropriate type.
Type inference enables a compiler to
deduce the type of a particular expression automatically when it compiles your code,
simply by examining the values you provide.
Because of type inference, Swift requires far fewer type declarations
than languages such as C or Objective-C.
Constants and variables are still explicitly typed,
but much of the work of specifying their type is done for you.
Type inference is particularly useful
when you declare a constant or variable with an initial value.
This is often done by assigning a *literal value* (or *literal*)
to the constant or variable at the point that you declare it.
(A literal

Contoh Kode / Perintah CLI:

```
let meaningOfLife = 42
// meaningOfLife is inferred to be of type Int

// ----------------------------------------

-->

Likewise, if you don't specify a type for a floating-point literal,
Swift infers that you want to create a `Double`:

// ----------------------------------------
```

```
<!--
  - test: `typeInference`
```

[swift_015] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Numeric Literals

Sumber: ht

Numeric Literals
Integer literals can be written as:
- A *decimal* number, with no prefix
- A *binary* number, with a `0b` prefix
- An *octal* number, with a `0o` prefix
- A *hexadecimal* number, with a `0x` prefix
All of these integer literals have a decimal value of `17`:
Floating-point literals can be decimal (with no prefix),
or hexadecimal (with a `0x` prefix).
They must always have a number (or hexadecimal number) on both sides of the decimal point.
Decimal floats can also have an optional *exponent*,
indicated by an uppercase or lowercase `e`;
hexadecimal floats must have an exponent,
indicated by an uppercase or lowercase `p`.
For decimal numbers with an exponent of `x`,
the base number is multiplied by 10?:
- `1.25e2` means 1.25 x 10², or `125.0`.
- `1.25e-2` means 1.25 x 10?², or `0.0125`.
For hexadecimal numbers with an exponent of `x`,
the base number is multiplied by 2?:
- `0xFp2` means 15 x 2², or `60.0`.
- `0xFp-2` means 15 x 2?², or `3.75`.
All of these floating-point literals have a decimal value of `12.1875`:
Numeric literals can contain extra formatting to make them easier to read.
Both integers and floats can be padded with extra zeros
and can contain underscores to help with readability.
Neither type of formatting affects the underlying value of the literal:

Contoh Kode / Perintah CLI:

```
let decimalInteger = 17
let binaryInteger = 0b10001       // 17 in binary notation
let octalInteger = 0o21           // 17 in octal notation
let hexadecimalInteger = 0x11     // 17 in hexadecimal notation

// ----------------------------------------

-->

Floating-point literals can be decimal (with no prefix),
or hexadecimal (with a `0x` prefix).
They must always have a number (or hexadecimal number) on both sides of the decimal point.
Decimal floats can also have an optional *exponent*,
indicated by an uppercase or lowercase `e`;
hexadecimal floats must have an exponent,
indicated by an uppercase or lowercase `p`.

<!--
```

```
  - test: `float-required-vs-optional-exponent-err`

// ----------------------------------------

-->

<!--
  - test: `float-required-vs-optional-exponent`
```

[swift_016] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Numeric Type Conversion

Sumber: ht

Numeric Type Conversion
Use the `Int` type for all general-purpose integer constants and variables in your code,
even if they're known to be nonnegative.
Using the default integer type in everyday situations means that
integer constants and variables are immediately interoperable in your code
and will match the inferred type for integer literal values.
Use other integer types only when they're specifically needed for the task at hand,
because of explicitly sized data from an external source,
or for performance, memory usage, or other necessary optimization.
Using explicitly sized types in these situations
helps to catch any accidental value overflows
and implicitly documents the nature of the data being used.

[swift_017] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Integer Conversion

Sumber: ht

Integer Conversion
The range of numbers that can be stored in an integer constant or variable
is different for each numeric type.
An `Int8` constant or variable can store numbers between `-128` and `127`,
whereas a `UInt8` constant or variable can store numbers between `0` and `255`.
A number that won't fit into a constant or variable of a sized integer type
is reported as an error when your code is compiled:
Because each numeric type can store a different range of values,
you must opt in to numeric type conversion on a case-by-case basis.
This opt-in approach prevents hidden conversion errors
and helps make type conversion intentions explicit in your code.
To convert one specific number type to another,
you initialize a new number of the desired type with the existing value.
In the example below,
the constant `twoThousand` is of type `UInt16`,
whereas the constant `one` is of type `UInt8`.
They can't be added together directly,
because they're not of the same type.
Instead, this example calls `UInt16(one)` to create
a new `UInt16` initialized with the value of `one`,
and uses this value in place of the original:

Because both sides of the addition are now of type `UInt16`,
the addition is allowed.
The output constant (`twoThousandAndOne`) is inferred to be of type `UInt16`,
because it's the sum of two `UInt16` values.
`SomeType(ofInitialValue)` is the default way to call the initializer of a Swift type
and pass in an initial value.
Behind the scenes, `UInt16` has an initializer that accepts a `UInt8` value,
and so this initializer is used to make a new `UInt16` from an existing `UInt8`.
You can't pass in *any* type here, however ---
it has to be a type for which `UInt16` provides an initializer.
Extending existing types to provide initializers that accept new types

Contoh Kode / Perintah CLI:

```
let cannotBeNegative: UInt8 = -1
// UInt8 can't store negative numbers, and so this will report an error
let tooBig: Int8 = Int8.max + 1
// Int8 can't store a number larger than its maximum value,
// and so this will also report an error

// ----------------------------------------

-->

Because each numeric type can store a different range of values,
you must opt in to numeric type conversion on a case-by-case basis.
This opt-in approach prevents hidden conversion errors
and helps make type conversion intentions explicit in your code.

To convert one specific number type to another,
you initialize a new number of the desired type with the existing value.
In the example below,
the constant `twoThousand` is of type `UInt16`,
whereas the constant `one` is of type `UInt8`.
They can't be added together directly,
because they're not of the same type.
Instead, this example calls `UInt16(one)` to create
a new `UInt16` initialized with the value of `one`,
and uses this value in place of the original:

// ----------------------------------------

<!--
  - test: `typeConversion`
```

[swift_018] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Integer and Floating-Point Conversion

Sumber: ht

Integer and Floating-Point Conversion
Conversions between integer and floating-point numeric types must be made explicit:
Here, the value of the constant `three` is used to create a new value of type `Double`,
so that both sides of the addition are of the same type.
Without this conversion in place, the addition would not be allowed.
Floating-point to integer conversion must also be made explicit.
An integer type can be initialized with a `Double` or `Float` value:

Floating-point values are always truncated when used to initialize a new integer value in this way.
This means that `4.75` becomes `4`, and `-3.9` becomes `-3`.
> Note: The rules for combining numeric constants and variables are different from
> the rules for numeric literals.
> The literal value `3` can be added directly to the literal value `0.14159`,
> because number literals don't have an explicit type in and of themselves.
> Their type is inferred only at the point that they're evaluated by the compiler.

Contoh Kode / Perintah CLI:

```
let three = 3
let pointOneFourOneFiveNine = 0.14159
let pi = Double(three) + pointOneFourOneFiveNine
// pi equals 3.14159, and is inferred to be of type Double

// ----------------------------------------

-->

Here, the value of the constant `three` is used to create a new value of type `Double`,
so that both sides of the addition are of the same type.
Without this conversion in place, the addition would not be allowed.

Floating-point to integer conversion must also be made explicit.
An integer type can be initialized with a `Double` or `Float` value:

// ----------------------------------------

<!--
  - test: `typeConversion`
```

[swift_019] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Type Aliases

Sumber: ht

Type Aliases
Type aliases define an alternative name for an existing type.
You define type aliases with the `typealias` keyword.
Type aliases are useful when you want to refer to an existing type
by a name that's contextually more appropriate,
such as when working with data of a specific size from an external source:
Once you define a type alias,
you can use the alias anywhere you might use the original name:
Here, `AudioSample` is defined as an alias for `UInt16`.
Because it's an alias,
the call to `AudioSample.min` actually calls `UInt16.min`,
which provides an initial value of `0` for the `maxAmplitudeFound` variable.

Contoh Kode / Perintah CLI:

```
typealias AudioSample = UInt16

// ----------------------------------------

-->

Once you define a type alias,
you can use the alias anywhere you might use the original name:
```

```
// ----------------------------------------

<!--
  - test: `typeAliases`
```

[swift_020] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Booleans

Sumber: ht

Booleans
Swift has a basic *Boolean* type, called `Bool`.
Boolean values are referred to as *logical*,
because they can only ever be true or false.
Swift provides two Boolean constant values,
`true` and `false`:
The types of `orangesAreOrange` and `turnipsAreDelicious`
have been inferred as `Bool` from the fact that
they were initialized with Boolean literal values.
As with `Int` and `Double` above,
you don't need to declare constants or variables as `Bool`
if you set them to `true` or `false` as soon as you create them.
Boolean values are particularly useful when you work with conditional statements
such as the `if` statement:
Conditional statements such as the `if` statement are covered in more detail in .
Swift's type safety prevents non-Boolean values from being substituted for `Bool`.
The following example reports a compile-time error:
However, the alternative example below is valid:
The result of the `i == 1` comparison is of type `Bool`,
and so this second example passes the type checking.
Comparisons like `i == 1` are discussed in .
As with other examples of type safety in Swift,
this approach avoids accidental errors
and ensures that the intention of a particular section of code is always clear.

Contoh Kode / Perintah CLI:

```
let orangesAreOrange = true
let turnipsAreDelicious = false

// ----------------------------------------

-->

The types of `orangesAreOrange` and `turnipsAreDelicious`
have been inferred as `Bool` from the fact that
they were initialized with Boolean literal values.
As with `Int` and `Double` above,
you don't need to declare constants or variables as `Bool`
if you set them to `true` or `false` as soon as you create them.

Boolean values are particularly useful when you work with conditional statements
such as the `if` statement:

// ----------------------------------------

<!--
  - test: `booleans`
```

[swift_021] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Tuples

Sumber: ht

Tuples
Tuples group multiple values into a single compound value.
The values within a tuple can be of any type
and don't have to be of the same type as each other.
In this example, `(404, "Not Found")` is a tuple that describes an *HTTP status code*.
An HTTP status code is a special value returned by a web server whenever you request a web page.
A status code of `404 Not Found` is returned if you request a webpage that doesn't exist.
The `(404, "Not Found")` tuple groups together an `Int` and a `String`
to give the HTTP status code two separate values:
a number and a human-readable description.
It can be described as "a tuple of type `(Int, String)`".
You can create tuples from any permutation of types,
and they can contain as many different types as you like.
There's nothing stopping you from having
a tuple of type `(Int, Int, Int)`, or `(String, Bool)`,
or indeed any other permutation you require.
You can *decompose* a tuple's contents into separate constants or variables,
which you then access as usual:
If you only need some of the tuple's values,
ignore parts of the tuple with an underscore (`_`)
when you decompose the tuple:
Alternatively,
access the individual element values in a tuple using index numbers starting at zero:
You can name the individual elements in a tuple when the tuple is defined:
If you name the elements in a tuple,
you can use the element names to access the values of those elements:
Tuples are particularly useful as the return values of functions.
A function that tries to retrieve a web page might return the `(Int, String)` tuple type
to describe the success or failure of the page retrieval.
By returning a tuple with two distinct values,
each of a different type,
the function provides more useful information about its outcome
than if it cou

Contoh Kode / Perintah CLI:

```
let http404Error = (404, "Not Found")
// http404Error is of type (Int, String), and equals (404, "Not Found")

// ----------------------------------------

-->

The `(404, "Not Found")` tuple groups together an `Int` and a `String`
to give the HTTP status code two separate values:
a number and a human-readable description.
It can be described as "a tuple of type `(Int, String)`".

You can create tuples from any permutation of types,
and they can contain as many different types as you like.
There's nothing stopping you from having
```

```
a tuple of type `(Int, Int, Int)`, or `(String, Bool)`,
or indeed any other permutation you require.

You can *decompose* a tuple's contents into separate constants or variables,
which you then access as usual:

// ----------------------------------------

<!--
  - test: `tuples`
```

[swift_022] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Optionals

Sumber: ht

Optionals
You use *optionals* in situations where a value may be absent.
An optional represents two possibilities:
Either there *is* a value of a specified type,
and you can unwrap the optional to access that value,
or there *isn't* a value at all.
As an example of a value that might be missing,
Swift's `Int` type has an initializer
that tries to convert a `String` value into an `Int` value.
However, only some strings can be converted into integers.
The string `"123"` can be converted into the numeric value `123`,
but the string `"hello, world"` doesn't have a corresponding numeric value.
The example below uses the initializer to try to convert a `String` into an `Int`:
Because the initializer in the code above might fail,
it returns an *optional* `Int`, rather than an `Int`.
To write an optional type,
you write a question mark (`?`)
after the name of the type that the optional contains ---
for example, the type of an optional `Int` is `Int?`.
An optional `Int` always contains
either some `Int` value or no value at all.
It can't contain anything else, like a `Bool` or `String` value.

Contoh Kode / Perintah CLI:

```
let possibleNumber = "123"
let convertedNumber = Int(possibleNumber)
// The type of convertedNumber is "optional Int"
```

[swift_023] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): nil

Sumber: ht

nil
You set an optional variable to a valueless state
by assigning it the special value `nil`:
If you define an optional variable without providing a default value,
the variable is automatically set to `nil`:
You can use an `if` statement to find out whether an optional contains a value

by comparing the optional against `nil`.
You perform this comparison with the "equal to" operator (`==`)
or the "not equal to" operator (`!=`).
If an optional has a value, it's considered as "not equal to" `nil`:
You can't use `nil` with non-optional constants or variables.
If a constant or variable in your code needs to work with
the absence of a value under certain conditions,
declare it as an optional value of the appropriate type.
A constant or variable that's declared as a non-optional value
is guaranteed to never contain a `nil` value.
If you try to assign `nil` to a non-optional value,
you'll get a compile-time error.
This separation of optional and non-optional values
lets you explicitly mark what information can be missing,
and makes it easier to write code that handle missing values.
You can't accidentally treat an optional as if it were non-optional
because this mistake produces an error at compile time.
After you unwrap the value,
none of the other code that works with that value needs to check for `nil`,
so there's no need to repeatedly check the same value
in different parts of your code.
When you access an optional value,
your code always handles both the `nil` and non-`nil` case.
There are several things you can do when a value is missing,
as described in the following sections:
- Skip the code that operates on the value when it's `nil`.
- Propagate the `nil` value,
by returning `nil`
or using the `?.` operator described in .
- Provide a fallback value, using the `??`

Contoh Kode / Perintah CLI:

```
var serverResponseCode: Int? = 404
// serverResponseCode contains an actual Int value of 404
serverResponseCode = nil
// serverResponseCode now contains no value

// ----------------------------------------

-->

If you define an optional variable without providing a default value,
the variable is automatically set to `nil`:

// ----------------------------------------

<!--
  - test: `optionals`
```

[swift_024] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Optional Binding

Sumber: ht

Optional Binding
You use optional binding to find out whether an optional contains a value,
and if so, to make that value available as a temporary constant or variable.
Optional binding can be used with `if`, `guard`, and `while` statements
to check for a value inside an optional,
and to extract that value into a constant or variable,
as part of a single action.
For more information about `if`, `guard`, and `while` statements,
see .
Write an optional binding for an `if` statement as follows:
You can rewrite the `possibleNumber` example from
the section
to use optional binding rather than forced unwrapping:
This code can be read as:
"If the optional `Int` returned by `Int(possibleNumber)` contains a value,
set a new constant called `actualNumber` to the value contained in the optional."
If the conversion is successful,
the `actualNumber` constant becomes available for use within
the first branch of the `if` statement.
It has already been initialized with the value contained within the optional,
and has the corresponding non-optional type.
In this case, the type of `Int(possibleNumber)` is `Int?`,
so the type of `actualNumber` is `Int`.
If you don't need to refer to the original, optional constant or variable
after accessing the value it contains,
you can use the same name for the new constant or variable:
This code starts by checking whether `myNumber` contains a value,
just like the code in the previous example.
If `myNumber` has a value,
the value of a new constant named `myNumber` is set to that value.
Inside the body of the `if` statement,
writing `myNumber` refers to that new non-optional constant.
Writing `myNumber` before or after the `if` statement
refers to the original optional integer constant.
Because this kind of code is so common,
you can use a shorter

Contoh Kode / Perintah CLI:

```
if let <#constantName#> = <#someOptional#> {
   <#statements#>
}

// ----------------------------------------

if let actualNumber = Int(possibleNumber) {
    print("The string \"\(possibleNumber)\" has an integer value of \(actualNumber)")
} else {
    print("The string \"\(possibleNumber)\" couldn't be converted to an integer")
}
// Prints "The string "123" has an integer value of 123".
```

```
// ----------------------------------------

-->

This code can be read as:

"If the optional `Int` returned by `Int(possibleNumber)` contains a value,
set a new constant called `actualNumber` to the value contained in the optional."

If the conversion is successful,
the `actualNumber` constant becomes available for use within
the first branch of the `if` statement.
It has already been initialized with the value contained within the optional,
and has the corresponding non-optional type.
In this case, the type of `Int(possibleNumber)` is `Int?`,
so the type of `actualNumber` is `Int`.

If you don't need to refer to the original, optional constant or variable
after accessing the value it contains,
you can use the same name for the new constant or variable:
```

[swift_025] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Providing a Fallback Value

Sumber: ht

Providing a Fallback Value
Another way to handle a missing value is to supply
a default value using the nil-coalescing operator (`??`).
If the optional on the left of the `??` isn't `nil`,
that value is unwrapped and used.
Otherwise, the value on the right of `??` is used.
For example,
the code below greets someone by name if one is specified,
and uses a generic greeting when the name is `nil`.
For more information about using `??` to provide a fallback value,
see .

Contoh Kode / Perintah CLI:

```
let name: String? = nil
let greeting = "Hello, " + (name ?? "friend") + "!"
print(greeting)
// Prints "Hello, friend!"
```

[swift_026] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Force Unwrapping

Sumber: ht

Force Unwrapping
When `nil` represents an unrecoverable failure,
such as a programmer error or corrupted state,
you can access the underlying value of the optional
by adding an exclamation mark (`!`) to the end of the optional's name.
This is known as *force unwrapping* the optional's value.
When you force unwrap a non-`nil` value,

the result is its unwrapped value.
Force unwrapping a `nil` value triggers a runtime error.
The `!` is, effectively, a shorter spelling of [`fatalError(_:file:line:)`][].
For example, the code below shows two equivalent approaches:
[`fatalError(_:file:line:)`]: https://developer.apple.com/documentation/swift/fatalerror(_:file:line:)
Both versions of the code above depend on `convertedNumber`
always containing a value.
Writing that requirement as part of the code,
using either of the approaches above,
lets your code check that the requirement is true at runtime.
For more information about enforcing data requirements
and checking assumptions at runtime,
see .

Contoh Kode / Perintah CLI:

```
let possibleNumber = "123"
let convertedNumber = Int(possibleNumber)

let number = convertedNumber!

guard let number = convertedNumber else {
    fatalError("The number was invalid")
}
```

[swift_027] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Implicitly Unwrapped Optionals

Sumber: ht

Implicitly Unwrapped Optionals
As described above,
optionals indicate that a constant or variable is allowed to have "no value".
Optionals can be checked with an `if` statement to see if a value exists,
and can be conditionally unwrapped with optional binding
to access the optional's value if it does exist.
Sometimes it's clear from a program's structure that an optional will *always* have a value,
after that value is first set.
In these cases, it's useful to remove the need
to check and unwrap the optional's value every time it's accessed,
because it can be safely assumed to have a value all of the time.
These kinds of optionals are defined as *implicitly unwrapped optionals*.
You write an implicitly unwrapped optional by placing an exclamation point (`String!`)
rather than a question mark (`String?`) after the type that you want to make optional.
Rather than placing an exclamation point after the optional's name when you use it,
you place an exclamation point after the optional's type when you declare it.
Implicitly unwrapped optionals are useful when
an optional's value is confirmed to exist immediately after the optional is first defined
and can definitely be assumed to exist at every point thereafter.
The primary use of implicitly unwrapped optionals in Swift is during class initialization,
as described in .
Don't use an implicitly unwrapped optional when there's a possibility of
a variable becoming `nil` at a later point.
Always use a normal optional type if you need to check for a `nil` value
during the lifetime of a variable.

An implicitly unwrapped optional is a normal optional behind the scenes,
but can also be used like a non-optional value,
without the need to unwrap the optional value each time it's accessed.
The following example shows the difference i

Contoh Kode / Perintah CLI:

```
let possibleString: String? = "An optional string."
let forcedString: String = possibleString! // Requires explicit unwrapping

let assumedString: String! = "An implicitly unwrapped optional string."
let implicitString: String = assumedString // Unwrapped automatically

// ----------------------------------------

-->

You can think of an implicitly unwrapped optional as
giving permission for the optional to be force-unwrapped if needed.
When you use an implicitly unwrapped optional value,
Swift first tries to use it as an ordinary optional value;
if it can't be used as an optional, Swift force-unwraps the value.
In the code above,
the optional value `assumedString` is force-unwrapped
before assigning its value to `implicitString`
because `implicitString` has an explicit, non-optional type of `String`.
In code below,
`optionalString` doesn't have an explicit type
so it's an ordinary optional.

// ----------------------------------------

<!--
  - test: `implicitlyUnwrappedOptionals`
```

[swift_028] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Memory Safety

Sumber: ht

Memory Safety
In addition to the checks that prevent type mismatches,
described above in ,
Swift also protects code against working with invalid memory.
This protection is known as *memory safety*
and includes the following requirements:
- Values are set before being read.
The protection against interacting with uninitialized regions of memory
is also known as *definite initialization*.
- Arrays and buffers are accessed only at valid indexes.
The protection against out-of-bounds access
is also known as *bounds safety*.
- Memory is accessed only during the value's lifetime.
The protection against use-after-free errors
is also known as *lifetime safety*.
- Access to memory overlaps only in provably safe ways.
The protection against possible data races in concurrent code
is also known as *thread safety*.

If you've worked in languages that don't provide these guarantees,
you may be familiar with some of the errors and bugs
named in the list above.
If you haven't encountered these issues, that's ok;
safe code in Swift avoids these problems.
For information about how Swift ensures you set initial values,
see ,
for information about how Swift checks memory safety in concurrent code,
see ,
and for information about how Swift checks overlapping accesses to memory,
see .
Sometimes you need to work outside of the bounds of safety ---
for example, because of limitations of the language or standard library ---
so Swift also provides unsafe versions of some APIs.
When you use types or methods whose name includes words such as
"unsafe", "unchecked", or "unmanaged",
you take on the responsibility for safety.
Safe code in Swift can still encounter errors and unexpected failures,
which might stop the program's execution.
Safety doesn't ensure that your code runs to completion.
Swift p

[swift_029] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Error Handling

Sumber: ht

Error Handling
You use *error handling* to respond to error conditions
your program may encounter during execution.
In contrast to optionals,
which can use the presence or absence of a value
to communicate success or failure of a function,
error handling allows you to determine the underlying cause of failure,
and, if necessary, propagate the error to another part of your program.
When a function encounters an error condition, it *throws* an error.
That function's caller can then *catch* the error and respond appropriately.
A function indicates that it can throw an error
by including the `throws` keyword in its declaration.
When you call a function that can throw an error,
you prepend the `try` keyword to the expression.
Swift automatically propagates errors out of their current scope
until they're handled by a `catch` clause.
A `do` statement creates a new containing scope,
which allows errors to be propagated to one or more `catch` clauses.
Here's an example of how error handling can be used
to respond to different error conditions:
In this example, the `makeASandwich()` function will throw an error
if no clean dishes are available
or if any ingredients are missing.
Because `makeASandwich()` can throw an error,
the function call is wrapped in a `try` expression.

By wrapping the function call in a `do` statement,
any errors that are thrown will be propagated
to the provided `catch` clauses.
If no error is thrown, the `eatASandwich()` function is called.
If an error is thrown and it matches the `SandwichError.outOfCleanDishes` case,
then the `washDishes()` function will be called.
If an error is thrown and it matches the `SandwichError.missingIngredients` case,
then the `buyGroceries(_:)` function is called
with the associated `[String]` value captured by the `catch`

Contoh Kode / Perintah CLI:

```
func canThrowAnError() throws {
    // this function may or may not throw an error
}

// ----------------------------------------

-->

A function indicates that it can throw an error
by including the `throws` keyword in its declaration.
When you call a function that can throw an error,
you prepend the `try` keyword to the expression.

Swift automatically propagates errors out of their current scope
until they're handled by a `catch` clause.

// ----------------------------------------

<!--
  - test: `errorHandling`
```

[swift_030] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Assertions and Preconditions

Sumber: ht

Assertions and Preconditions
Assertions and *preconditions*
are checks that happen at runtime.
You use them to make sure an essential condition is satisfied
before executing any further code.
If the Boolean condition in the assertion or precondition
evaluates to `true`,
code execution continues as usual.
If the condition evaluates to `false`,
the current state of the program is invalid;
code execution ends, and your app is terminated.
You use assertions and preconditions
to express the assumptions you make
and the expectations you have
while coding,
so you can include them as part of your code.
Assertions help you find mistakes and incorrect assumptions during development,
and preconditions help you detect issues in production.
In addition to verifying your expectations at runtime,

assertions and preconditions also become a useful form of documentation
within the code.
Unlike the error conditions discussed in above,
assertions and preconditions aren't used
for recoverable or expected errors.
Because a failed assertion or precondition
indicates an invalid program state,
there's no way to catch a failed assertion.
Recovering from an invalid state is impossible.
When an assertion fails,
at least one piece of the program's data is invalid ---
but you don't know why it's invalid
or whether an additional state is also invalid.
Using assertions and preconditions
isn't a substitute for designing your code in such a way
that invalid conditions are unlikely to arise.
However,
using them to enforce valid data and state
causes your app to terminate more predictably
if an invalid state occurs,
and helps make the problem easier to debug.
When assumptions aren't checked,
you might not notice this kind problem until much later
when code elsewhere starts failing visibly,
and after

[swift_031] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Debugging with Assertions

Sumber: ht

Debugging with Assertions
You write an assertion by calling the
`assert(_:_:file:line:)`) function
from the Swift standard library.
You pass this function an expression that evaluates to `true` or `false`
and a message to display if the result of the condition is `false`.
For example:
In this example, code execution continues if `age >= 0` evaluates to `true`,
that is, if the value of `age` is nonnegative.
If the value of `age` is negative, as in the code above,
then `age >= 0` evaluates to `false`,
and the assertion fails, terminating the application.
You can omit the assertion message ---
for example, when it would just repeat the condition as prose.
If the code already checks the condition,
you use the
`assertionFailure(_:file:line:)`) function
to indicate that an assertion has failed.
For example:

Contoh Kode / Perintah CLI:

```
let age = -3
```

```
assert(age >= 0, "A person's age can't be less than zero.")
// This assertion fails because -3 isn't >= 0.

// ----------------------------------------

-->

In this example, code execution continues if `age >= 0` evaluates to `true`,
that is, if the value of `age` is nonnegative.
If the value of `age` is negative, as in the code above,
then `age >= 0` evaluates to `false`,
and the assertion fails, terminating the application.

You can omit the assertion message ---
for example, when it would just repeat the condition as prose.

// ----------------------------------------

<!--
  - test: `assertions-2`
```

[swift_032] Swift - The Basics & Optionals

The Basics (Constants let, Variables var, Type Annotations, Optionals ?/!, Nil-Coalescing): Enforcing Preconditions

Sumber: ht

Enforcing Preconditions
Use a precondition whenever a condition has the potential to be false,
but must *definitely* be true for your code to continue execution.
For example, use a precondition to check that a subscript isn't out of bounds,
or to check that a function has been passed a valid value.
You write a precondition by calling the
`precondition(_:_:file:line:)`) function.
You pass this function an expression that evaluates to `true` or `false`
and a message to display if the result of the condition is `false`.
For example:
You can also call the
`preconditionFailure(_:file:line:)`) function
to indicate that a failure has occurred ---
for example, if the default case of a switch was taken,
but all valid input data should have been handled
by one of the switch's other cases.
> Note: If you compile in unchecked mode (`-Ounchecked`),
> preconditions aren't checked.
> The compiler assumes that preconditions are always true,
> and it optimizes your code accordingly.
> However, the `fatalError(_:file:line:)` function always halts execution,
> regardless of optimization settings.
>
> You can use the `fatalError(_:file:line:)` function
> during prototyping and early development
> to create stubs for functionality that hasn't been implemented yet,
> by writing `fatalError("Unimplemented")` as the stub implementation.
> Because fatal errors are never optimized out,
> unlike assertions or preconditions,

> you can be sure that execution always halts
> if it encounters a stub implementation.

Contoh Kode / Perintah CLI:

```
// In the implementation of a subscript...
precondition(index > 0, "Index must be greater than zero.")
```

[swift_033] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Overview

Sumber: ht

Control Flow
Structure code with branches, loops, and early exits.
Swift provides a variety of control flow statements.
These include `while` loops to perform a task multiple times;
`if`, `guard`, and `switch` statements
to execute different branches of code based on certain conditions;
and statements such as `break` and `continue`
to transfer the flow of execution to another point in your code.
Swift provides a `for`-`in` loop that makes it easy to iterate over
arrays, dictionaries, ranges, strings, and other sequences.
Swift also provides `defer` statements,
which wrap code to be executed when leaving the current scope.
Swift's `switch` statement is considerably more powerful
than its counterpart in many C-like languages.
Cases can match many different patterns,
including interval matches, tuples, and casts to a specific type.
Matched values in a `switch` case can be bound to temporary constants or variables
for use within the case's body,
and complex matching conditions can be expressed with a `where` clause for each case.

[swift_034] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): For-In Loops

Sumber: ht

For-In Loops
You use the `for`-`in` loop to iterate over a sequence,
such as items in an array, ranges of numbers, or characters in a string.
This example uses a `for`-`in` loop to iterate over the items in an array:
You can also iterate over a dictionary to access its key-value pairs.
Each item in the dictionary is returned as a `(key, value)` tuple
when the dictionary is iterated,
and you can decompose the `(key, value)` tuple's members as explicitly named constants
for use within the body of the `for`-`in` loop.
In the code example below, the dictionary's keys are decomposed into a constant called `animalName`,
and the dictionary's values are decomposed into a constant called `legCount`.
The contents of a `Dictionary` are inherently unordered,
and iterating over them doesn't guarantee the order
in which they will be retrieved.
In particular,
the order you insert items into a `Dictionary`
doesn't define the order they're iterated.

For more about arrays and dictionaries, see .
You can also use `for`-`in` loops with numeric ranges.
This example prints the first few entries in a five-times table:
The sequence being iterated over is
a range of numbers from `1` to `5`, inclusive,
as indicated by the use of the closed range operator (`...`).
The value of `index` is set to the first number in the range (`1`),
and the statements inside the loop are executed.
In this case, the loop contains only one statement,
which prints an entry from the five-times table for the current value of `index`.
After the statement is executed,
the value of `index` is updated to contain the second value in the range (`2`),
and the `print(_:separator:terminator:)` function is called again.
This process continues until the end of the range is reached.
In the example above, `index` is a constant wh

Contoh Kode / Perintah CLI:

```
let names = ["Anna", "Alex", "Brian", "Jack"]
for name in names {
    print("Hello, \(name)!")
}
// Hello, Anna!
// Hello, Alex!
// Hello, Brian!
// Hello, Jack!

// ----------------------------------------

-->

You can also iterate over a dictionary to access its key-value pairs.
Each item in the dictionary is returned as a `(key, value)` tuple
when the dictionary is iterated,
and you can decompose the `(key, value)` tuple's members as explicitly named constants
for use within the body of the `for`-`in` loop.
In the code example below, the dictionary's keys are decomposed into a constant called `animalName`,
and the dictionary's values are decomposed into a constant called `legCount`.

// ----------------------------------------

<!--
  - test: `forLoops`
```

[swift_035] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): While Loops

Sumber: ht

While Loops
A `while` loop performs a set of statements until a condition becomes `false`.
These kinds of loops are best used when
the number of iterations isn't known before the first iteration begins.
Swift provides two kinds of `while` loops:
- `while` evaluates its condition at the start of each pass through the loop.
- `repeat`-`while` evaluates its condition at the end of each pass through the loop.

[swift_036] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): While

Sumber: ht

While
A `while` loop starts by evaluating a single condition.
If the condition is `true`,
a set of statements is repeated until the condition becomes `false`.
Here's the general form of a `while` loop:
This example plays a simple game of *Snakes and Ladders*
(also known as *Chutes and Ladders*):

The rules of the game are as follows:
- The board has 25 squares, and the aim is to land on or beyond square 25.
- The player's starting square is "square zero",
which is just off the bottom-left corner of the board.
- Each turn, you roll a six-sided dice and move by that number of squares,
following the horizontal path indicated by the dotted arrow above.
- If your turn ends at the bottom of a ladder, you move up that ladder.
- If your turn ends at the head of a snake, you move down that snake.
The game board is represented by an array of `Int` values.
Its size is based on a constant called `finalSquare`,
which is used to initialize the array
and also to check for a win condition later in the example.
Because the players start off the board, on "square zero",
the board is initialized with 26 zero `Int` values, not 25.
Some squares are then set to have more specific values for the snakes and ladders.
Squares with a ladder base have a positive number to move you up the board,
whereas squares with a snake head have a negative number to move you back down the board.
Square 3 contains the bottom of a ladder that moves you up to square 11.
To represent this, `board[03]` is equal to `+08`,
which is equivalent to an integer value of `8`
(the difference between `3` and `11`).
To align the values and statements,
the unary plus operator (`+i`) is explicitly used with
the unary minus operator (`-i`)
and numbers lower than `10` are padded with zeros.
(Neither styli

Contoh Kode / Perintah CLI:

```
while <#condition#> {
   <#statements#>
}

// ----------------------------------------

let finalSquare = 25
var board = [Int](repeating: 0, count: finalSquare + 1)

// ----------------------------------------

-->

Some squares are then set to have more specific values for the snakes and ladders.
Squares with a ladder base have a positive number to move you up the board,
whereas squares with a snake head have a negative number to move you back down the board.
```

[swift_037] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Repeat-While

Sumber: ht

Repeat-While
The other variation of the `while` loop,
known as the `repeat`-`while` loop,
performs a single pass through the loop block first,
before considering the loop's condition.
It then continues to repeat the loop until the condition is `false`.
> Note: The `repeat`-`while` loop in Swift is analogous to
> a `do`-`while` loop in other languages.
Here's the general form of a `repeat`-`while` loop:
Here's the *Snakes and Ladders* example again,
written as a `repeat`-`while` loop rather than a `while` loop.
The values of `finalSquare`, `board`, `square`, and `diceRoll`
are initialized in exactly the same way as with a `while` loop.
In this version of the game,
the *first* action in the loop is to check for a ladder or a snake.
No ladder on the board takes the player straight to square 25,
and so it isn't possible to win the game by moving up a ladder.
Therefore, it's safe to check for a snake or a ladder as the first action in the loop.
At the start of the game, the player is on "square zero".
`board[0]` always equals `0` and has no effect.
After the code checks for snakes and ladders,
the dice is rolled and the player is moved forward by `diceRoll` squares.
The current loop execution then ends.
The loop's condition (`while square < finalSquare`) is the same as before,
but this time it's not evaluated until the *end* of the first run through the loop.
The structure of the `repeat`-`while` loop is better suited to this game
than the `while` loop in the previous example.
In the `repeat`-`while` loop above,
`square += board[square]` is always executed *immediately after*
the loop's `while` condition confirms that `square` is still on the board.
This behavior removes the need for the array bounds check
seen in the `while` loop version of the game described earlier.

Contoh Kode / Perintah CLI:

```
repeat {
   <#statements#>
} while <#condition#>

// ----------------------------------------

let finalSquare = 25
var board = [Int](repeating: 0, count: finalSquare + 1)
board[03] = +08; board[06] = +11; board[09] = +09; board[10] = +02
board[14] = -10; board[19] = -11; board[22] = -02; board[24] = -08
var square = 0
var diceRoll = 0

// ----------------------------------------

-->
```

```
In this version of the game,
the *first* action in the loop is to check for a ladder or a snake.
No ladder on the board takes the player straight to square 25,
and so it isn't possible to win the game by moving up a ladder.
Therefore, it's safe to check for a snake or a ladder as the first action in the loop.

At the start of the game, the player is on "square zero".
`board[0]` always equals `0` and has no effect.
```

[swift_038] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Conditional Statements

Sumber: ht

Conditional Statements
It's often useful to execute different pieces of code based on certain conditions.
You might want to run an extra piece of code when an error occurs,
or to display a message when a value becomes too high or too low.
To do this, you make parts of your code *conditional*.
Swift provides two ways to add conditional branches to your code:
the `if` statement and the `switch` statement.
Typically, you use the `if` statement
to evaluate simple conditions with only a few possible outcomes.
The `switch` statement is better suited to
more complex conditions with multiple possible permutations
and is useful in situations where pattern matching can help select
an appropriate code branch to execute.

[swift_039] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): If

Sumber: ht

If
In its simplest form,
the `if` statement has a single `if` condition.
It executes a set of statements only if that condition is `true`.
The example above checks whether the temperature
is less than or equal to 32 degrees Fahrenheit
(the freezing point of water).
If it is, a message is printed.
Otherwise, no message is printed,
and code execution continues after the `if` statement's closing brace.
The `if` statement can provide an alternative set of statements,
known as an *else clause*,
for situations when the `if` condition is `false`.
These statements are indicated by the `else` keyword.
One of these two branches is always executed.
Because the temperature has increased to `40` degrees Fahrenheit,
it's no longer cold enough to advise wearing a scarf
and so the `else` branch is triggered instead.
You can chain multiple `if` statements together
to consider additional clauses.
Here, an additional `if` statement was added to respond to particularly warm temperatures.

The final `else` clause remains,
and it prints a response for any temperatures that aren't too warm or too cold.
The final `else` clause is optional, however,
and can be excluded if the set of conditions doesn't need to be complete.
Because the temperature isn't cold enough to trigger the `if` condition
or warm enough to trigger the `else if` condition,
no message is printed.
Swift provides a shorthand spelling of `if`
that you can use when setting values.
For example,
consider the following code:
Here, each of the branches sets a value for the `weatherAdvice` constant,
which is printed after the `if` statement.
Using the alternate syntax,
known as an `if` expression,
you can write this code more concisely:
In this `if` expression version,
each branch contains a single value.
If a branch's condition is

Contoh Kode / Perintah CLI:

```
var temperatureInFahrenheit = 30
if temperatureInFahrenheit <= 32 {
    print("It's very cold. Consider wearing a scarf.")
}
// Prints "It's very cold. Consider wearing a scarf."

// ----------------------------------------

-->

The example above checks whether the temperature
is less than or equal to 32 degrees Fahrenheit
(the freezing point of water).
If it is, a message is printed.
Otherwise, no message is printed,
and code execution continues after the `if` statement's closing brace.

The `if` statement can provide an alternative set of statements,
known as an *else clause*,
for situations when the `if` condition is `false`.
These statements are indicated by the `else` keyword.

// ----------------------------------------

<!--
  - test: `ifElse`
```

[swift_040] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Switch

Sumber: ht

Switch
A `switch` statement considers a value
and compares it against several possible matching patterns.
It then executes an appropriate block of code,
based on the first pattern that matches successfully.

A `switch` statement provides an alternative to the `if` statement
for responding to multiple potential states.
In its simplest form, a `switch` statement compares a value against
one or more values of the same type.
Every `switch` statement consists of multiple possible *cases*,
each of which begins with the `case` keyword.
In addition to comparing against specific values,
Swift provides several ways for each case to specify
more complex matching patterns.
These options are described later in this chapter.
Like the body of an `if` statement, each `case` is a separate branch of code execution.
The `switch` statement determines which branch should be selected.
This procedure is known as *switching* on the value that's being considered.
Every `switch` statement must be *exhaustive*.
That is, every possible value of the type being considered
must be matched by one of the `switch` cases.
If it's not appropriate to provide a case for every possible value,
you can define a default case to cover any values that aren't addressed explicitly.
This default case is indicated by the `default` keyword,
and must always appear last.
This example uses a `switch` statement to consider
a single lowercase character called `someCharacter`:
The `switch` statement's first case matches
the first letter of the English alphabet, `a`,
and its second case matches the last letter, `z`.
Because the `switch` must have a case for every possible character,
not just every alphabetic character,
this `switch` statement uses a `default` case
to match all characters other than `a` and `z`.
This provi

Contoh Kode / Perintah CLI:

```
switch <#some value to consider#> {
case <#value 1#>:
    <#respond to value 1#>
case <#value 2#>,
    <#value 3#>:
    <#respond to value 2 or 3#>
default:
    <#otherwise, do something else#>
}

// ----------------------------------------

let someCharacter: Character = "z"
switch someCharacter {
case "a":
    print("The first letter of the Latin alphabet")
case "z":
    print("The last letter of the Latin alphabet")
default:
    print("Some other character")
}
// Prints "The last letter of the Latin alphabet".

// ----------------------------------------
```

```
-->

The `switch` statement's first case matches
the first letter of the English alphabet, `a`,
and its second case matches the last letter, `z`.
Because the `switch` must have a case for every possible character,
not just every alphabetic character,
this `switch` statement uses a `default` case
to match all characters other than `a` and `z`.
This provision ensures that the `switch` statement is exhaustive.

Like `if` statements,
`switch` statements also have an expression form:
```

[swift_041] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Patterns

Sumber: ht

Patterns
In the previous examples, each switch case includes a pattern
that indicates what values match that case.
You can also use a pattern as the condition for an `if` statement.
Here's what that looks like:
In this code,
the condition for the `if` statement starts with `case`,
indicating that the condition is a pattern instead of a Boolean value.
If the pattern matches,
then the condition for the `if` is considered to be true,
and so the code in the body of the `if` statement runs.
The patterns you can write after `if case`
are the same as the patterns you can write in a switch case.
In a `for`-`in` loop,
you can give names to parts of the value using a value binding pattern,
even without writing `case` in your code:
The `for`-`in` loop above iterates over an array of tuples,
binding the first and second elements of the tuples
to the `x` and `y` constants.
The statements inside the loop can use those constants,
such as the `if` statement that checks whether the point lies on the x-axis.
A more concise way to write this code
is to combine the value bindings and condition
using a `for`-`case`-`in` loop.
The code below has the same behavior as the `for`-`in` loop above:
In this code,
the condition is integrated into the `for`-`case`-`in` loop
as part of the pattern.
The statements in the `for`-`case`-`in` loop run only for points on the x-axis.
This code produces the same result as the `for`-`in` loop above,
but is a more compact way to iterate
over only certain elements in a collection.
A `for`-`case`-`in` loop can also include a `where` clause,
to check for an additional condition.
The statements inside the loop run
only when `where` clause matches the current element.

For example:
This code binds the first and second elements of the tuple
to `x` and `y` as consta

Contoh Kode / Perintah CLI:

```
let somePoint = (12, 100)
if case (let x, 100) = somePoint {
    print("Found a point on the y=100 line, at \(x)")
}
// Prints "Found a point on the y=100 line, at 12".

// ----------------------------------------

let points = [(10, 0), (30, -30), (-20, 0)]

for (x, y) in points {
    if y == 0 {
        print("Found a point on the x-axis at \(x)")
    }
}
// Prints "Found a point on the x-axis at 10".
// Prints "Found a point on the x-axis at -20".

// ----------------------------------------

for case (let x, 0) in points {
    print("Found a point on the x-axis at \(x)")
}
// Prints "Found a point on the x-axis at 10".
// Prints "Found a point on the x-axis at -20".
```

[swift_042] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Control Transfer Statements

Sumber: ht

Control Transfer Statements
Control transfer statements change the order in which your code is executed,
by transferring control from one piece of code to another.
Swift has five control transfer statements:
- `continue`
- `break`
- `fallthrough`
- `return`
- `throw`
The `continue`, `break`, and `fallthrough` statements are described below.
The `return` statement is described in ,
and the `throw` statement is described in .

[swift_043] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Continue

Sumber: ht

Continue
The `continue` statement tells a loop to stop what it's doing
and start again at the beginning of the next iteration through the loop.
It says "I am done with the current loop iteration"
without leaving the loop altogether.
The following example removes all vowels and spaces from a lowercase string
to create a cryptic puzzle phrase:
The code above calls the `continue` keyword whenever it matches a vowel or a space,
causing the current iteration of the loop to end immediately
and to jump straight to the start of the next iteration.

Contoh Kode / Perintah CLI:

```
let puzzleInput = "great minds think alike"
var puzzleOutput = ""
let charactersToRemove: [Character] = ["a", "e", "i", "o", "u", " "]
for character in puzzleInput {
    if charactersToRemove.contains(character) {
        continue
    }
    puzzleOutput.append(character)
}
print(puzzleOutput)
// Prints "grtmndsthnklk".
```

[swift_044] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Break

Sumber: ht

Break
The `break` statement ends execution of an entire control flow statement immediately.
The `break` statement can be used inside a `switch` or loop statement
when you want to terminate the execution of the `switch` or loop statement
earlier than would otherwise be the case.
Break in a Loop Statement
When used inside a loop statement,
`break` ends the loop's execution immediately
and transfers control to the code after the loop's closing brace (`}`).
No further code from the current iteration of the loop is executed,
and no further iterations of the loop are started.
Break in a Switch Statement
When used inside a `switch` statement,
`break` causes the `switch` statement to end its execution immediately
and to transfer control to the code after
the `switch` statement's closing brace (`}`).
This behavior can be used to match and ignore one or more cases in a `switch` statement.
Because Swift's `switch` statement is exhaustive
and doesn't allow empty cases,
it's sometimes necessary to deliberately match and ignore a case
in order to make your intentions explicit.
You do this by writing the `break` statement as the entire body of the case you want to ignore.
When that case is matched by the `switch` statement,

the `break` statement inside the case ends the `switch` statement's execution immediately.
> Note: A `switch` case that contains only a comment is reported as a compile-time error.
> Comments aren't statements and don't cause a `switch` case to be ignored.
> Always use a `break` statement to ignore a `switch` case.
The following example switches on a `Character` value
and determines whether it represents a number symbol in one of four languages.
For brevity, multiple values are covered in a single `switch` case.
This example checks `numberSymbol` to d

Contoh Kode / Perintah CLI:

```
let numberSymbol: Character = "?"  // Chinese symbol for the number 3
var possibleIntegerValue: Int?
switch numberSymbol {
case "1", "?", "?", "?":
    possibleIntegerValue = 1
case "2", "?", "?", "?":
    possibleIntegerValue = 2
case "3", "?", "?", "?":
    possibleIntegerValue = 3
case "4", "?", "?", "?":
    possibleIntegerValue = 4
default:
    break
}
if let integerValue = possibleIntegerValue {
    print("The integer value of \(numberSymbol) is \(integerValue).")
} else {
    print("An integer value couldn't be found for \(numberSymbol).")
}
// Prints "The integer value of ? is 3."
```

[swift_045] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Fallthrough

Sumber: ht

Fallthrough
In Swift, `switch` statements don't fall through the bottom of each case and into the next one.
That is, the entire `switch` statement completes its execution as soon as the first matching case is completed.
By contrast, C requires you to insert an explicit `break` statement
at the end of every `switch` case to prevent fallthrough.
Avoiding default fallthrough means that Swift `switch` statements are
much more concise and predictable than their counterparts in C,
and thus they avoid executing multiple `switch` cases by mistake.
If you need C-style fallthrough behavior,
you can opt in to this behavior on a case-by-case basis with the `fallthrough` keyword.
The example below uses `fallthrough` to create a textual description of a number.
This example declares a new `String` variable called `description`
and assigns it an initial value.
The function then considers the value of `integerToDescribe` using a `switch` statement.
If the value of `integerToDescribe` is one of the prime numbers in the list,
the function appends text to the end of `description`,
to note that the number is prime.
It then uses the `fallthrough` keyword to "fall into" the `default` case as well.
The `default` case adds some extra text to the end of the description,
and the `switch` statement is complete.
Unless the value of `integerToDescribe` is in the list of known prime numbers,

it isn't matched by the first `switch` case at all.
Because there are no other specific cases,
`integerToDescribe` is matched by the `default` case.
After the `switch` statement has finished executing,
the number's description is printed using the `print(_:separator:terminator:)` function.
In this example,
the number `5` is correctly identified as a prime number.
> Note: The `fallthrough` keyword doesn't che

Contoh Kode / Perintah CLI:

```
let integerToDescribe = 5
var description = "The number \(integerToDescribe) is"
switch integerToDescribe {
case 2, 3, 5, 7, 11, 13, 17, 19:
    description += " a prime number, and also"
    fallthrough
default:
    description += " an integer."
}
print(description)
// Prints "The number 5 is a prime number, and also an integer."
```

[swift_046] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Labeled Statements

Sumber: ht

Labeled Statements
In Swift, you can nest loops and conditional statements
inside other loops and conditional statements
to create complex control flow structures.
However, loops and conditional statements can both use the `break` statement
to end their execution prematurely.
Therefore, it's sometimes useful to be explicit about
which loop or conditional statement you want a `break` statement to terminate.
Similarly, if you have multiple nested loops,
it can be useful to be explicit about which loop the `continue` statement
should affect.
To achieve these aims,
you can mark a loop statement or conditional statement with a *statement label*.
With a conditional statement,
you can use a statement label with the `break` statement
to end the execution of the labeled statement.
With a loop statement,
you can use a statement label with the `break` or `continue` statement
to end or continue the execution of the labeled statement.
A labeled statement is indicated by placing
a label on the same line as the statement's introducer keyword, followed by a colon.
Here's an example of this syntax for a `while` loop,
although the principle is the same for all loops and `switch` statements:
The following example uses the `break` and `continue` statements
with a labeled `while` loop for an adapted version of the *Snakes and Ladders* game
that you saw earlier in this chapter.
This time around, the game has an extra rule:

- To win, you must land *exactly* on square 25.
If a particular dice roll would take you beyond square 25,
you must roll again until you roll the exact number needed to land on square 25.
The game board is the same as before.

The values of `finalSquare`, `board`, `square`, and `diceRoll`
are initialized in the same way as before:
This version of t

Contoh Kode / Perintah CLI:

```
<#label name#>: while <#condition#> {
   <#statements#>
}

// ----------------------------------------

let finalSquare = 25
var board = [Int](repeating: 0, count: finalSquare + 1)
board[03] = +08; board[06] = +11; board[09] = +09; board[10] = +02
board[14] = -10; board[19] = -11; board[22] = -02; board[24] = -08
var square = 0
var diceRoll = 0

// ----------------------------------------

-->

This version of the game uses a `while` loop and a `switch` statement
to implement the game's logic.
The `while` loop has a statement label called `gameLoop`
to indicate that it's the main game loop for the Snakes and Ladders game.

The `while` loop's condition is `while square != finalSquare`,
to reflect that you must land exactly on square 25.
```

[swift_047] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Early Exit

Sumber: ht

Early Exit
A `guard` statement, like an `if` statement,
executes statements depending on the Boolean value of an expression.
You use a `guard` statement to require that a condition must be true
in order for the code after the `guard` statement to be executed.
Unlike an `if` statement,
a `guard` statement always has an `else` clause ---
the code inside the `else` clause is executed if the condition isn't true.
If the `guard` statement's condition is met,
code execution continues after the `guard` statement's closing brace.
Any variables or constants that were assigned values
using an optional binding as part of the condition
are available for the rest of the code block
that the `guard` statement appears in.
If that condition isn't met,
the code inside the `else` branch is executed.
That branch must transfer control to exit the code block
in which the `guard` statement appears.

It can do this with a control transfer statement
such as `return`, `break`, `continue`, or `throw`,
or it can call a function or method
that doesn't return, such as `fatalError(_:file:line:)`.
Using a `guard` statement for requirements
improves the readability of your code,
compared to doing the same check with an `if` statement.
It lets you write the code that's typically executed
without wrapping it in an `else` block,
and it lets you keep the code that handles a violated requirement
next to the requirement.

Contoh Kode / Perintah CLI:

```
func greet(person: [String: String]) {
    guard let name = person["name"] else {
        return
    }

    print("Hello \(name)!")

    guard let location = person["location"] else {
        print("I hope the weather is nice near you.")
        return
    }

    print("I hope the weather is nice in \(location).")
}

greet(person: ["name": "John"])
// Prints "Hello John!"
// Prints "I hope the weather is nice near you."
greet(person: ["name": "Jane", "location": "Cupertino"])
// Prints "Hello Jane!"
// Prints "I hope the weather is nice in Cupertino."
```

[swift_048] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Deferred Actions

Sumber: ht

Deferred Actions
Unlike control-flow constructs like `if` and `while`,
which let you control whether part of your code is executed
or how many times it gets executed,
`defer` controls *when* a piece of code is executed.
You use a `defer` block to write code that will be executed later,
when your program reaches the end of the current scope.
For example:
In the example above,
the code inside of the `defer` block is executed
before exiting the body of the `if` statement.
First, the code in the `if` statement runs,
which increments `score` by five.
Then, before exiting the `if` statement's scope,
the deferred code is run, which prints `score`.
The code inside of the `defer` always runs,

regardless of how the program exits that scope.
That includes code like an early exit from a function,
breaking out of a `for` loop,
or throwing an error.
This behavior makes `defer` useful for operations
where you need to guarantee a pair of actions happen ---
like manually allocating and freeing memory,
opening and closing low-level file descriptors,
and beginning and ending transactions in a database ---
because you can write both actions next to each other in your code.
For example,
the following code gives a temporary bonus to the score,
by adding and subtracting 100 inside a chunk of code:
If you write more than one `defer` block in the same scope,
the first one you specify is the last one to run.
If your program stops running ---
for example, because of a runtime error or a crash ---
deferred code doesn't execute.
However, deferred code does execute after an error is thrown;
for information about using `defer` with error handling,
see .

Contoh Kode / Perintah CLI:

```
var score = 1
if score < 10 {
    defer {
        print(score)
    }
    score += 5
}
// Prints "6".

// ----------------------------------------

-->

In the example above,
the code inside of the `defer` block is executed
before exiting the body of the `if` statement.
First, the code in the `if` statement runs,
which increments `score` by five.
Then, before exiting the `if` statement's scope,
the deferred code is run, which prints `score`.

The code inside of the `defer` always runs,
regardless of how the program exits that scope.
That includes code like an early exit from a function,
breaking out of a `for` loop,
or throwing an error.
This behavior makes `defer` useful for operations
where you need to guarantee a pair of actions happen ---
like manually allocating and freeing memory,
opening and closing low-level file descriptors,
and beginning and ending transactions in a database ---
because you can write both actions next to each other in your code.
For example,
the following code gives a temporary bonus to the score,
by adding and subtracting 100 inside a chunk of code:
```

```
// ----------------------------------------

<!--
  - test: `defer-paired-actions`
```

[swift_049] Swift - Control Flow & Pattern Matching

Control Flow (for-in, while, if, guard let, switch with tuples & where clauses): Checking API Availability

Sumber: ht

Checking API Availability
Swift has built-in support for checking API availability,
which ensures that you don't accidentally use APIs that are unavailable
on a given deployment target.
The compiler uses availability information in the SDK
to verify that all of the APIs used in your code
are available on the deployment target specified by your project.
Swift reports an error at compile time
if you try to use an API that isn't available.
You use an *availability condition* in an `if` or `guard` statement
to conditionally execute a block of code,
depending on whether the APIs you want to use are available at runtime.
The compiler uses the information from the availability condition
when it verifies that the APIs in that block of code are available.
The availability condition above specifies that in iOS,
the body of the `if` statement executes only in iOS 10 and later;
in macOS, only in macOS 10.12 and later.
The last argument, `*`, is required and specifies that on any other platform,
the body of the `if` executes on the minimum deployment target specified by your target.
In its general form,
the availability condition takes a list of platform names and versions.
You use platform names such as `iOS`, `macOS`, `watchOS`, `tvOS`, and `visionOS` ---
for the full list, see .
In addition to specifying major version numbers like iOS 8 or macOS 10.10,
you can specify minor versions numbers like iOS 11.2.6 and macOS 10.13.3.
When you use an availability condition with a `guard` statement,
it refines the availability information that's used
for the rest of the code in that code block.
In the example above,
the `ColorPreference` structure requires macOS 10.12 or later.
The `chooseBestColor()` function begins with an availability guard.
If the platform version is too old to use `

Contoh Kode / Perintah CLI:

```
if #available(iOS 10, macOS 10.12, *) {
    // Use iOS 10 APIs on iOS, and use macOS 10.12 APIs on macOS
} else {
    // Fall back to earlier iOS and macOS APIs
}

// ----------------------------------------

-->
```

```
The availability condition above specifies that in iOS,
the body of the `if` statement executes only in iOS 10 and later;
in macOS, only in macOS 10.12 and later.
The last argument, `*`, is required and specifies that on any other platform,
the body of the `if` executes on the minimum deployment target specified by your target.

In its general form,
the availability condition takes a list of platform names and versions.
You use platform names such as `iOS`, `macOS`, `watchOS`, `tvOS`, and `visionOS` ---
for the full list, see <doc:Attributes#Declaration-Attributes>.
In addition to specifying major version numbers like iOS 8 or macOS 10.10,
you can specify minor versions numbers like iOS 11.2.6 and macOS 10.13.3.

// ----------------------------------------

When you use an availability condition with a `guard` statement,
it refines the availability information that's used
for the rest of the code in that code block.
```

[swift_050] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Overview

Sumber: ht

Functions
Define and call functions, label their arguments, and use their return values.
Functions are self-contained chunks of code that perform a specific task.
You give a function a name that identifies what it does,
and this name is used to "call" the function to perform its task when needed.
Swift's unified function syntax is flexible enough to express anything from
a simple C-style function with no parameter names
to a complex Objective-C-style method
with names and argument labels for each parameter.
Parameters can provide default values to simplify function calls
and can be passed as in-out parameters,
which modify a passed variable once the function has completed its execution.
Every function in Swift has a type,
consisting of the function's parameter types and return type.
You can use this type like any other type in Swift,
which makes it easy to pass functions as parameters to other functions,
and to return functions from functions.
Functions can also be written within other functions
to encapsulate useful functionality within a nested function scope.

[swift_051] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Defining and Calling Functions

Sumber: ht

Defining and Calling Functions
When you define a function,
you can optionally define one or more named, typed values that the function takes as input,
known as *parameters*.
You can also optionally define
a type of value that the function will pass back as output when it's done,

known as its *return type*.
Every function has a *function name*,
which describes the task that the function performs.
To use a function, you "call" that function with its name
and pass it input values (known as *arguments*)
that match the types of the function's parameters.
A function's arguments must always be provided in the same order
as the function's parameter list.
The function in the example below is called `greet(person:)`,
because that's what it does ---
it takes a person's name as input and returns a greeting for that person.
To accomplish this, you define one input parameter ---
a `String` value called `person` ---
and a return type of `String`,
which will contain a greeting for that person:
All of this information is rolled up into the function's *definition*,
which is prefixed with the `func` keyword.
You indicate the function's return type with the *return arrow* `->`
(a hyphen followed by a right angle bracket),
which is followed by the name of the type to return.
The definition describes what the function does,
what it expects to receive,
and what it returns when it's done.
The definition makes it easy for the function to be called unambiguously
from elsewhere in your code:
You call the `greet(person:)` function
by passing it a `String` value after the `person` argument label,
such as `greet(person: "Anna")`.
Because the function returns a `String` value,
`greet(person:)` can be wrapped in a call to the `print(_:separator:terminator:)` function
to print that string and see it

Contoh Kode / Perintah CLI:

```
func greet(person: String) -> String {
    let greeting = "Hello, " + person + "!"
    return greeting
}

// ----------------------------------------

-->

All of this information is rolled up into the function's *definition*,
which is prefixed with the `func` keyword.
You indicate the function's return type with the *return arrow* `->`
(a hyphen followed by a right angle bracket),
which is followed by the name of the type to return.

The definition describes what the function does,
what it expects to receive,
and what it returns when it's done.
The definition makes it easy for the function to be called unambiguously
from elsewhere in your code:

// ----------------------------------------
```

```
<!--
  - test: `definingAndCalling`
```

[swift_052] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Function Parameters and Return Values

Sumber: ht

Function Parameters and Return Values
Function parameters and return values are extremely flexible in Swift.
You can define anything from a simple utility function with a single unnamed parameter
to a complex function with expressive parameter names and different parameter options.

[swift_053] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Functions Without Parameters

Sumber: ht

Functions Without Parameters
Functions aren't required to define input parameters.
Here's a function with no input parameters,
which always returns the same `String` message whenever it's called:
The function definition still needs parentheses after the function's name,
even though it doesn't take any parameters.
The function name is also followed by
an empty pair of parentheses when the function is called.

Contoh Kode / Perintah CLI:

```
func sayHelloWorld() -> String {
    return "hello, world"
}
print(sayHelloWorld())
// Prints "hello, world".
```

[swift_054] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Functions With Multiple Parameters

Sumber: ht

Functions With Multiple Parameters
Functions can have multiple input parameters,
which are written within the function's parentheses, separated by commas.
This function takes a person's name
and whether they have already been greeted as input,
and returns an appropriate greeting for that person:
You call the `greet(person:alreadyGreeted:)` function
by passing it both a `String` argument value labeled `person`
and a `Bool` argument value labeled `alreadyGreeted`
in parentheses, separated by commas.
Note that this function is distinct from the `greet(person:)` function
shown in an earlier section.
Although both functions have names that begin with `greet`,
the `greet(person:alreadyGreeted:)` function takes two arguments
but the `greet(person:)` function takes only one.

Contoh Kode / Perintah CLI:

```
func greet(person: String, alreadyGreeted: Bool) -> String {
    if alreadyGreeted {
        return greetAgain(person: person)
    } else {
        return greet(person: person)
    }
}
print(greet(person: "Tim", alreadyGreeted: true))
// Prints "Hello again, Tim!"
```

[swift_055] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Functions Without Return Values

Sumber: ht

Functions Without Return Values
Functions aren't required to define a return type.
Here's a version of the `greet(person:)` function,
which prints its own `String` value rather than returning it:
Because it doesn't need to return a value,
the function's definition doesn't include the return arrow (`->`)
or a return type.
> Note: Strictly speaking, this version of the `greet(person:)` function *does* still return a value,
> even though no return value is defined.
> Functions without a defined return type return a special value of type `Void`.
> This is simply an empty tuple,
> which is written as `()`.
The return value of a function can be ignored when it's called:
The first function, `printAndCount(string:)`,
prints a string, and then returns its character count as an `Int`.
The second function, `printWithoutCounting(string:)`,
calls the first function, but ignores its return value.
When the second function is called,
the message is still printed by the first function,
but the returned value isn't used.
> Note: Return values can be ignored,
> but a function that says it will return a value must always do so.
> A function with a defined return type
> can't allow control to fall out of the bottom of the function
> without returning a value,
> and attempting to do so will result in a compile-time error.

Contoh Kode / Perintah CLI:

```
func greet(person: String) {
    print("Hello, \(person)!")
}
greet(person: "Dave")
// Prints "Hello, Dave!"

// ----------------------------------------

-->

Because it doesn't need to return a value,
the function's definition doesn't include the return arrow (`->`)
```

```
or a return type.

> Note: Strictly speaking, this version of the `greet(person:)` function *does* still return a value,
> even though no return value is defined.
> Functions without a defined return type return a special value of type `Void`.
> This is simply an empty tuple,
> which is written as `()`.

The return value of a function can be ignored when it's called:

// ----------------------------------------

<!--
  - test: `functionsWithoutReturnValues`
```

[swift_056] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Functions with Multiple Return Values

Sumber: ht

Functions with Multiple Return Values
You can use a tuple type as the return type for a function
to return multiple values as part of one compound return value.
The example below defines a function called `minMax(array:)`,
which finds the smallest and largest numbers in an array of `Int` values:
The `minMax(array:)` function returns a tuple containing two `Int` values.
These values are labeled `min` and `max`
so that they can be accessed by name when querying the function's return value.
The body of the `minMax(array:)` function starts by setting
two working variables called `currentMin` and `currentMax`
to the value of the first integer in the array.
The function then iterates over the remaining values in the array
and checks each value to see if it's smaller or larger than
the values of `currentMin` and `currentMax` respectively.
Finally, the overall minimum and maximum values are returned as
a tuple of two `Int` values.
Because the tuple's member values are named as part of the function's return type,
they can be accessed with dot syntax to retrieve the minimum and maximum found values:
Note that the tuple's members don't need to be named
at the point that the tuple is returned from the function,
because their names are already specified as part of the function's return type.
Optional Tuple Return Types
If the tuple type to be returned from a function
has the potential to have "no value" for the entire tuple,
you can use an *optional* tuple return type to reflect the fact that
the entire tuple can be `nil`.
You write an optional tuple return type by placing a question mark
after the tuple type's closing parenthesis,
such as `(Int, Int)?` or `(String, Int, Bool)?`.
> Note: An optional tuple type such as `(Int, Int)?`
> is different from a tuple that contains

Contoh Kode / Perintah CLI:

```
func minMax(array: [Int]) -> (min: Int, max: Int) {
    var currentMin = array[0]
```

```
    var currentMax = array[0]
    for value in array[1..<array.count] {
        if value < currentMin {
            currentMin = value
        } else if value > currentMax {
            currentMax = value
        }
    }
    return (currentMin, currentMax)
}

// ----------------------------------------

-->

The `minMax(array:)` function returns a tuple containing two `Int` values.
These values are labeled `min` and `max`
so that they can be accessed by name when querying the function's return value.

The body of the `minMax(array:)` function starts by setting
two working variables called `currentMin` and `currentMax`
to the value of the first integer in the array.
The function then iterates over the remaining values in the array
and checks each value to see if it's smaller or larger than
the values of `currentMin` and `currentMax` respectively.
Finally, the overall minimum and maximum values are returned as
a tuple of two `Int` values.

Because the tuple's member values are named as part of the function's return type,
they can be accessed with dot syntax to retrieve the minimum and maximum found values:

// ----------------------------------------

<!--
  - test: `tupleTypesAsReturnTypes`
```

[swift_057] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Functions With an Implicit Return

Sumber: ht

Functions With an Implicit Return
If the entire body of the function is a single expression,
the function implicitly returns that expression.
For example,
both functions below have the same behavior:
The entire definition of the `greeting(for:)` function
is the greeting message that it returns,
which means it can use this shorter form.
The `anotherGreeting(for:)` function returns the same greeting message,
using the `return` keyword like a longer function.
Any function that you write as just one `return` line can omit the `return`.
As you'll see in ,
property getters can also use an implicit return.
> Note: The code you write as an implicit return value
> needs to return some value.
> For example,

> you can't use `print(13)`
> as an implicit return value.
> However, you can use a function that never returns
> like `fatalError("Oh no!")`
> as an implicit return value,
> because Swift knows that the implicit return doesn't happen.

Contoh Kode / Perintah CLI:

```
func greeting(for person: String) -> String {
    "Hello, " + person + "!"
}
print(greeting(for: "Dave"))
// Prints "Hello, Dave!"

func anotherGreeting(for person: String) -> String {
    return "Hello, " + person + "!"
}
print(anotherGreeting(for: "Dave"))
// Prints "Hello, Dave!"

// ----------------------------------------

-->

The entire definition of the `greeting(for:)` function
is the greeting message that it returns,
which means it can use this shorter form.
The `anotherGreeting(for:)` function returns the same greeting message,
using the `return` keyword like a longer function.
Any function that you write as just one `return` line can omit the `return`.

As you'll see in <doc:Properties#Shorthand-Getter-Declaration>,
property getters can also use an implicit return.

> Note: The code you write as an implicit return value
> needs to return some value.
> For example,
> you can't use `print(13)`
> as an implicit return value.
> However, you can use a function that never returns
> like `fatalError("Oh no!")`
> as an implicit return value,
> because Swift knows that the implicit return doesn't happen.

<!--
  - test: `implicit-return-print-instead`
```

[swift_058] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Function Argument Labels and Parameter Names

Sumber: ht

Function Argument Labels and Parameter Names
Each function parameter has both an *argument label*
and a *parameter name*.
The argument label is used when calling the function;
each argument is written in the function call with its argument label before it.
The parameter name is used in the implementation of the function.
By default, parameters

use their parameter name as their argument label.
All parameters must have unique names.
Although it's possible for multiple parameters
to have the same argument label,
unique argument labels help make your code more readable.

Contoh Kode / Perintah CLI:

```
func someFunction(firstParameterName: Int, secondParameterName: Int) {
    // In the function body, firstParameterName and secondParameterName
    // refer to the argument values for the first and second parameters.
}
someFunction(firstParameterName: 1, secondParameterName: 2)

// ----------------------------------------

-->

All parameters must have unique names.
Although it's possible for multiple parameters
to have the same argument label,
unique argument labels help make your code more readable.

<!--
  - test: `non-unique-external-name`
```

[swift_059] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Specifying Argument Labels

Sumber: ht

Specifying Argument Labels
You write an argument label before the parameter name,
separated by a space:
Here's a variation of the `greet(person:)` function
that takes a person's name and hometown
and returns a greeting:
The use of argument labels can allow a function
to be called in an expressive, sentence-like manner,
while still providing a function body that's readable and clear in intent.

Contoh Kode / Perintah CLI:

```
func someFunction(argumentLabel parameterName: Int) {
    // In the function body, parameterName refers to the argument value
    // for that parameter.
}

// ----------------------------------------

-->

Here's a variation of the `greet(person:)` function
that takes a person's name and hometown
and returns a greeting:

// ----------------------------------------

<!--
  - test: `externalParameterNames`
```

[swift_060] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Omitting Argument Labels

Sumber: ht

Omitting Argument Labels
If you don't want an argument label for a parameter,
write an underscore (`_`) instead of an explicit argument label for that parameter.
If a parameter has an argument label,
the argument *must* be labeled when you call the function.

Contoh Kode / Perintah CLI:

```
func someFunction(_ firstParameterName: Int, secondParameterName: Int) {
    // In the function body, firstParameterName and secondParameterName
    // refer to the argument values for the first and second parameters.
}
someFunction(1, secondParameterName: 2)
```

[swift_061] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Default Parameter Values

Sumber: ht

Default Parameter Values
You can define a *default value* for any parameter in a function
by assigning a value to the parameter after that parameter's type.
If a default value is defined, you can omit that parameter when calling the function.
Place parameters that don't have default values
at the beginning of a function's parameter list,
before the parameters that have default values.
Parameters that don't have default values
are usually more important to the function's meaning ---
writing them first makes it easier to recognize
that the same function is being called,
regardless of whether any default parameters are omitted.

Contoh Kode / Perintah CLI:

```
func someFunction(parameterWithoutDefault: Int, parameterWithDefault: Int = 12) {
    // If you omit the second argument when calling this function, then
    // the value of parameterWithDefault is 12 inside the function body.
}
someFunction(parameterWithoutDefault: 3, parameterWithDefault: 6) // parameterWithDefault is 6
someFunction(parameterWithoutDefault: 4) // parameterWithDefault is 12
```

[swift_062] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Variadic Parameters

Sumber: ht

Variadic Parameters
A *variadic parameter* accepts zero or more values of a specified type.
You use a variadic parameter to specify that the parameter can be passed
a varying number of input values when the function is called.
Write variadic parameters by inserting three period characters (`...`)
after the parameter's type name.

The values passed to a variadic parameter are made available within the function's body
as an array of the appropriate type.
For example, a variadic parameter with a name of `numbers` and a type of `Double...`
is made available within the function's body as
a constant array called `numbers` of type `[Double]`.
The example below calculates the *arithmetic mean*
(also known as the *average*) for a list of numbers of any length:
A function can have multiple variadic parameters.
The first parameter that comes after a variadic parameter
must have an argument label.
The argument label makes it unambiguous
which arguments are passed to the variadic parameter
and which arguments are passed to the parameters
that come after the variadic parameter.

Contoh Kode / Perintah CLI:

```
func arithmeticMean(_ numbers: Double...) -> Double {
    var total: Double = 0
    for number in numbers {
        total += number
    }
    return total / Double(numbers.count)
}
arithmeticMean(1, 2, 3, 4, 5)
// returns 3.0, which is the arithmetic mean of these five numbers
arithmeticMean(3, 8.25, 18.75)
// returns 10.0, which is the arithmetic mean of these three numbers

// ----------------------------------------

-->

<!--
  Rewrite the above to avoid bare expressions.
  Tracking bug is <rdar://problem/35301593>
-->

A function can have multiple variadic parameters.
The first parameter that comes after a variadic parameter
must have an argument label.
The argument label makes it unambiguous
which arguments are passed to the variadic parameter
and which arguments are passed to the parameters
that come after the variadic parameter.

<!--
  - test: `variadic-parameters-and-labels`

// ----------------------------------------

-->

<!--
  - test: `variadic-parameters-and-labels-failure`
```

[swift_063] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): In-Out Parameters

Sumber: ht

In-Out Parameters
Function parameters are constants by default.
Trying to change the value of a function parameter
from within the body of that function results in a compile-time error.
This means that you can't change the value of a parameter by mistake.
If you want a function to modify a parameter's value,
and you want those changes to persist after the function call has ended,
define that parameter as an *in-out parameter* instead.
You write an in-out parameter by placing the `inout` keyword
right before a parameter's type.
An in-out parameter has a value that's passed *in* to the function,
is modified by the function,
and is passed back *out* of the function to replace the original value.
For a detailed discussion of the behavior of in-out parameters
and associated compiler optimizations,
see .
You can only pass a variable as the argument for an in-out parameter.
You can't pass a constant or a literal value as the argument,
because constants and literals can't be modified.
You place an ampersand (`&`) directly before a variable's name
when you pass it as an argument to an in-out parameter,
to indicate that it can be modified by the function.
> Note: In-out parameters can't have default values,
> and variadic parameters can't be marked as `inout`.
Here's an example of a function called `swapTwoInts(_:_:)`,
which has two in-out integer parameters called `a` and `b`:
The `swapTwoInts(_:_:)` function simply swaps the value of `b` into `a`,
and the value of `a` into `b`.
The function performs this swap by storing the value of `a` in
a temporary constant called `temporaryA`, assigning the value of `b` to `a`,
and then assigning `temporaryA` to `b`.
You can call the `swapTwoInts(_:_:)` function with two variables of type `Int`
to swap their values.
Note that the names

Contoh Kode / Perintah CLI:

```
func swapTwoInts(_ a: inout Int, _ b: inout Int) {
    let temporaryA = a
    a = b
    b = temporaryA
}

// ----------------------------------------

-->

The `swapTwoInts(_:_:)` function simply swaps the value of `b` into `a`,
and the value of `a` into `b`.
The function performs this swap by storing the value of `a` in
a temporary constant called `temporaryA`, assigning the value of `b` to `a`,
```

```
and then assigning `temporaryA` to `b`.

You can call the `swapTwoInts(_:_:)` function with two variables of type `Int`
to swap their values.
Note that the names of `someInt` and `anotherInt` are prefixed with an ampersand
when they're passed to the `swapTwoInts(_:_:)` function:

// ----------------------------------------

<!--
  - test: `inoutParameters`
```

[swift_064] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Function Types

Sumber: ht

Function Types
Every function has a specific *function type*,
made up of the parameter types and the return type of the function.
For example:
This example defines two simple mathematical functions
called `addTwoInts` and `multiplyTwoInts`.
These functions each take two `Int` values,
and return an `Int` value, which is the result of
performing an appropriate mathematical operation.
The type of both of these functions is `(Int, Int) -> Int`.
This can be read as:
"A function that has two parameters, both of type `Int`,
and that returns a value of type `Int`."
Here's another example, for a function with no parameters or return value:
The type of this function is `() -> Void`,
or "a function that has no parameters, and returns `Void`."

Contoh Kode / Perintah CLI:

```
func addTwoInts(_ a: Int, _ b: Int) -> Int {
    return a + b
}
func multiplyTwoInts(_ a: Int, _ b: Int) -> Int {
    return a * b
}

// ----------------------------------------

-->

This example defines two simple mathematical functions
called `addTwoInts` and `multiplyTwoInts`.
These functions each take two `Int` values,
and return an `Int` value, which is the result of
performing an appropriate mathematical operation.

The type of both of these functions is `(Int, Int) -> Int`.
This can be read as:

"A function that has two parameters, both of type `Int`,
and that returns a value of type `Int`."
```

```
Here's another example, for a function with no parameters or return value:

// ----------------------------------------

<!--
  - test: `functionTypes`
```

[swift_065] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Using Function Types

Sumber: ht

Using Function Types
You use function types just like any other types in Swift.
For example, you can define a constant or variable to be of a function type
and assign an appropriate function to that variable:
This can be read as:
"Define a variable called `mathFunction`,
which has a type of 'a function that takes two `Int` values,
and returns an `Int` value.'
Set this new variable to refer to the function called `addTwoInts`."
The `addTwoInts(_:_:)` function has the same type as the `mathFunction` variable,
and so this assignment is allowed by Swift's type-checker.
You can now call the assigned function with the name `mathFunction`:
A different function with the same matching type can be assigned to the same variable,
in the same way as for nonfunction types:
As with any other type,
you can leave it to Swift to infer the function type
when you assign a function to a constant or variable:

Contoh Kode / Perintah CLI:

```
var mathFunction: (Int, Int) -> Int = addTwoInts

// ----------------------------------------

-->

This can be read as:

"Define a variable called `mathFunction`,
which has a type of 'a function that takes two `Int` values,
and returns an `Int` value.'
Set this new variable to refer to the function called `addTwoInts`."

The `addTwoInts(_:_:)` function has the same type as the `mathFunction` variable,
and so this assignment is allowed by Swift's type-checker.

You can now call the assigned function with the name `mathFunction`:

// ----------------------------------------

<!--
  - test: `functionTypes`
```

[swift_066] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Function Types as Parameter Types

Sumber: ht

Function Types as Parameter Types
You can use a function type such as `(Int, Int) -> Int`
as a parameter type for another function.
This enables you to leave some aspects of a function's implementation
for the function's caller to provide when the function is called.
Here's an example to print the results of the math functions from above:
This example defines a function called `printMathResult(_:_:_:)`, which has three parameters.
The first parameter is called `mathFunction`, and is of type `(Int, Int) -> Int`.
You can pass any function of that type as the argument for this first parameter.
The second and third parameters are called `a` and `b`, and are both of type `Int`.
These are used as the two input values for the provided math function.
When `printMathResult(_:_:_:)` is called,
it's passed the `addTwoInts(_:_:)` function, and the integer values `3` and `5`.
It calls the provided function with the values `3` and `5`, and prints the result of `8`.
The role of `printMathResult(_:_:_:)` is to print the result of
a call to a math function of an appropriate type.
It doesn't matter what that function's implementation actually does ---
it matters only that the function is of the correct type.
This enables `printMathResult(_:_:_:)` to hand off some of its functionality
to the caller of the function in a type-safe way.

Contoh Kode / Perintah CLI:

```
func printMathResult(_ mathFunction: (Int, Int) -> Int, _ a: Int, _ b: Int) {
    print("Result: \(mathFunction(a, b))")
}
printMathResult(addTwoInts, 3, 5)
// Prints "Result: 8".
```

[swift_067] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Function Types as Return Types

Sumber: ht

Function Types as Return Types
You can use a function type as the return type of another function.
You do this by writing a complete function type
immediately after the return arrow (`->`) of the returning function.
The next example defines two simple functions called `stepForward(_:)` and `stepBackward(_:)`.
The `stepForward(_:)` function returns a value one more than its input value,
and the `stepBackward(_:)` function returns a value one less than its input value.
Both functions have a type of `(Int) -> Int`:
Here's a function called `chooseStepFunction(backward:)`,
whose return type is `(Int) -> Int`.
The `chooseStepFunction(backward:)` function returns the `stepForward(_:)` function
or the `stepBackward(_:)` function based on a Boolean parameter called `backward`:
You can now use `chooseStepFunction(backward:)` to obtain a function
that will step in one direction or the other:
The example above determines whether a positive or negative step is needed
to move a variable called `currentValue` progressively closer to zero.

`currentValue` has an initial value of `3`,
which means that `currentValue > 0` returns `true`,
causing `chooseStepFunction(backward:)` to return the `stepBackward(_:)` function.
A reference to the returned function is stored in a constant called `moveNearerToZero`.
Now that `moveNearerToZero` refers to the correct function,
it can be used to count to zero:

Contoh Kode / Perintah CLI:

```
func stepForward(_ input: Int) -> Int {
    return input + 1
}
func stepBackward(_ input: Int) -> Int {
    return input - 1
}

// ----------------------------------------

-->

Here's a function called `chooseStepFunction(backward:)`,
whose return type is `(Int) -> Int`.
The `chooseStepFunction(backward:)` function returns the `stepForward(_:)` function
or the `stepBackward(_:)` function based on a Boolean parameter called `backward`:

// ----------------------------------------

<!--
  - test: `functionTypes`
```

[swift_068] Swift - Functions & Parameters

Functions (Argument Labels, Default Parameter Values, inout parameters, Function Types): Nested Functions

Sumber: ht

Nested Functions
All of the functions you have encountered so far in this chapter
have been examples of *global functions*, which are defined at a global scope.
You can also define functions inside the bodies of other functions,
known as *nested functions*.
Nested functions are hidden from the outside world by default,
but can still be called and used by their enclosing function.
An enclosing function can also return one of its nested functions
to allow the nested function to be used in another scope.
You can rewrite the `chooseStepFunction(backward:)` example above
to use and return nested functions:

Contoh Kode / Perintah CLI:

```
func chooseStepFunction(backward: Bool) -> (Int) -> Int {
    func stepForward(input: Int) -> Int { return input + 1 }
    func stepBackward(input: Int) -> Int { return input - 1 }
    return backward ? stepBackward : stepForward
}
var currentValue = -4
let moveNearerToZero = chooseStepFunction(backward: currentValue > 0)
// moveNearerToZero now refers to the nested stepForward() function
while currentValue != 0 {
    print("\(currentValue)... ")
    currentValue = moveNearerToZero(currentValue)
```

```
}
print("zero!")
// -4...
// -3...
// -2...
// -1...
// zero!
```

[swift_069] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Overview

Sumber: ht

Structures and Classes
Model custom types that encapsulate data.
Structures and *classes* are general-purpose,
flexible constructs that become the building blocks of your program's code.
You define properties and methods to add functionality to your structures and classes
using the same syntax you use to define constants, variables, and functions.
Unlike other programming languages,
Swift doesn't require you to create separate interface and implementation files
for custom structures and classes.
In Swift, you define a structure or class in a single file,
and the external interface to that class or structure is
automatically made available for other code to use.
> Note: An instance of a class is traditionally known as an *object*.
> However, Swift structures and classes
> are much closer in functionality than in other languages,
> and much of this chapter describes functionality that applies to
> instances of *either* a class or a structure type.
> Because of this, the more general term *instance* is used.

[swift_070] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Comparing Structures and Classes

Sumber: ht

Comparing Structures and Classes
Structures and classes in Swift have many things in common.
Both can:
- Define properties to store values
- Define methods to provide functionality
- Define subscripts to provide access to their values using subscript syntax
- Define initializers to set up their initial state
- Be extended to expand their functionality beyond a default implementation
- Conform to protocols to provide standard functionality of a certain kind
For more information, see
, , , ,
, and .
Classes have additional capabilities that structures don't have:
- Inheritance enables one class to inherit the characteristics of another.
- Type casting enables you to check and interpret the type of a class instance at runtime.
- Deinitializers enable an instance of a class to free up any resources it has assigned.

- Reference counting allows more than one reference to a class instance.
For more information, see
, , ,
and .
The additional capabilities that classes support
come at the cost of increased complexity.
As a general guideline,
prefer structures because they're easier to reason about,
and use classes when they're appropriate or necessary.
In practice, this means most of the custom types you define
will be structures and enumerations.
For a more detailed comparison,
see Choosing Between Structures and Classes.
> Note: Classes and actors share many of the same characteristics and behaviors.
> For information about actors, see .

[swift_071] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Definition Syntax

Sumber: ht

Definition Syntax
Structures and classes have a similar definition syntax.
You introduce structures with the `struct` keyword
and classes with the `class` keyword.
Both place their entire definition within a pair of braces:
> Note: Whenever you define a new structure or class,
> you define a new Swift type.
> Give types `UpperCamelCase` names
> (such as `SomeStructure` and `SomeClass` here)
> to match the capitalization of standard Swift types
> (such as `String`, `Int`, and `Bool`).
> Give properties and methods `lowerCamelCase` names
> (such as `frameRate` and `incrementCount`)
> to differentiate them from type names.
Here's an example of a structure definition and a class definition:
The example above defines a new structure called `Resolution`,
to describe a pixel-based display resolution.
This structure has two stored properties called `width` and `height`.
Stored properties are constants or variables that are bundled up and stored
as part of the structure or class.
These two properties are inferred to be of type `Int`
by setting them to an initial integer value of `0`.
The example above also defines a new class called `VideoMode`,
to describe a specific video mode for video display.
This class has four variable stored properties.
The first, `resolution`, is initialized with a new `Resolution` structure instance,
which infers a property type of `Resolution`.
For the other three properties,
new `VideoMode` instances will be initialized with
an `interlaced` setting of `false` (meaning "noninterlaced video"),
a playback frame rate of `0.0`,

and an optional `String` value called `name`.
The `name` property is automatically given a default value of `nil`,
or "no `name` value", because it's of an optional type.

Contoh Kode / Perintah CLI:

```
struct SomeStructure {
    // structure definition goes here
}
class SomeClass {
    // class definition goes here
}

// ----------------------------------------

-->

> Note: Whenever you define a new structure or class,
> you define a new Swift type.
> Give types `UpperCamelCase` names
> (such as `SomeStructure` and `SomeClass` here)
> to match the capitalization of standard Swift types
> (such as `String`, `Int`, and `Bool`).
> Give properties and methods `lowerCamelCase` names
> (such as `frameRate` and `incrementCount`)
> to differentiate them from type names.

Here's an example of a structure definition and a class definition:

// ----------------------------------------

<!--
  - test: `ClassesAndStructures`
```

[swift_072] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Structure and Class Instances

Sumber: ht

Structure and Class Instances
The `Resolution` structure definition and the `VideoMode` class definition
only describe what a `Resolution` or `VideoMode` will look like.
They themselves don't describe a specific resolution or video mode.
To do that, you need to create an instance of the structure or class.
The syntax for creating instances is very similar for both structures and classes:
Structures and classes both use initializer syntax for new instances.
The simplest form of initializer syntax uses the type name of the class or structure
followed by empty parentheses, such as `Resolution()` or `VideoMode()`.
This creates a new instance of the class or structure,
with any properties initialized to their default values.
Class and structure initialization is described in more detail
in .

Contoh Kode / Perintah CLI:

```
let someResolution = Resolution()
let someVideoMode = VideoMode()
```

[swift_073] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Accessing Properties

Sumber: ht

Accessing Properties
You can access the properties of an instance using *dot syntax*.
In dot syntax, you write the property name immediately after the instance name,
separated by a period (`.`), without any spaces:
In this example,
`someResolution.width` refers to the `width` property of `someResolution`,
and returns its default initial value of `0`.
You can drill down into subproperties,
such as the `width` property in the `resolution` property of a `VideoMode`:
You can also use dot syntax to assign a new value to a variable property:

Contoh Kode / Perintah CLI:

```
print("The width of someResolution is \(someResolution.width)")
// Prints "The width of someResolution is 0".

// ----------------------------------------

-->

In this example,
`someResolution.width` refers to the `width` property of `someResolution`,
and returns its default initial value of `0`.

You can drill down into subproperties,
such as the `width` property in the `resolution` property of a `VideoMode`:

// ----------------------------------------

<!--
  - test: `ClassesAndStructures`
```

[swift_074] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Memberwise Initializers for Structure Types

Sumber: ht

Memberwise Initializers for Structure Types
All structures have an automatically generated *memberwise initializer*,
which you can use to initialize the member properties of new structure instances.
Initial values for the properties of the new instance
can be passed to the memberwise initializer by name:
Unlike structures, class instances don't receive a default memberwise initializer.
Initializers are described in more detail in .

Contoh Kode / Perintah CLI:

```
let vga = Resolution(width: 640, height: 480)

// ----------------------------------------

-->

Unlike structures, class instances don't receive a default memberwise initializer.
Initializers are described in more detail in <doc:Initialization>.
```

```
<!--
  - test: `classesDontHaveADefaultMemberwiseInitializer`
```

[swift_075] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Structures and Enumerations Are Value Types

Sumber: ht

Structures and Enumerations Are Value Types
A *value type* is a type whose value is *copied*
when it's assigned to a variable or constant,
or when it's passed to a function.
You've actually been using value types extensively throughout the previous chapters.
In fact, all of the basic types in Swift ---
integers, floating-point numbers, Booleans, strings, arrays and dictionaries ---
are value types, and are implemented as structures behind the scenes.
All structures and enumerations are value types in Swift.
This means that any structure and enumeration instances you create ---
and any value types they have as properties ---
are always copied when they're passed around in your code.
> Note: Collections defined by the Swift standard library
> like arrays, dictionaries, and strings
> use an optimization to reduce the performance cost of copying.
> Instead of making a copy immediately,
> these collections share the memory where the elements are stored
> between the original instance and any copies.
> If one of the copies of the collection is modified,
> the elements are copied just before the modification.
> The behavior you see in your code
> is always as if a copy took place immediately.
Consider this example, which uses the `Resolution` structure from the previous example:
This example declares a constant called `hd`
and sets it to a `Resolution` instance initialized with
the width and height of full HD video
(1920 pixels wide by 1080 pixels high).
It then declares a variable called `cinema`
and sets it to the current value of `hd`.
Because `Resolution` is a structure,
a *copy* of the existing instance is made,
and this new copy is assigned to `cinema`.
Even though `hd` and `cinema` now have the same width and height,
they're two completely different instances behind

Contoh Kode / Perintah CLI:

```
let hd = Resolution(width: 1920, height: 1080)
var cinema = hd

// ----------------------------------------

-->

This example declares a constant called `hd`
and sets it to a `Resolution` instance initialized with
the width and height of full HD video
```

```
(1920 pixels wide by 1080 pixels high).

It then declares a variable called `cinema`
and sets it to the current value of `hd`.
Because `Resolution` is a structure,
a *copy* of the existing instance is made,
and this new copy is assigned to `cinema`.
Even though `hd` and `cinema` now have the same width and height,
they're two completely different instances behind the scenes.

Next, the `width` property of `cinema` is amended to be
the width of the slightly wider 2K standard used for digital cinema projection
(2048 pixels wide and 1080 pixels high):

// ----------------------------------------

<!--
  - test: `ClassesAndStructures`
```

[swift_076] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Classes Are Reference Types

Sumber: ht

Classes Are Reference Types
Unlike value types, *reference types* are *not* copied
when they're assigned to a variable or constant,
or when they're passed to a function.
Rather than a copy, a reference to the same existing instance is used.
Here's an example, using the `VideoMode` class defined above:
This example declares a new constant called `tenEighty`
and sets it to refer to a new instance of the `VideoMode` class.
The video mode is assigned a copy of the HD resolution of `1920` by `1080` from before.
It's set to be interlaced,
its name is set to `"1080i"`,
and its frame rate is set to `25.0` frames per second.
Next, `tenEighty` is assigned to a new constant, called `alsoTenEighty`,
and the frame rate of `alsoTenEighty` is modified:
Because classes are reference types,
`tenEighty` and `alsoTenEighty` actually both refer to the *same* `VideoMode` instance.
Effectively, they're just two different names for the same single instance,
as shown in the figure below:

Checking the `frameRate` property of `tenEighty`
shows that it correctly reports the new frame rate of `30.0`
from the underlying `VideoMode` instance:
This example also shows how reference types can be harder to reason about.
If `tenEighty` and `alsoTenEighty` were far apart in your program's code,
it could be difficult to find all the ways that the video mode is changed.
Wherever you use `tenEighty`,
you also have to think about the code that uses `alsoTenEighty`,
and vice versa.
In contrast, value types are easier to reason about
because all of the code that interacts with the same value
is close together in your source files.

Note that `tenEighty` and `alsoTenEighty` are declared as *constants*,
rather than variables.
However, you can still change `tenEighty.frameRate` and `alsoTen

Contoh Kode / Perintah CLI:

```
let tenEighty = VideoMode()
tenEighty.resolution = hd
tenEighty.interlaced = true
tenEighty.name = "1080i"
tenEighty.frameRate = 25.0

// ----------------------------------------

-->

This example declares a new constant called `tenEighty`
and sets it to refer to a new instance of the `VideoMode` class.
The video mode is assigned a copy of the HD resolution of `1920` by `1080` from before.
It's set to be interlaced,
its name is set to `"1080i"`,
and its frame rate is set to `25.0` frames per second.

Next, `tenEighty` is assigned to a new constant, called `alsoTenEighty`,
and the frame rate of `alsoTenEighty` is modified:

// ----------------------------------------

<!--
  - test: `ClassesAndStructures`
```

[swift_077] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Identity Operators

Sumber: ht

Identity Operators
Because classes are reference types,
it's possible for multiple constants and variables to refer to
the same single instance of a class behind the scenes.
(The same isn't true for structures and enumerations,
because they're always copied when they're assigned to a constant or variable,
or passed to a function.)
It can sometimes be useful to find out whether two constants or variables refer to
exactly the same instance of a class.
To enable this, Swift provides two identity operators:
- Identical to (`===`)
- Not identical to (`!==`)
Use these operators to check whether two constants or variables refer to the same single instance:
Note that *identical to* (represented by three equal signs, or `===`)
doesn't mean the same thing as *equal to* (represented by two equal signs, or `==`).
Identical to means that
two constants or variables of class type refer to exactly the same class instance.
Equal to means that
two instances are considered equal or equivalent in value,
for some appropriate meaning of *equal*, as defined by the type's designer.
When you define your own custom structures and classes,

it's your responsibility to decide what qualifies as two instances being equal.
The process of defining your own implementations of the `==` and `!=` operators
is described in .

Contoh Kode / Perintah CLI:

```
-->

<!--
  - test: `enumerationsDontSupportTheIdentityOperators`

// ----------------------------------------

-->

It can sometimes be useful to find out whether two constants or variables refer to
exactly the same instance of a class.
To enable this, Swift provides two identity operators:

- Identical to (`===`)
- Not identical to (`!==`)

Use these operators to check whether two constants or variables refer to the same single instance:

// ----------------------------------------

<!--
  - test: `ClassesAndStructures`
```

[swift_078] Swift - Structures and Classes

Structures and Classes (Value Types vs Reference Types, Mutating methods, Identity operators): Pointers

Sumber: ht

Pointers
If you have experience with C, C++, or Objective-C,
you may know that these languages use *pointers* to refer to addresses in memory.
A Swift constant or variable that refers to an instance of some reference type
is similar to a pointer in C,
but isn't a direct pointer to an address in memory,
and doesn't require you to write an asterisk (`*`)
to indicate that you are creating a reference.
Instead, these references are defined like any other constant or variable in Swift.
The Swift standard library provides pointer and buffer types
that you can use if you need to interact with pointers directly ---
see Manual Memory Management.

[swift_079] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Overview

Sumber: ht

Protocols
Define requirements that conforming types must implement.
A *protocol* defines a blueprint of
methods, properties, and other requirements
that suit a particular task or piece of functionality.

The protocol can then be *adopted* by a class, structure, or enumeration
to provide an actual implementation of those requirements.
Any type that satisfies the requirements of a protocol is said to
conform to that protocol.
In addition to specifying requirements that conforming types must implement,
you can extend a protocol to implement some of these requirements
or to implement additional functionality that conforming types can take advantage of.

[swift_080] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Protocol Syntax

Sumber: ht

Protocol Syntax
You define protocols in a very similar way to classes, structures, and enumerations:
Custom types state that they adopt a particular protocol
by placing the protocol's name after the type's name,
separated by a colon, as part of their definition.
Multiple protocols can be listed, and are separated by commas:
If a class has a superclass, list the superclass name
before any protocols it adopts, followed by a comma:
> Note: Because protocols are types,
> begin their names with a capital letter
> (such as `FullyNamed` and `RandomNumberGenerator`)
> to match the names of other types in Swift
> (such as `Int`, `String`, and `Double`).

Contoh Kode / Perintah CLI:

```
protocol SomeProtocol {
    // protocol definition goes here
}

// ----------------------------------------

-->

Custom types state that they adopt a particular protocol
by placing the protocol's name after the type's name,
separated by a colon, as part of their definition.
Multiple protocols can be listed, and are separated by commas:

// ----------------------------------------

<!--
  - test: `protocolSyntax`
```

[swift_081] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Property Requirements

Sumber: ht

Property Requirements
A protocol can require any conforming type to provide
an instance property or type property with a particular name and type.
The protocol doesn't specify whether the property should be

a stored property or a computed property ---
it only specifies the required property name and type.
The protocol also specifies whether each property must be gettable
or gettable *and* settable.
If a protocol requires a property to be gettable and settable,
that property requirement can't be fulfilled by
a constant stored property or a read-only computed property.
If the protocol only requires a property to be gettable,
the requirement can be satisfied by any kind of property,
and it's valid for the property to be also settable
if this is useful for your own code.
Property requirements are always declared as variable properties,
prefixed with the `var` keyword.
Gettable and settable properties are indicated by writing
`{ get set }` after their type declaration,
and gettable properties are indicated by writing `{ get }`.
Always prefix type property requirements with the `static` keyword
when you define them in a protocol.
This rule pertains even though type property requirements can be prefixed with
the `class` or `static` keyword when implemented by a class:
Here's an example of a protocol with a single instance property requirement:
The `FullyNamed` protocol requires a conforming type to provide a fully qualified name.
The protocol doesn't specify anything else about the nature of the conforming type ---
it only specifies that the type must be able to provide a full name for itself.
The protocol states that any `FullyNamed` type must have
a gettable instance property called `fullName`, which is of type `String`.
Here's an example of a simple structure that adopts

Contoh Kode / Perintah CLI:

```
protocol SomeProtocol {
    var mustBeSettable: Int { get set }
    var doesNotNeedToBeSettable: Int { get }
}

// ----------------------------------------

-->

Always prefix type property requirements with the `static` keyword
when you define them in a protocol.
This rule pertains even though type property requirements can be prefixed with
the `class` or `static` keyword when implemented by a class:

// ----------------------------------------

<!--
  - test: `instanceProperties`
```

[swift_082] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Method Requirements

Sumber: ht

Method Requirements
Protocols can require specific instance methods and type methods
to be implemented by conforming types.
These methods are written as part of the protocol's definition
in exactly the same way as for normal instance and type methods,
but without curly braces or a method body.
Variadic parameters are allowed, subject to the same rules as for normal methods.
Default values, however, can't be specified for method parameters within a protocol's definition.
As with type property requirements,
you always prefix type method requirements with the `static` keyword
when they're defined in a protocol.
This is true even though type method requirements are prefixed with
the `class` or `static` keyword when implemented by a class:
The following example defines a protocol with a single instance method requirement:
This protocol, `RandomNumberGenerator`, requires any conforming type
to have an instance method called `random`,
which returns a `Double` value whenever it's called.
Although it's not specified as part of the protocol,
it's assumed that this value will be
a number from `0.0` up to (but not including) `1.0`.
The `RandomNumberGenerator` protocol doesn't make any assumptions
about how each random number will be generated ---
it simply requires the generator to provide a standard way
to generate a new random number.
Here's an implementation of a class that adopts and conforms to
the `RandomNumberGenerator` protocol.
This class implements a pseudorandom number generator algorithm known as
a *linear congruential generator*:

Contoh Kode / Perintah CLI:

```
protocol SomeProtocol {
    static func someTypeMethod()
}

// ----------------------------------------

-->

The following example defines a protocol with a single instance method requirement:

// ----------------------------------------

<!--
  - test: `protocols`
```

[swift_083] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Mutating Method Requirements

Sumber: ht

Mutating Method Requirements
It's sometimes necessary for a method to modify (or *mutate*) the instance it belongs to.
For instance methods on value types (that is, structures and enumerations)
you place the `mutating` keyword before a method's `func` keyword
to indicate that the method is allowed to modify the instance it belongs to
and any properties of that instance.
This process is described in .
If you define a protocol instance method requirement
that's intended to mutate instances of any type that adopts the protocol,
mark the method with the `mutating` keyword
as part of the protocol's definition.
This enables structures and enumerations to adopt the protocol
and satisfy that method requirement.
> Note: If you mark a protocol instance method requirement as `mutating`,
> you don't need to write the `mutating` keyword when writing
> an implementation of that method for a class.
> The `mutating` keyword is only used by structures and enumerations.
The example below defines a protocol called `Togglable`,
which defines a single instance method requirement called `toggle`.
As its name suggests, the `toggle()` method is intended to
toggle or invert the state of any conforming type,
typically by modifying a property of that type.
The `toggle()` method is marked with the `mutating` keyword
as part of the `Togglable` protocol definition,
to indicate that the method is expected to mutate the state of a conforming instance
when it's called:
If you implement the `Togglable` protocol for a structure or enumeration,
that structure or enumeration can conform to the protocol
by providing an implementation of the `toggle()` method
that's also marked as `mutating`.
The example below defines an enumeration called `OnOffSwitch`.
This enumeration toggles between two states,
indica

Contoh Kode / Perintah CLI:

```
protocol Togglable {
    mutating func toggle()
}

// ----------------------------------------

-->

If you implement the `Togglable` protocol for a structure or enumeration,
that structure or enumeration can conform to the protocol
by providing an implementation of the `toggle()` method
that's also marked as `mutating`.

The example below defines an enumeration called `OnOffSwitch`.
This enumeration toggles between two states,
```

```
indicated by the enumeration cases `on` and `off`.
The enumeration's `toggle` implementation is marked as `mutating`,
to match the `Togglable` protocol's requirements:

// ----------------------------------------

<!--
  - test: `mutatingRequirements`
```

[swift_084] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Initializer Requirements

Sumber: ht

Initializer Requirements
Protocols can require specific initializers
to be implemented by conforming types.
You write these initializers as part of the protocol's definition
in exactly the same way as for normal initializers,
but without curly braces or an initializer body:

Contoh Kode / Perintah CLI:

```
protocol SomeProtocol {
    init(someParameter: Int)
}
```

[swift_085] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Class Implementations of Protocol Initializer Requirements

Sumber: ht

Class Implementations of Protocol Initializer Requirements
You can implement a protocol initializer requirement on a conforming class
as either a designated initializer or a convenience initializer.
In both cases,
you must mark the initializer implementation with the `required` modifier:
The use of the `required` modifier ensures that
you provide an explicit or inherited implementation of the initializer requirement
on all subclasses of the conforming class,
such that they also conform to the protocol.
For more information on required initializers,
see .
> Note: You don't need to mark protocol initializer implementations with the `required` modifier
> on classes that are marked with the `final` modifier,
> because final classes can't be subclassed.
> For more about the `final` modifier, see .
If a subclass overrides a designated initializer from a superclass,
and also implements a matching initializer requirement from a protocol,
mark the initializer implementation with both the `required` and `override` modifiers:

Contoh Kode / Perintah CLI:

```
class SomeClass: SomeProtocol {
    required init(someParameter: Int) {
        // initializer implementation goes here
    }
}
```

```
// ----------------------------------------

-->

<!--
  - test: `protocolInitializerRequirementsCanBeImplementedAsDesignatedOrConvenience`

// ----------------------------------------

-->

The use of the `required` modifier ensures that
you provide an explicit or inherited implementation of the initializer requirement
on all subclasses of the conforming class,
such that they also conform to the protocol.

For more information on required initializers,
see <doc:Initialization#Required-Initializers>.

<!--
  - test: `protocolInitializerRequirementsRequireTheRequiredModifierOnTheImplementingClass`
```

[swift_086] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Failable Initializer Requirements

Sumber: ht

Failable Initializer Requirements
Protocols can define failable initializer requirements for conforming types,
as defined in .
A failable initializer requirement can be satisfied by
a failable or nonfailable initializer on a conforming type.
A nonfailable initializer requirement can be satisfied by
a nonfailable initializer or an implicitly unwrapped failable initializer.

Contoh Kode / Perintah CLI:

```
-->

<!--
  - test: `failableRequirementCanBeSatisfiedByIUOInitializer`

// ----------------------------------------

-->

<!--
  - test: `iuoRequirementCanBeSatisfiedByFailableInitializer`

// ----------------------------------------

-->

<!--
  - test: `iuoRequirementCanBeSatisfiedByIUOInitializer`
```

[swift_087] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Protocols that Have Only Semantic Requirements

Sumber: ht

Protocols that Have Only Semantic Requirements
All of the example protocols above require some methods or properties,
but a protocol declaration doesn't have to include any requirements.
You can also use a protocol to describe *semantic* requirements ---
that is, requirements about how values of those types behave
and about operations that they support.
The Swift standard library defines several protocols
that don't have any required methods or properties:
- [`Sendable`][] for values that can be shared across concurrency domains,
as discussed in .
- [`Copyable`][] for values that Swift can copy
when you pass them to a function,
as discussed in .
- [`BitwiseCopyable`][] for values that can be copied, bit-by-bit.
[`BitwiseCopyable`]: https://developer.apple.com/documentation/swift/bitwisecopyable
[`Copyable`]: https://developer.apple.com/documentation/swift/copyable
[`Sendable`]: https://developer.apple.com/documentation/swift/sendable
For information about these protocols' requirements,
see the overview in their documentation.
You use the same syntax to adopt these protocols
as you do to adopt other protocols.
The only difference is that you don't include
method or property declarations that implement the protocol's requirements.
For example:
The code above defines a new structure.
Because `Copyable` has only semantic requirements,
there isn't any code in the structure declaration to adopt the protocol.
Similarly, because `BitwiseCopyable` has only semantic requirements,
the extension that adopts that protocol has an empty body.
You usually don't need to write conformance to these protocols ---
instead, Swift implicitly adds the conformance for you,
as described in .

Contoh Kode / Perintah CLI:

```
struct MyStruct: Copyable {
    var counter = 12
}

extension MyStruct: BitwiseCopyable { }
```

[swift_088] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Protocols as Types

Sumber: ht

Protocols as Types
Protocols don't actually implement any functionality themselves.
Regardless, you can use a protocol as a type in your code.
The most common way to use a protocol as a type

is to use a protocol as a generic constraint.
Code with generic constraints can work with
any type that conforms to the protocol,
and the specific type is chosen by the code that uses the API.
For example,
when you call a function that takes an argument
and that argument's type is generic,
the caller chooses the type.
Code with an opaque type
works with some type that conforms to the protocol.
The underlying type is known at compile time,
and the API implementation chooses that type,
but that type's identity is hidden from clients of the API.
Using an opaque type lets you prevent implementation details of an API
from leaking through the layer of abstraction ---
for example, by hiding the specific return type from a function,
and only guaranteeing that the value conforms to a given protocol.
Code with a boxed protocol type
works with any type, chosen at runtime, that conforms to the protocol.
To support this runtime flexibility,
Swift adds a level of indirection when necessary ---
known as a *box*,
which has a performance cost.
Because of this flexibility,
Swift doesn't know the underlying type at compile time,
which means you can access only the members
that are required by the protocol.
Accessing any other APIs on the underlying type
requires casting at runtime.
For information about using protocols as generic constraints,
see .
For information about opaque types, and boxed protocol types,
see .

[swift_089] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Delegation

Sumber: ht

Delegation
Delegation is a design pattern that enables
a class or structure to hand off (or *delegate*)
some of its responsibilities to an instance of another type.
This design pattern is implemented by defining
a protocol that encapsulates the delegated responsibilities,
such that a conforming type (known as a delegate)
is guaranteed to provide the functionality that has been delegated.
Delegation can be used to respond to a particular action,
or to retrieve data from an external source without needing to know
the underlying type of that source.
The example below defines a dice game
and a nested protocol for a delegate

that tracks the game's progress:
The `DiceGame` class implements a game where
each player takes a turn rolling dice,
and the player who rolls the highest number wins the round.
It uses a linear congruential generator
from the example earlier in the chapter,
to generate random numbers for dice rolls.
The `DiceGame.Delegate` protocol can be adopted
to track the progress of a dice game.
Because the `DiceGame.Delegate` protocol
is always used in the context of a dice game,
it's nested inside of the `DiceGame` class.
Protocols can be nested
inside of type declarations like structures and classes,
as long as the outer declaration isn't generic.
For information about nesting types, see .
To prevent strong reference cycles,
delegates are declared as weak references.
For information about weak references,
see .
Marking the protocol as class-only
lets the `DiceGame` class
declare that its delegate must use a weak reference.
A class-only protocol
is marked by its inheritance from `AnyObject`,
as discussed in .
`DiceGame.Delegate` provides three methods for tracking the progress of a game.
These three methods are incorporated into the game logic
in the `play(ro

Contoh Kode / Perintah CLI:

```
class DiceGame {
    let sides: Int
    let generator = LinearCongruentialGenerator()
    weak var delegate: Delegate?

    init(sides: Int) {
        self.sides = sides
    }

    func roll() -> Int {
        return Int(generator.random() * Double(sides)) + 1
    }

    func play(rounds: Int) {
        delegate?.gameDidStart(self)
        for round in 1...rounds {
            let player1 = roll()
            let player2 = roll()
            if player1 == player2 {
                delegate?.game(self, didEndRound: round, winner: nil)
            } else if player1 > player2 {
                delegate?.game(self, didEndRound: round, winner: 1)
            } else {
                delegate?.game(self, didEndRound: round, winner: 2)
            }
        }
```

```
        delegate?.gameDidEnd(self)
    }

    protocol Delegate: AnyObject {
        func gameDidStart(_ game: DiceGame)
        func game(_ game: DiceGame, didEndRound round: Int, winner: Int?)
        func gameDidEnd(_ game: DiceGame)
    }
}

// ----------------------------------------

class DiceGameTracker: DiceGame.Delegate {
    var playerScore1 = 0
    var playerScore2 = 0
    func gameDidStart(_ game: DiceGame) {
        print("Started a new game")
        playerScore1 = 0
        playerScore2 = 0
    }
    func game(_ game: DiceGame, didEndRound round: Int, winner: Int?) {
        switch winner {
            case 1:
                playerScore1 += 1
                print("Player 1 won round \(round)")
            case 2: playerScore2 += 1
                print("Player 2 won round \(round)")
            default:
                print("The round was a draw")
        }
    }
    func gameDidEnd(_ game: DiceGame) {
        if playerScore1 == playerScore2 {
            print("The game ended in a draw.")
        } else if playerScore1 > playerScore2 {
            print("Player 1 won!")
        } else {
            print("Player 2 won!")
        }
    }
}

// ----------------------------------------

let tracker = DiceGameTracker()
let game = DiceGame(sides: 6)
game.delegate = tracker
game.play(rounds: 3)
// Started a new game
// Player 2 won round 1
// Player 2 won round 2
// Player 1 won round 3
// Player 2 won!
```

[swift_090] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Adding Protocol Conformance with an Extension

Sumber: ht

Adding Protocol Conformance with an Extension
You can extend an existing type to adopt and conform to a new protocol,
even if you don't have access to the source code for the existing type.
Extensions can add new properties, methods, and subscripts to an existing type,
and are therefore able to add any requirements that a protocol may demand.
For more about extensions, see .
> Note: Existing instances of a type automatically adopt and conform to a protocol
> when that conformance is added to the instance's type in an extension.
For example, this protocol, called `TextRepresentable`, can be implemented by
any type that has a way to be represented as text.
This might be a description of itself, or a text version of its current state:
The `Dice` class from above can be extended to adopt and conform to `TextRepresentable`:
This extension adopts the new protocol in exactly the same way
as if `Dice` had provided it in its original implementation.
The protocol name is provided after the type name, separated by a colon,
and an implementation of all requirements of the protocol
is provided within the extension's curly braces.
Any `Dice` instance can now be treated as `TextRepresentable`:
Similarly, the `SnakesAndLadders` game class can be extended to
adopt and conform to the `TextRepresentable` protocol:

Contoh Kode / Perintah CLI:

```
protocol TextRepresentable {
    var textualDescription: String { get }
}

// ----------------------------------------

-->

The `Dice` class from above can be extended to adopt and conform to `TextRepresentable`:

<!--
  No "from above" xref because
  even though Dice isn't defined in the section immediately previous
  it's part of a running example and Dice is used in that section.
-->

// ----------------------------------------

<!--
  - test: `protocols`
```

[swift_091] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Conditionally Conforming to a Protocol

Sumber: ht

Conditionally Conforming to a Protocol
A generic type may be able to satisfy the requirements of a protocol
only under certain conditions,
such as when the type's generic parameter conforms to the protocol.
You can make a generic type conditionally conform to a protocol
by listing constraints when extending the type.
Write these constraints after the name of the protocol you're adopting
by writing a generic `where` clause.
For more about generic `where` clauses, see .
The following extension
makes `Array` instances conform to the `TextRepresentable` protocol
whenever they store elements of a type that conforms to `TextRepresentable`.

Contoh Kode / Perintah CLI:

```
extension Array: TextRepresentable where Element: TextRepresentable {
    var textualDescription: String {
        let itemsAsText = self.map { $0.textualDescription }
        return "[" + itemsAsText.joined(separator: ", ") + "]"
    }
}
let myDice = [d6, d12]
print(myDice.textualDescription)
// Prints "[A 6-sided dice, A 12-sided dice]".
```

[swift_092] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Declaring Protocol Adoption with an Extension

Sumber: ht

Declaring Protocol Adoption with an Extension
If a type already conforms to all of the requirements of a protocol,
but hasn't yet stated that it adopts that protocol,
you can make it adopt the protocol with an empty extension:
Instances of `Hamster` can now be used wherever `TextRepresentable` is the required type:
> Note: Types don't automatically adopt a protocol just by satisfying its requirements.
> They must always explicitly declare their adoption of the protocol.

Contoh Kode / Perintah CLI:

```
struct Hamster {
    var name: String
    var textualDescription: String {
        return "A hamster named \(name)"
    }
}
extension Hamster: TextRepresentable {}

// ----------------------------------------

-->

Instances of `Hamster` can now be used wherever `TextRepresentable` is the required type:

// ----------------------------------------
```

```
<!--
  - test: `protocols`
```

[swift_093] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Adopting a Protocol Using a Synthesized Implementation

Sumber: ht

Adopting a Protocol Using a Synthesized Implementation
Swift can automatically provide the protocol conformance
for `Equatable`, `Hashable`, and `Comparable`
in many simple cases.
Using this synthesized implementation
means you don't have to write repetitive boilerplate code
to implement the protocol requirements yourself.
Swift provides a synthesized implementation of `Equatable`
for the following kinds of custom types:
- Structures that have only stored properties that conform to the `Equatable` protocol
- Enumerations that have only associated types that conform to the `Equatable` protocol
- Enumerations that have no associated types
To receive a synthesized implementation of `==`,
declare conformance to `Equatable`
in the file that contains the original declaration,
without implementing an `==` operator yourself.
The `Equatable` protocol provides a default implementation of `!=`.
The example below defines a `Vector3D` structure
for a three-dimensional position vector `(x, y, z)`,
similar to the `Vector2D` structure.
Because the `x`, `y`, and `z` properties are all of an `Equatable` type,
`Vector3D` receives synthesized implementations
of the equivalence operators.
Swift provides a synthesized implementation of `Hashable`
for the following kinds of custom types:
- Structures that have only stored properties that conform to the `Hashable` protocol
- Enumerations that have only associated types that conform to the `Hashable` protocol
- Enumerations that have no associated types
To receive a synthesized implementation of `hash(into:)`,
declare conformance to `Hashable`
in the file that contains the original declaration,
without implementing a `hash(into:)` method yourself.
Swift provides a synthesized implementation of `Comparable`
for enumerations that don't have a

Contoh Kode / Perintah CLI:

```
struct Vector3D: Equatable {
    var x = 0.0, y = 0.0, z = 0.0
}

let twoThreeFour = Vector3D(x: 2.0, y: 3.0, z: 4.0)
let anotherTwoThreeFour = Vector3D(x: 2.0, y: 3.0, z: 4.0)
if twoThreeFour == anotherTwoThreeFour {
    print("These two vectors are also equivalent.")
}
// Prints "These two vectors are also equivalent."
```

```
// ----------------------------------------

-->

<!--
  Need to cross reference here from "Adopting Common Protocols"
  https://developer.apple.com/documentation/swift/adopting_common_protocols

  Discussion in the article calls out that
  enums without associated values are Equatable & Hashable
  even if you don't declare the protocol conformance.
-->

Swift provides a synthesized implementation of `Hashable`
for the following kinds of custom types:

- Structures that have only stored properties that conform to the `Hashable` protocol
- Enumerations that have only associated types that conform to the `Hashable` protocol
- Enumerations that have no associated types

To receive a synthesized implementation of `hash(into:)`,
declare conformance to `Hashable`
in the file that contains the original declaration,
without implementing a `hash(into:)` method yourself.

Swift provides a synthesized implementation of `Comparable`
for enumerations that don't have a raw value.
If the enumeration has associated types,
they must all conform to the `Comparable` protocol.
To receive a synthesized implementation of `<`,
declare conformance to `Comparable`
in the file that contains the original enumeration declaration,
without implementing a `<` operator yourself.
The `Comparable` protocol's default implementation
of `<=`, `>`, and `>=` provides the remaining comparison operators.

The example below defines a `SkillLevel` enumeration
with cases for beginners, intermediates, and experts.
Experts are additionally ranked by the number of stars they have.

// ----------------------------------------

<!--
  - test: `comparable-enum-synthesis`
```

[swift_094] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Implicit Conformance to a Protocol

Sumber: ht

Implicit Conformance to a Protocol
Some protocols are so common that you would write them
almost every time you declare a new type.
For the following protocols,
Swift automatically infers the conformance
when you define a type that implements the protocol's requirements,
so you don't have to write them yourself:
- [`Copyable`][]

- [`Sendable`][]
- [`BitwiseCopyable`][]
You can still write the conformance explicitly,
but it doesn't change how your code behaves.
To suppress an implicit conformance,
write a tilde (`~`) before the protocol name in the conformance list:
The code above shows part of a wrapper around POSIX file descriptors.
The `FileDescriptor` structure
satisfies all of the requirements of the `Sendable` protocol,
which normally makes it sendable.
However,
writing `~Sendable` suppresses this implicit conformance.
Even though file descriptors use integers
to identify and interact with open files,
and integer values are sendable,
making it nonsendable can help avoid certain kinds of bugs.
Another way to suppress implicit conformance
is with an extension that you mark as unavailable:
When you write `~Sendable` in one place in your code,
as in the previous example,
code elsewhere in your program can still
extend the `FileDescriptor` type to add `Sendable` conformance.
In contrast,
the unavailable extension in this example
suppresses the implicit conformance to `Sendable`
and also prevents any extensions elsewhere in your code
from adding `Sendable` conformance to the type.
> Note:
> In addition to the protocols discussed above,
> distributed actors implicitly conform to the [`Codable`][] protocol.
[`Codable`]: https://developer.apple.com/documentation/swift/codable

Contoh Kode / Perintah CLI:

```
struct FileDescriptor: ~Sendable {
    let rawValue: Int
}

// ----------------------------------------

@available(*, unavailable)
extension FileDescriptor: Sendable { }
```

[swift_095] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Collections of Protocol Types

Sumber: ht

Collections of Protocol Types
A protocol can be used as the type to be stored in
a collection such as an array or a dictionary,
as mentioned in .
This example creates an array of `TextRepresentable` things:
It's now possible to iterate over the items in the array,
and print each item's textual description:

Note that the `thing` constant is of type `TextRepresentable`.
It's not of type `Dice`, or `DiceGame`, or `Hamster`,
even if the actual instance behind the scenes is of one of those types.
Nonetheless, because it's of type `TextRepresentable`,
and anything that's `TextRepresentable` is known to have a `textualDescription` property,
it's safe to access `thing.textualDescription` each time through the loop.

Contoh Kode / Perintah CLI:

```
let things: [TextRepresentable] = [game, d12, simonTheHamster]

// ----------------------------------------

-->

It's now possible to iterate over the items in the array,
and print each item's textual description:

// ----------------------------------------

<!--
  - test: `protocols`
```

[swift_096] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Protocol Inheritance

Sumber: ht

Protocol Inheritance
A protocol can *inherit* one or more other protocols
and can add further requirements on top of the requirements it inherits.
The syntax for protocol inheritance is similar to the syntax for class inheritance,
but with the option to list multiple inherited protocols, separated by commas:
Here's an example of a protocol that inherits
the `TextRepresentable` protocol from above:
This example defines a new protocol, `PrettyTextRepresentable`,
which inherits from `TextRepresentable`.
Anything that adopts `PrettyTextRepresentable` must satisfy all of the requirements
enforced by `TextRepresentable`,
plus the additional requirements enforced by `PrettyTextRepresentable`.
In this example, `PrettyTextRepresentable` adds a single requirement
to provide a gettable property called `prettyTextualDescription` that returns a `String`.
The `SnakesAndLadders` class can be extended to adopt and conform to `PrettyTextRepresentable`:
This extension states that it adopts the `PrettyTextRepresentable` protocol
and provides an implementation of the `prettyTextualDescription` property
for the `SnakesAndLadders` type.
Anything that's `PrettyTextRepresentable` must also be `TextRepresentable`,
and so the implementation of `prettyTextualDescription` starts
by accessing the `textualDescription` property
from the `TextRepresentable` protocol to begin an output string.
It appends a colon and a line break,
and uses this as the start of its pretty text representation.
It then iterates through the array of board squares,
and appends a geometric shape to represent the contents of each square:
- If the square's value is greater than `0`, it's the base of a ladder,
and is represented by `?`.

- If the square's value is less than `0`, it's the head of a snake,
and is represented b

Contoh Kode / Perintah CLI:

```
protocol InheritingProtocol: SomeProtocol, AnotherProtocol {
    // protocol definition goes here
}

// ----------------------------------------

-->

Here's an example of a protocol that inherits
the `TextRepresentable` protocol from above:

// ----------------------------------------

<!--
  - test: `protocols`
```

[swift_097] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Class-Only Protocols

Sumber: ht

Class-Only Protocols
You can limit protocol adoption to class types (and not structures or enumerations)
by adding the `AnyObject` protocol to a protocol's inheritance list.
In the example above, `SomeClassOnlyProtocol` can only be adopted by class types.
It's a compile-time error to write a structure or enumeration definition
that tries to adopt `SomeClassOnlyProtocol`.
> Note: Use a class-only protocol when the behavior defined by that protocol's requirements
> assumes or requires that a conforming type has
> reference semantics rather than value semantics.
> For more about reference and value semantics,
> see
> and .
You can also name a specific class in a protocol's inheritance list
to require that all conforming types are subclasses of that class.
In the example above, `Widget` can only be adopted by subclasses of `Renderer`.
Because the protocol guarantees that every conforming type is a `Renderer` subclass,
the `Widget` protocol extension can call `render(_:)` directly.
A superclass requirement is more restrictive than using `AnyObject`:
it limits conformance to a specific class hierarchy,
not just any class type.
Use this when the protocol's implementation
depends on the concrete behavior of a particular base class.

Contoh Kode / Perintah CLI:

```
protocol SomeClassOnlyProtocol: AnyObject, SomeInheritedProtocol {
    // class-only protocol definition goes here
}

// ----------------------------------------

-->
```

```
In the example above, `SomeClassOnlyProtocol` can only be adopted by class types.
It's a compile-time error to write a structure or enumeration definition
that tries to adopt `SomeClassOnlyProtocol`.

> Note: Use a class-only protocol when the behavior defined by that protocol's requirements
> assumes or requires that a conforming type has
> reference semantics rather than value semantics.
> For more about reference and value semantics,
> see <doc:ClassesAndStructures#Structures-and-Enumerations-Are-Value-Types>
> and <doc:ClassesAndStructures#Classes-Are-Reference-Types>.

<!--
  - test: `anyobject-doesn't-have-to-be-first`

// ----------------------------------------

-->

<!--
  TODO: a Cacheable protocol might make a good example here?
-->

You can also name a specific class in a protocol's inheritance list
to require that all conforming types are subclasses of that class.
```

[swift_098] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Protocol Composition

Sumber: ht

Protocol Composition
It can be useful to require a type to conform to multiple protocols at the same time.
You can combine multiple protocols into a single requirement
with a *protocol composition*.
Protocol compositions behave as if you
defined a temporary local protocol that has the combined requirements
of all protocols in the composition.
Protocol compositions don't define any new protocol types.
Protocol compositions have the form `SomeProtocol & AnotherProtocol`.
You can list as many protocols as you need,
separating them with ampersands (`&`).
In addition to its list of protocols,
a protocol composition can also contain one class type,
which you can use to specify a required superclass.
Here's an example that combines two protocols called `Named` and `Aged`
into a single protocol composition requirement on a function parameter:
In this example,
the `Named` protocol
has a single requirement for a gettable `String` property called `name`.
The `Aged` protocol
has a single requirement for a gettable `Int` property called `age`.
Both protocols are adopted by a structure called `Person`.
The example also defines a `wishHappyBirthday(to:)` function.
The type of the `celebrator` parameter is `Named & Aged`,
which means "any type that conforms to both the `Named` and `Aged` protocols."
It doesn't matter which specific type is passed to the function,

as long as it conforms to both of the required protocols.
The example then creates a new `Person` instance called `birthdayPerson`
and passes this new instance to the `wishHappyBirthday(to:)` function.
Because `Person` conforms to both protocols, this call is valid,
and the `wishHappyBirthday(to:)` function can print its birthday greeting.
Here's an example that combines
the `Named` protocol from the previous example
with a

Contoh Kode / Perintah CLI:

```
protocol Named {
    var name: String { get }
}
protocol Aged {
    var age: Int { get }
}
struct Person: Named, Aged {
    var name: String
    var age: Int
}
func wishHappyBirthday(to celebrator: Named & Aged) {
    print("Happy birthday, \(celebrator.name), you're \(celebrator.age)!")
}
let birthdayPerson = Person(name: "Malcolm", age: 21)
wishHappyBirthday(to: birthdayPerson)
// Prints "Happy birthday, Malcolm, you're 21!"

// ----------------------------------------

-->

In this example,
the `Named` protocol
has a single requirement for a gettable `String` property called `name`.
The `Aged` protocol
has a single requirement for a gettable `Int` property called `age`.
Both protocols are adopted by a structure called `Person`.

The example also defines a `wishHappyBirthday(to:)` function.
The type of the `celebrator` parameter is `Named & Aged`,
which means "any type that conforms to both the `Named` and `Aged` protocols."
It doesn't matter which specific type is passed to the function,
as long as it conforms to both of the required protocols.

The example then creates a new `Person` instance called `birthdayPerson`
and passes this new instance to the `wishHappyBirthday(to:)` function.
Because `Person` conforms to both protocols, this call is valid,
and the `wishHappyBirthday(to:)` function can print its birthday greeting.

Here's an example that combines
the `Named` protocol from the previous example
with a `Location` class:

// ----------------------------------------

<!--
  - test: `protocolComposition`
```

[swift_099] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Checking for Protocol Conformance

Sumber: ht

Checking for Protocol Conformance
You can use the `is` and `as` operators described in
to check for protocol conformance, and to cast to a specific protocol.
Checking for and casting to a protocol
follows exactly the same syntax as checking for and casting to a type:
- The `is` operator returns `true` if an instance conforms to a protocol
and returns `false` if it doesn't.
- The `as?` version of the downcast operator returns
an optional value of the protocol's type,
and this value is `nil` if the instance doesn't conform to that protocol.
- The `as!` version of the downcast operator forces the downcast to the protocol type
and triggers a runtime error if the downcast doesn't succeed.
This example defines a protocol called `HasArea`,
with a single property requirement of a gettable `Double` property called `area`:
Here are two classes, `Circle` and `Country`,
both of which conform to the `HasArea` protocol:
The `Circle` class implements the `area` property requirement
as a computed property, based on a stored `radius` property.
The `Country` class implements the `area` requirement directly as a stored property.
Both classes correctly conform to the `HasArea` protocol.
Here's a class called `Animal`, which doesn't conform to the `HasArea` protocol:
The `Circle`, `Country` and `Animal` classes don't have a shared base class.
Nonetheless, they're all classes, and so instances of all three types
can be used to initialize an array that stores values of type `AnyObject`:
The `objects` array is initialized with an array literal containing
a `Circle` instance with a radius of 2 units;
a `Country` instance initialized with
the surface area of the United Kingdom in square kilometers;
and an `Animal` instance with four legs.
The `objects` array can now be iterated,
and each obje

Contoh Kode / Perintah CLI:

```
protocol HasArea {
    var area: Double { get }
}

// ----------------------------------------

-->

Here are two classes, `Circle` and `Country`,
both of which conform to the `HasArea` protocol:

// ----------------------------------------

<!--
  - test: `protocolConformance`
```

[swift_100] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Optional Protocol Requirements

Sumber: ht

Optional Protocol Requirements
You can define *optional requirements* for protocols.
These requirements don't have to be implemented by types that conform to the protocol.
Optional requirements are prefixed by the `optional` modifier
as part of the protocol's definition.
Optional requirements are available so that you can write code
that interoperates with Objective-C.
Both the protocol and the optional requirement
must be marked with the `@objc` attribute.
Note that `@objc` protocols can be adopted only by classes,
not by structures or enumerations.
When you use a method or property in an optional requirement,
its type automatically becomes an optional.
For example,
a method of type `(Int) -> String` becomes `((Int) -> String)?`.
Note that the entire function type
is wrapped in the optional,
not the method's return value.
An optional protocol requirement can be called with optional chaining,
to account for the possibility that the requirement was not implemented
by a type that conforms to the protocol.
You check for an implementation of an optional method
by writing a question mark after the name of the method when it's called,
such as `someOptionalMethod?(someArgument)`.
For information on optional chaining, see .
The following example defines an integer-counting class called `Counter`,
which uses an external data source to provide its increment amount.
This data source is defined by the `CounterDataSource` protocol,
which has two optional requirements:
The `CounterDataSource` protocol defines
an optional method requirement called `increment(forCount:)`
and an optional property requirement called `fixedIncrement`.
These requirements define two different ways for data sources to provide
an appropriate increment amount for a `Counter` instance.
> Note: Strictly speak

Contoh Kode / Perintah CLI:

```
@objc protocol CounterDataSource {
    @objc optional func increment(forCount count: Int) -> Int
    @objc optional var fixedIncrement: Int { get }
}

// ----------------------------------------

-->

The `CounterDataSource` protocol defines
an optional method requirement called `increment(forCount:)`
and an optional property requirement called `fixedIncrement`.
These requirements define two different ways for data sources to provide
```

```
an appropriate increment amount for a `Counter` instance.

> Note: Strictly speaking, you can write a custom class
> that conforms to `CounterDataSource` without implementing
> *either* protocol requirement.
> They're both optional, after all.
> Although technically allowed, this wouldn't make for a very good data source.

The `Counter` class, defined below,
has an optional `dataSource` property of type `CounterDataSource?`:

// ----------------------------------------

<!--
  - test: `protocolConformance`
```

[swift_101] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Protocol Extensions

Sumber: ht

Protocol Extensions
Protocols can be extended to provide method,
initializer, subscript, and computed property implementations
to conforming types.
This allows you to define behavior on protocols themselves,
rather than in each type's individual conformance or in a global function.
For example, the `RandomNumberGenerator` protocol can be extended
to provide a `randomBool()` method,
which uses the result of the required `random()` method
to return a random `Bool` value:
By creating an extension on the protocol,
all conforming types automatically gain this method implementation
without any additional modification.
Protocol extensions can add implementations to conforming types
but can't make a protocol extend or inherit from another protocol.
Protocol inheritance is always specified in the protocol declaration itself.

Contoh Kode / Perintah CLI:

```
extension RandomNumberGenerator {
    func randomBool() -> Bool {
        return random() > 0.5
    }
}

// ----------------------------------------

-->

By creating an extension on the protocol,
all conforming types automatically gain this method implementation
without any additional modification.

// ----------------------------------------

<!--
  - test: `protocols`
```

[swift_102] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Providing Default Implementations

Sumber: ht

Providing Default Implementations
You can use protocol extensions to provide a default implementation
to any method or computed property requirement of that protocol.
If a conforming type provides its own implementation of a required method or property,
that implementation will be used instead of the one provided by the extension.
> Note: Protocol requirements with default implementations provided by extensions
> are distinct from optional protocol requirements.
> Although conforming types don't have to provide their own implementation of either,
> requirements with default implementations can be called without optional chaining.
For example, the `PrettyTextRepresentable` protocol,
which inherits the `TextRepresentable` protocol
can provide a default implementation of its required `prettyTextualDescription` property
to simply return the result of accessing the `textualDescription` property:

Contoh Kode / Perintah CLI:

```
extension PrettyTextRepresentable  {
    var prettyTextualDescription: String {
        return textualDescription
    }
}
```

[swift_103] Swift - Protocols & Polymorphism

Protocols (Property/Method requirements, Protocol Inheritance, Delegation, any/some): Adding Constraints to Protocol Extensions

Sumber: ht

Adding Constraints to Protocol Extensions
When you define a protocol extension,
you can specify constraints that conforming types
must satisfy before the methods and properties of the extension are available.
You write these constraints after the name of the protocol you're extending
by writing a generic `where` clause.
For more about generic `where` clauses, see .
For example,
you can define an extension to the `Collection` protocol
that applies to any collection whose elements conform
to the `Equatable` protocol.
By constraining a collection's elements to the `Equatable` protocol,
a part of the Swift standard library,
you can use the `==` and `!=` operators to check for equality and inequality between two elements.
The `allEqual()` method returns `true`
only if all the elements in the collection are equal.
Consider two arrays of integers,
one where all the elements are the same,
and one where they aren't:
Because arrays conform to `Collection`
and integers conform to `Equatable`,
`equalNumbers` and `differentNumbers` can use the `allEqual()` method:
> Note: If a conforming type satisfies the requirements for multiple constrained extensions

> that provide implementations for the same method or property,
> Swift uses the implementation corresponding to the most specialized constraints.

Contoh Kode / Perintah CLI:

```
extension Collection where Element: Equatable {
    func allEqual() -> Bool {
        for element in self {
            if element != self.first {
                return false
            }
        }
        return true
    }
}

// -----------------------------------------

-->

The `allEqual()` method returns `true`
only if all the elements in the collection are equal.

Consider two arrays of integers,
one where all the elements are the same,
and one where they aren't:

// -----------------------------------------

<!--
  - test: `protocols`
```

[swift_104] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Overview

Sumber: ht

Generics
Write code that works for multiple types and specify requirements for those types.
Generic code enables you to write flexible, reusable functions and types
that can work with any type, subject to requirements that you define.
You can write code that avoids duplication
and expresses its intent in a clear, abstracted manner.
Generics are one of the most powerful features of Swift,
and much of the Swift standard library is built with generic code.
In fact, you've been using generics throughout the *Language Guide*,
even if you didn't realize it.
For example, Swift's `Array` and `Dictionary` types
are both generic collections.
You can create an array that holds `Int` values,
or an array that holds `String` values,
or indeed an array for any other type that can be created in Swift.
Similarly, you can create a dictionary to store values of any specified type,
and there are no limitations on what that type can be.

[swift_105] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): The Problem that Generics Solve

Sumber: ht

The Problem that Generics Solve
Here's a standard, nongeneric function called `swapTwoInts(_:_:)`,
which swaps two `Int` values:
This function makes use of in-out parameters to swap the values of `a` and `b`,
as described in .
The `swapTwoInts(_:_:)` function swaps the original value of `b` into `a`,
and the original value of `a` into `b`.
You can call this function to swap the values in two `Int` variables:
The `swapTwoInts(_:_:)` function is useful, but it can only be used with `Int` values.
If you want to swap two `String` values,
or two `Double` values,
you have to write more functions,
such as the `swapTwoStrings(_:_:)` and `swapTwoDoubles(_:_:)` functions shown below:
You may have noticed that the bodies of
the `swapTwoInts(_:_:)`, `swapTwoStrings(_:_:)`, and `swapTwoDoubles(_:_:)` functions are identical.
The only difference is the type of the values that they accept
(`Int`, `String`, and `Double`).
It's more useful, and considerably more flexible,
to write a single function that swaps two values of *any* type.
Generic code enables you to write such a function.
(A generic version of these functions is defined below.)
> Note: In all three functions,
> the types of `a` and `b` must be the same.
> If `a` and `b` aren't of the same type,
> it isn't possible to swap their values.
> Swift is a type-safe language,
> and doesn't allow (for example) a variable of type `String`
> and a variable of type `Double`
> to swap values with each other.
> Attempting to do so results in a compile-time error.

Contoh Kode / Perintah CLI:

```
func swapTwoInts(_ a: inout Int, _ b: inout Int) {
    let temporaryA = a
    a = b
    b = temporaryA
}

// ----------------------------------------

-->

This function makes use of in-out parameters to swap the values of `a` and `b`,
as described in <doc:Functions#In-Out-Parameters>.

The `swapTwoInts(_:_:)` function swaps the original value of `b` into `a`,
and the original value of `a` into `b`.
You can call this function to swap the values in two `Int` variables:

// ----------------------------------------
```

```
<!--
  - test: `whyGenerics`
```

[swift_106] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Generic Functions

Sumber: ht

Generic Functions
Generic functions can work with any type.
Here's a generic version of the `swapTwoInts(_:_:)` function from above,
called `swapTwoValues(_:_:)`:
The body of the `swapTwoValues(_:_:)` function
is identical to the body of the `swapTwoInts(_:_:)` function.
However, the first line of `swapTwoValues(_:_:)`
is slightly different from `swapTwoInts(_:_:)`.
Here's how the first lines compare:
The generic version of the function
uses a *placeholder* type name (called `T`, in this case)
instead of an *actual* type name (such as `Int`, `String`, or `Double`).
The placeholder type name doesn't say anything about what `T` must be,
but it *does* say that both `a` and `b` must be of the same type `T`,
whatever `T` represents.
The actual type to use in place of `T`
is determined each time the `swapTwoValues(_:_:)` function is called.
The other difference between a generic function and a nongeneric function
is that the generic function's name (`swapTwoValues(_:_:)`)
is followed by the placeholder type name (`T`) inside angle brackets (``).
The brackets tell Swift that `T` is a placeholder type name
within the `swapTwoValues(_:_:)` function definition.
Because `T` is a placeholder, Swift doesn't look for an actual type called `T`.
The `swapTwoValues(_:_:)` function can now be called in the same way as `swapTwoInts`,
except that it can be passed two values of *any* type,
as long as both of those values are of the same type as each other.
Each time `swapTwoValues(_:_:)` is called,
the type to use for `T` is inferred from the types of values passed to the function.
In the two examples below, `T` is inferred to be `Int` and `String` respectively:
> Note: The `swapTwoValues(_:_:)` function defined above is inspired by
> a generic function called `swap`, which is part of

Contoh Kode / Perintah CLI:

```
func swapTwoValues<T>(_ a: inout T, _ b: inout T) {
    let temporaryA = a
    a = b
    b = temporaryA
}

// ----------------------------------------

-->

<!--
  This could be done in one line using a tuple pattern: (a, b) = (b, a)
  That's probably not as approachable here, and the novel syntax to avoid an
  explicit placeholder variable might distract from the discussion of
```

```
  generics.
-->

The body of the `swapTwoValues(_:_:)` function
is identical to the body of the `swapTwoInts(_:_:)` function.
However, the first line of `swapTwoValues(_:_:)`
is slightly different from `swapTwoInts(_:_:)`.
Here's how the first lines compare:

// ----------------------------------------

<!--
  - test: `genericFunctionsComparison`
```

[swift_107] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Type Parameters

Sumber: ht

Type Parameters
In the `swapTwoValues(_:_:)` example above,
the placeholder type `T` is an example of a *type parameter*.
Type parameters specify and name a placeholder type,
and are written immediately after the function's name,
between a pair of matching angle brackets (such as ``).
Once you specify a type parameter,
you can use it to define the type of a function's parameters
(such as the `a` and `b` parameters of the `swapTwoValues(_:_:)` function),
or as the function's return type,
or as a type annotation within the body of the function.
In each case, the type parameter
is replaced with an *actual* type whenever the function is called.
(In the `swapTwoValues(_:_:)` example above,
`T` was replaced with `Int` the first time the function was called,
and was replaced with `String` the second time it was called.)
You can provide more than one type parameter
by writing multiple type parameter names within the angle brackets,
separated by commas.

[swift_108] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Naming Type Parameters

Sumber: ht

Naming Type Parameters
In most cases, type parameters have descriptive names,
such as `Key` and `Value` in `Dictionary`
and `Element` in `Array`,
which tells the reader about the relationship between the type parameter
and the generic type or function it's used in.
However, when there isn't a meaningful relationship between them,
it's traditional to name them using single letters such as `T`, `U`, and `V`,
such as `T` in the `swapTwoValues(_:_:)` function above.
Use upper camel case names for type parameters,

like `T` and `MyTypeParameter`,
to indicate that they're a placeholder for a *type*, not a value.
> Note:
> If you don't need to name a type parameter
> and its generic type constraints are simple,
> there's an alternate, lightweight syntax you can use instead,
> as described in .

[swift_109] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Generic Types

Sumber: ht

Generic Types
In addition to generic functions,
Swift enables you to define your own *generic types*.
These are custom classes, structures, and enumerations
that can work with *any* type, in a similar way to `Array` and `Dictionary`.
This section shows you how to write a generic collection type called `Stack`.
A stack is an ordered set of values, similar to an array,
but with a more restricted set of operations than Swift's `Array` type.
An array allows new items to be inserted and removed at any location in the array.
A stack, however, allows new items to be appended only to the end of the collection
(known as *pushing* a new value on to the stack).
Similarly, a stack allows items to be removed only from the end of the collection
(known as *popping* a value off the stack).
> Note: The concept of a stack is used by the `UINavigationController` class
> to model the view controllers in its navigation hierarchy.
> You call the `UINavigationController` class
> `pushViewController(_:animated:)` method to add (or push)
> a view controller on to the navigation stack,
> and its `popViewControllerAnimated(_:)` method to remove (or pop)
> a view controller from the navigation stack.
> A stack is a useful collection model whenever you need a strict
> "last in, first out" approach to managing a collection.
The illustration below shows the push and pop behavior for a stack:

1. There are currently three values on the stack.
2. A fourth value is pushed onto the top of the stack.
3. The stack now holds four values, with the most recent one at the top.
4. The top item in the stack is popped.
5. After popping a value, the stack once again holds three values.
Here's how to write a nongeneric version of a stack,
in this case for a stack of `Int` values:
This structure

Contoh Kode / Perintah CLI:

```
struct IntStack {
    var items: [Int] = []
    mutating func push(_ item: Int) {
        items.append(item)
    }
    mutating func pop() -> Int {
        return items.removeLast()
```

```
    }
}

// ----------------------------------------

-->

This structure uses an `Array` property called `items` to store the values in the stack.
`Stack` provides two methods, `push` and `pop`,
to push and pop values on and off the stack.
These methods are marked as `mutating`,
because they need to modify (or *mutate*) the structure's `items` array.

The `IntStack` type shown above can only be used with `Int` values, however.
It would be much more useful to define a *generic* `Stack` structure,
that can manage a stack of *any* type of value.

Here's a generic version of the same code:

// ----------------------------------------

<!--
  - test: `genericStack`
```

[swift_110] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Extending a Generic Type

Sumber: ht

Extending a Generic Type
When you extend a generic type,
you don't provide a type parameter list as part of the extension's definition.
Instead, the type parameter list from the *original* type definition
is available within the body of the extension,
and the original type parameter names are used to refer to
the type parameters from the original definition.
The following example extends the generic `Stack` type to add
a read-only computed property called `topItem`,
which returns the top item on the stack without popping it from the stack:
The `topItem` property returns an optional value of type `Element`.
If the stack is empty, `topItem` returns `nil`;
if the stack isn't empty, `topItem` returns the final item in the `items` array.
Note that this extension doesn't define a type parameter list.
Instead, the `Stack` type's existing type parameter name, `Element`,
is used within the extension to indicate the optional type of
the `topItem` computed property.
The `topItem` computed property can now be used with any `Stack` instance
to access and query its top item without removing it.
Extensions of a generic type can also include requirements
that instances of the extended type must satisfy
in order to gain the new functionality,
as discussed in below.

Contoh Kode / Perintah CLI:

```
extension Stack {
    var topItem: Element? {
        return items.isEmpty ? nil : items[items.count - 1]
```

```
    }
}

// ----------------------------------------

-->

The `topItem` property returns an optional value of type `Element`.
If the stack is empty, `topItem` returns `nil`;
if the stack isn't empty, `topItem` returns the final item in the `items` array.

Note that this extension doesn't define a type parameter list.
Instead, the `Stack` type's existing type parameter name, `Element`,
is used within the extension to indicate the optional type of
the `topItem` computed property.

The `topItem` computed property can now be used with any `Stack` instance
to access and query its top item without removing it.

// ----------------------------------------

<!--
  - test: `genericStack`
```

[swift_111] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Type Constraints

Sumber: ht

Type Constraints
The `swapTwoValues(_:_:)` function and the `Stack` type can work with any type.
However, it's sometimes useful to enforce
certain *type constraints* on the types that can be used with
generic functions and generic types.
Type constraints specify that a type parameter must
inherit from a specific class,
or conform to a particular protocol or protocol composition.
For example,
Swift's `Dictionary` type places a limitation on
the types that can be used as keys for a dictionary.
As described in ,
the type of a dictionary's keys must be *hashable*.
That is, it must provide a way to make itself uniquely representable.
`Dictionary` needs its keys to be hashable so that it can
check whether it already contains a value for a particular key.
Without this requirement, `Dictionary` couldn't tell
whether it should insert or replace a value for a particular key,
nor would it be able to find a value for a given key that's already in the dictionary.
This requirement is enforced by a type constraint on the key type for `Dictionary`,
which specifies that the key type must conform to the `Hashable` protocol,
a special protocol defined in the Swift standard library.
All of Swift's basic types (such as `String`, `Int`, `Double`, and `Bool`)
are hashable by default.
For information about
making your own custom types conform to the `Hashable` protocol,

see Conforming to the Hashable Protocol.
You can define your own type constraints when creating custom generic types,
and these constraints provide much of the power of generic programming.
Abstract concepts like `Hashable`
characterize types in terms of their conceptual characteristics,
rather than their concrete type.

[swift_112] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Type Constraint Syntax

Sumber: ht

Type Constraint Syntax
You write type constraints by placing a single class or protocol constraint
after a type parameter's name, separated by a colon,
as part of the type parameter list.
The basic syntax for type constraints on a generic function is shown below
(although the syntax is the same for generic types):
The hypothetical function above has two type parameters.
The first type parameter, `T`, has a type constraint
that requires `T` to be a subclass of `SomeClass`.
The second type parameter, `U`, has a type constraint
that requires `U` to conform to the protocol `SomeProtocol`.

Contoh Kode / Perintah CLI:

```
func someFunction<T: SomeClass, U: SomeProtocol>(someT: T, someU: U) {
    // function body goes here
}
```

[swift_113] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Type Constraints in Action

Sumber: ht

Type Constraints in Action
Here's a nongeneric function called `findIndex(ofString:in:)`,
which is given a `String` value to find
and an array of `String` values within which to find it.
The `findIndex(ofString:in:)` function returns an optional `Int` value,
which will be the index of the first matching string in the array if it's found,
or `nil` if the string can't be found:
The `findIndex(ofString:in:)` function can be used to find a string value in an array of strings:
The principle of finding the index of a value in an array isn't useful only for strings, however.
You can write the same functionality as a generic function
by replacing any mention of strings with values of some type `T` instead.
Here's how you might expect a generic version of `findIndex(ofString:in:)`,
called `findIndex(of:in:)`, to be written.
Note that the return type of this function is still `Int?`,
because the function returns an optional index number,
not an optional value from the array.
Be warned, though --- this function doesn't compile,
for reasons explained after the example:
This function doesn't compile as written above.

The problem lies with the equality check, "`if value == valueToFind`".
Not every type in Swift can be compared with the equal to operator (`==`).
If you create your own class or structure to represent a complex data model, for example,
then the meaning of "equal to" for that class or structure
isn't something that Swift can guess for you.
Because of this, it isn't possible to guarantee that this code will work
for *every* possible type `T`,
and an appropriate error is reported when you try to compile the code.
All is not lost, however.
The Swift standard library defines a protocol called `Equatable`,
which requires any conforming type to implement
the equal to opera

Contoh Kode / Perintah CLI:

```
func findIndex(ofString valueToFind: String, in array: [String]) -> Int? {
    for (index, value) in array.enumerated() {
        if value == valueToFind {
            return index
        }
    }
    return nil
}

// ----------------------------------------

-->

The `findIndex(ofString:in:)` function can be used to find a string value in an array of strings:

// ----------------------------------------

<!--
  - test: `typeConstraints`
```

[swift_114] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Associated Types

Sumber: ht

Associated Types
When defining a protocol,
it's sometimes useful to declare one or more associated types
as part of the protocol's definition.
An *associated type* gives a placeholder name
to a type that's used as part of the protocol.
The actual type to use for that associated type
isn't specified until the protocol is adopted.
Associated types are specified with the `associatedtype` keyword.

[swift_115] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Associated Types in Action

Sumber: ht

Associated Types in Action
Here's an example of a protocol called `Container`,
which declares an associated type called `Item`:
The `Container` protocol defines three required capabilities
that any container must provide:
- It must be possible to add a new item to the container with an `append(_:)` method.
- It must be possible to access a count of the items in the container
through a `count` property that returns an `Int` value.
- It must be possible to retrieve each item in the container with a subscript
that takes an `Int` index value.
This protocol doesn't specify how the items in the container should be stored
or what type they're allowed to be.
The protocol only specifies the three bits of functionality
that any type must provide in order to be considered a `Container`.
A conforming type can provide additional functionality,
as long as it satisfies these three requirements.
Any type that conforms to the `Container` protocol must be able to specify
the type of values it stores.
Specifically, it must ensure that only items of the right type
are added to the container,
and it must be clear about the type of the items returned by its subscript.
To define these requirements,
the `Container` protocol needs a way to refer to
the type of the elements that a container will hold,
without knowing what that type is for a specific container.
The `Container` protocol needs to specify that
any value passed to the `append(_:)` method
must have the same type as the container's element type,
and that the value returned by the container's subscript
will be of the same type as the container's element type.
To achieve this,
the `Container` protocol declares an associated type called `Item`,
written as `associatedtype Item`.
The protocol doesn't define what `Item` is ---
that info

Contoh Kode / Perintah CLI:

```
protocol Container {
    associatedtype Item
    mutating func append(_ item: Item)
    var count: Int { get }
    subscript(i: Int) -> Item { get }
}

// ----------------------------------------

-->

The `Container` protocol defines three required capabilities
that any container must provide:
```

```
- It must be possible to add a new item to the container with an `append(_:)` method.
- It must be possible to access a count of the items in the container
  through a `count` property that returns an `Int` value.
- It must be possible to retrieve each item in the container with a subscript
  that takes an `Int` index value.

This protocol doesn't specify how the items in the container should be stored
or what type they're allowed to be.
The protocol only specifies the three bits of functionality
that any type must provide in order to be considered a `Container`.
A conforming type can provide additional functionality,
as long as it satisfies these three requirements.

Any type that conforms to the `Container` protocol must be able to specify
the type of values it stores.
Specifically, it must ensure that only items of the right type
are added to the container,
and it must be clear about the type of the items returned by its subscript.

To define these requirements,
the `Container` protocol needs a way to refer to
the type of the elements that a container will hold,
without knowing what that type is for a specific container.
The `Container` protocol needs to specify that
any value passed to the `append(_:)` method
must have the same type as the container's element type,
and that the value returned by the container's subscript
will be of the same type as the container's element type.

To achieve this,
the `Container` protocol declares an associated type called `Item`,
written as  `associatedtype Item`.
The protocol doesn't define what `Item` is ---
that information is left for any conforming type to provide.
Nonetheless, the `Item` alias provides a way to refer to
the type of the items in a `Container`,
and to define a type for use with the `append(_:)` method and subscript,
to ensure that the expected behavior of any `Container` is enforced.

Here's a version of the nongeneric `IntStack` type
from <doc:Generics#Generic-Types> above,
adapted to conform to the `Container` protocol:

// ----------------------------------------

<!--
  - test: `associatedTypes`
```

[swift_116] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Extending an Existing Type to Specify an Associated Type

Sumber: ht

Extending an Existing Type to Specify an Associated Type
You can extend an existing type to add conformance to a protocol,
as described in .
This includes a protocol with an associated type.
Swift's `Array` type already provides an `append(_:)` method,

a `count` property, and a subscript with an `Int` index to retrieve its elements.
These three capabilities match the requirements of the `Container` protocol.
This means that you can extend `Array` to conform to the `Container` protocol
simply by declaring that `Array` adopts the protocol.
You do this with an empty extension,
as described in :
Array's existing `append(_:)` method and subscript enable Swift to infer
the appropriate type to use for `Item`,
just as for the generic `Stack` type above.
After defining this extension, you can use any `Array` as a `Container`.

Contoh Kode / Perintah CLI:

```
extension Array: Container {}
```

[swift_117] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Adding Constraints to an Associated Type

Sumber: ht

Adding Constraints to an Associated Type
You can add type constraints to an associated type in a protocol
to require that conforming types satisfy those constraints.
For example,
the following code defines a version of `Container`
that requires the items in the container to be equatable.
To conform to this version of `Container`,
the container's `Item` type has to conform to the `Equatable` protocol.

Contoh Kode / Perintah CLI:

```
protocol Container {
    associatedtype Item: Equatable
    mutating func append(_ item: Item)
    var count: Int { get }
    subscript(i: Int) -> Item { get }
}
```

[swift_118] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Using a Protocol in Its Associated Type's Constraints

Sumber: ht

Using a Protocol in Its Associated Type's Constraints
A protocol can appear as part of its own requirements.
For example,
here's a protocol that refines the `Container` protocol,
adding the requirement of a `suffix(_:)` method.
The `suffix(_:)` method
returns a given number of elements from the end of the container,
storing them in an instance of the `Suffix` type.
In this protocol,
`Suffix` is an associated type,
like the `Item` type in the `Container` example above.
`Suffix` has two constraints:
It must conform to the `SuffixableContainer` protocol

(the protocol currently being defined),
and its `Item` type must be the same
as the container's `Item` type.
The constraint on `Item` is a generic `where` clause,
which is discussed in below.
Here's an extension of the `Stack` type
from above
that adds conformance to the `SuffixableContainer` protocol:
In the example above,
the `Suffix` associated type for `Stack` is also `Stack`,
so the suffix operation on `Stack` returns another `Stack`.
Alternatively,
a type that conforms to `SuffixableContainer`
can have a `Suffix` type that's different from itself ---
meaning the suffix operation can return a different type.
For example,
here's an extension to the nongeneric `IntStack` type
that adds `SuffixableContainer` conformance,
using `Stack` as its suffix type instead of `IntStack`:

Contoh Kode / Perintah CLI:

```
protocol SuffixableContainer: Container {
    associatedtype Suffix: SuffixableContainer where Suffix.Item == Item
    func suffix(_ size: Int) -> Suffix
}

// ----------------------------------------

-->

In this protocol,
`Suffix` is an associated type,
like the `Item` type in the `Container` example above.
`Suffix` has two constraints:
It must conform to the `SuffixableContainer` protocol
(the protocol currently being defined),
and its `Item` type must be the same
as the container's `Item` type.
The constraint on `Item` is a generic `where` clause,
which is discussed in <doc:Generics#Associated-Types-with-a-Generic-Where-Clause> below.

Here's an extension of the `Stack` type
from <doc:Generics#Generic-Types> above
that adds conformance to the `SuffixableContainer` protocol:

// ----------------------------------------

<!--
  - test: `associatedTypes`
```

[swift_119] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Generic Where Clauses

Sumber: ht

Generic Where Clauses
Type constraints, as described in ,
enable you to define requirements on the type parameters associated with
a generic function, subscript, or type.
It can also be useful to define requirements for associated types.
You do this by defining a *generic where clause*.
A generic `where` clause enables you to require that
an associated type must conform to a certain protocol,
or that certain type parameters and associated types must be the same.
A generic `where` clause starts with the `where` keyword,
followed by constraints for associated types
or equality relationships between types and associated types.
You write a generic `where` clause right before the opening curly brace
of a type or function's body.
The example below defines a generic function called `allItemsMatch`,
which checks to see if two `Container` instances contain
the same items in the same order.
The function returns a Boolean value of `true` if all items match
and a value of `false` if they don't.
The two containers to be checked don't have to be
the same type of container (although they can be),
but they do have to hold the same type of items.
This requirement is expressed through a combination of type constraints
and a generic `where` clause:
This function takes two arguments called
`someContainer` and `anotherContainer`.
The `someContainer` argument is of type `C1`,
and the `anotherContainer` argument is of type `C2`.
Both `C1` and `C2` are type parameters
for two container types to be determined when the function is called.
The following requirements are placed on the function's two type parameters:
- `C1` must conform to the `Container` protocol (written as `C1: Container`).
- `C2` must also conform to the `Container` protocol (written as `C2: Container`).
- The `Item` for `C1

Contoh Kode / Perintah CLI:

```
func allItemsMatch<C1: Container, C2: Container>
        (_ someContainer: C1, _ anotherContainer: C2) -> Bool
        where C1.Item == C2.Item, C1.Item: Equatable {

    // Check that both containers contain the same number of items.
    if someContainer.count != anotherContainer.count {
        return false
    }

    // Check each pair of items to see if they're equivalent.
    for i in 0..<someContainer.count {
        if someContainer[i] != anotherContainer[i] {
            return false
        }
```

```
    }

    // All items match, so return true.
    return true
}

// ----------------------------------------

-->

This function takes two arguments called
`someContainer` and `anotherContainer`.
The `someContainer` argument is of type `C1`,
and the `anotherContainer` argument is of type `C2`.
Both `C1` and `C2` are type parameters
for two container types to be determined when the function is called.

The following requirements are placed on the function's two type parameters:

- `C1` must conform to the `Container` protocol (written as `C1: Container`).
- `C2` must also conform to the `Container` protocol (written as `C2: Container`).
- The `Item` for `C1` must be the same as the `Item` for `C2`
  (written as `C1.Item == C2.Item`).
- The `Item` for `C1` must conform to the `Equatable` protocol
  (written as `C1.Item: Equatable`).

The first and second requirements are defined in the function's type parameter list,
and the third and fourth requirements are defined in the function's generic `where` clause.

These requirements mean:

- `someContainer` is a container of type `C1`.
- `anotherContainer` is a container of type `C2`.
- `someContainer` and `anotherContainer` contain the same type of items.
- The items in `someContainer` can be checked with the not equal operator (`!=`)
  to see if they're different from each other.

The third and fourth requirements combine to mean that
the items in `anotherContainer` can *also* be checked with the `!=` operator,
because they're exactly the same type as the items in `someContainer`.

These requirements enable the `allItemsMatch(_:_:)` function to compare the two containers,
even if they're of a different container type.

The `allItemsMatch(_:_:)` function starts by checking that
both containers contain the same number of items.
If they contain a different number of items, there's no way that they can match,
and the function returns `false`.

After making this check, the function iterates over all of the items in `someContainer`
with a `for`-`in` loop and the half-open range operator (`..<`).
For each item, the function checks whether the item from `someContainer` isn't equal to
the corresponding item in `anotherContainer`.
If the two items aren't equal, then the two containers don't match,
and the function returns `false`.

If the loop finishes without finding a mismatch,
the two containers match, and the function returns `true`.

Here's how the `allItemsMatch(_:_:)` function looks in action:

// ----------------------------------------
```

```
<!--
  - test: `associatedTypes`
```

[swift_120] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Extensions with a Generic Where Clause

Sumber: ht

Extensions with a Generic Where Clause
You can also use a generic `where` clause as part of an extension.
The example below
extends the generic `Stack` structure from the previous examples
to add an `isTop(_:)` method.
This new `isTop(_:)` method
first checks that the stack isn't empty,
and then compares the given item
against the stack's topmost item.
If you tried to do this without a generic `where` clause,
you would have a problem:
The implementation of `isTop(_:)` uses the `==` operator,
but the definition of `Stack` doesn't require
its items to be equatable,
so using the `==` operator results in a compile-time error.
Using a generic `where` clause
lets you add a new requirement to the extension,
so that the extension adds the `isTop(_:)` method
only when the items in the stack are equatable.
Here's how the `isTop(_:)` method looks in action:
If you try to call the `isTop(_:)` method
on a stack whose elements aren't equatable,
you'll get a compile-time error.
You can use a generic `where` clause with extensions to a protocol.
The example below extends the `Container` protocol from the previous examples
to add a `startsWith(_:)` method.
The `startsWith(_:)` method
first makes sure that the container has at least one item,
and then it checks
whether the first item in the container matches the given item.
This new `startsWith(_:)` method
can be used with any type that conforms to the `Container` protocol,
including the stacks and arrays used above,
as long as the container's items are equatable.
The generic `where` clause in the example above
requires `Item` to conform to a protocol,
but you can also write a generic `where` clauses that require `Item`
to be a specific type.
For example:
This example adds an `average()` method
to containers whose `Item` type is `Doub

Contoh Kode / Perintah CLI:

```
extension Stack where Element: Equatable {
```

```
    func isTop(_ item: Element) -> Bool {
        guard let topItem = items.last else {
            return false
        }
        return topItem == item
    }
}

// ----------------------------------------

-->

This new `isTop(_:)` method
first checks that the stack isn't empty,
and then compares the given item
against the stack's topmost item.
If you tried to do this without a generic `where` clause,
you would have a problem:
The implementation of `isTop(_:)` uses the `==` operator,
but the definition of `Stack` doesn't require
its items to be equatable,
so using the `==` operator results in a compile-time error.
Using a generic `where` clause
lets you add a new requirement to the extension,
so that the extension adds the `isTop(_:)` method
only when the items in the stack are equatable.

Here's how the `isTop(_:)` method looks in action:

// ----------------------------------------

<!--
  - test: `associatedTypes`
```

[swift_121] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Contextual Where Clauses

Sumber: ht

Contextual Where Clauses
You can write a generic `where` clause
as part of a declaration that doesn't have its own generic type constraints,
when you're already working in the context of generic types.
For example,
you can write a generic `where` clause
on a subscript of a generic type
or on a method in an extension to a generic type.
The `Container` structure is generic,
and the `where` clauses in the example below
specify what type constraints have to be satisfied
to make these new methods available on a container.
This example
adds an `average()` method to `Container` when the items are integers,
and it adds an `endsWith(_:)` method when the items are equatable.
Both functions include a generic `where` clause
that adds type constraints to the generic `Item` type parameter
from the original declaration of `Container`.

If you want to write this code without using contextual `where` clauses,
you write two extensions,
one for each generic `where` clause.
The example above and the example below have the same behavior.
In the version of this example that uses contextual `where` clauses,
the implementation of `average()` and `endsWith(_:)`
are both in the same extension
because each method's generic `where` clause
states the requirements that need to be satisfied
to make that method available.
Moving those requirements to the extensions' generic `where` clauses
makes the methods available in the same situations,
but requires one extension per requirement.

Contoh Kode / Perintah CLI:

```
extension Container {
    func average() -> Double where Item == Int {
        var sum = 0.0
        for index in 0..<count {
            sum += Double(self[index])
        }
        return sum / Double(count)
    }
    func endsWith(_ item: Item) -> Bool where Item: Equatable {
        return count >= 1 && self[count-1] == item
    }
}
let numbers = [1260, 1200, 98, 37]
print(numbers.average())
// Prints "648.75".
print(numbers.endsWith(37))
// Prints "true".

// ----------------------------------------

-->

This example
adds an `average()` method to `Container` when the items are integers,
and it adds an `endsWith(_:)` method when the items are equatable.
Both functions include a generic `where` clause
that adds type constraints to the generic `Item` type parameter
from the original declaration of `Container`.

If you want to write this code without using contextual `where` clauses,
you write two extensions,
one for each generic `where` clause.
The example above and the example below have the same behavior.

// ----------------------------------------

<!--
  - test: `associatedTypes-err`
```

[swift_122] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Associated Types with a Generic Where Clause

Sumber: ht

Associated Types with a Generic Where Clause
You can include a generic `where` clause on an associated type.
For example, suppose you want to make a version of `Container`
that includes an iterator,
like what the `Sequence` protocol uses in the Swift standard library.
Here's how you write that:
The generic `where` clause on `Iterator` requires that
the iterator must traverse over elements
of the same item type as the container's items,
regardless of the iterator's type.
The `makeIterator()` function provides access to a container's iterator.
For a protocol that inherits from another protocol,
you add a constraint to an inherited associated type
by including the generic `where` clause in the protocol declaration.
For example, the following code
declares a `ComparableContainer` protocol
that requires `Item` to conform to `Comparable`:

Contoh Kode / Perintah CLI:

```
protocol Container {
    associatedtype Item
    mutating func append(_ item: Item)
    var count: Int { get }
    subscript(i: Int) -> Item { get }

    associatedtype Iterator: IteratorProtocol where Iterator.Element == Item
    func makeIterator() -> Iterator
}

// ----------------------------------------

-->

<!--
  Adding makeIterator() to Container lets it conform to Sequence,
  although we don't call that out here.
-->

The generic `where` clause on `Iterator` requires that
the iterator must traverse over elements
of the same item type as the container's items,
regardless of the iterator's type.
The `makeIterator()` function provides access to a container's iterator.

<!--
  This example requires SE-0157 Recursive protocol constraints
  which is tracked by rdar://20531108

   that accepts a ranged of indexes it its subscript
   and returns a subcontainer ---
   similar to how ``Collection`` works in the Swift standard library.

   .. testcode:: associatedTypes-subcontainer
```

```
      -> protocol Container {
            associatedtype Item
            associatedtype SubContainer: Container where SubContainer.Item == Item

            mutating func append(_ item: Item)
            var count: Int { get }
            subscript(i: Int) -> Item { get }
            subscript(range: Range<Int>) -> SubContainer { get }
         }

   The generic ``where`` clause on ``SubContainer`` requires that
   the subcontainer must have the same item type as the container has,
   regardless of what type the subcontainer is.
   The original container and the subcontainer
   could be represented by the same type
   or by different types.
   The new subscript that accepts a range
   uses this new associated type as its return value.
-->

For a protocol that inherits from another protocol,
you add a constraint to an inherited associated type
by including the generic `where` clause in the protocol declaration.
For example, the following code
declares a `ComparableContainer` protocol
that requires `Item` to conform to `Comparable`:

// ----------------------------------------

<!--
  - test: `associatedTypes`
```

[swift_123] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Generic Subscripts

Sumber: ht

Generic Subscripts
Subscripts can be generic,
and they can include generic `where` clauses.
You write the placeholder type name inside angle brackets after `subscript`,
and you write a generic `where` clause right before the opening curly brace
of the subscript's body.
For example:
This extension to the `Container` protocol
adds a subscript that takes a sequence of indices
and returns an array containing the items at each given index.
This generic subscript is constrained as follows:
- The generic parameter `Indices` in angle brackets
has to be a type that conforms to the `Sequence` protocol
from the Swift standard library.
- The subscript takes a single parameter, `indices`,
which is an instance of that `Indices` type.
- The generic `where` clause requires
that the iterator for the sequence
must traverse over elements of type `Int`.
This ensures that the indices in the sequence

are the same type as the indices used for a container.
Taken together, these constraints mean that
the value passed for the `indices` parameter
is a sequence of integers.

Contoh Kode / Perintah CLI:

```
extension Container {
    subscript<Indices: Sequence>(indices: Indices) -> [Item]
            where Indices.Iterator.Element == Int {
        var result: [Item] = []
        for index in indices {
            result.append(self[index])
        }
        return result
    }
}

// ----------------------------------------

-->

<!--
  - test: `genericSubscript`
```

[swift_124] Swift - Generics & Associated Types

Generics (Type Constraints, Associated Types, where clauses, Generic Subscripts): Implicit Constraints

Sumber: ht

Implicit Constraints
In addition to constraints you write explicitly,
many places in your generic code also implicitly require
conformance to some very common protocols like [`Copyable`][].
These generic constraints that you don't have to write
are known as *implicit constraints*.
For example, both of the following function declarations
require `MyType` to be copyable:
[`Copyable`]: https://developer.apple.com/documentation/swift/copyable
In the code above,
the first declaration has an implicit constraint,
and the second version lists the conformance explicitly.
In most code,
types also implicitly conform to these common protocols.
For more information,
see .
Because most types in Swift conform to these protocols,
writing them almost everywhere would be repetitive.
Instead, by marking only the exceptions,
you call out the places that omit a common constraint.
To suppress an implicit constraint,
write the protocol name with a tilde (`~`) in front of it.
You can read `~Copyable` as "maybe copyable" ---
this suppressed constraint allows
both copyable and noncopyable types in this position.
Note that `~Copyable` doesn't *require* the type to be noncopyable.

For example:
In the code above,
the function `f()` implicitly requires `MyType` to be copyable.
Within the function body,
the value of `x` is copied to `x1` and `x2` in the assignment.
In contrast, `g()` suppresses the implicit constraint on `AnotherType`,
which allows you to pass either a copyable or noncopyable value.
Within the function body,
you can't copy the value of `y`
because `AnotherType` might be noncopyable.
Assignment consumes the value of `y`
and it's an error to consume that value more than once.
Noncopyable values like `y`
must be passed as in-out, borrowing, or consuming parameters ---
for more inform

Contoh Kode / Perintah CLI:

```
function someFunction<MyType> { ... }
function someFunction<MyType: Copyable> { ... }

// ----------------------------------------

func f<MyType>(x: inout MyType) {
    let x1 = x  // The value of x1 is a copy of x's value.
    let x2 = x  // The value of x2 is a copy of x's value.
}

func g<AnotherType: ~Copyable>(y: inout AnotherType) {
    let y1 = y  // The assignment consumes y's value.
    let y2 = y  // Error: Value consumed more than once.
}
```

[swift_125] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Overview

Sumber: ht

Error Handling
Respond to and recover from errors.
Error handling is the process of responding to
and recovering from error conditions in your program.
Swift provides first-class support for
throwing, catching, propagating, and manipulating
recoverable errors at runtime.
Some operations
aren't guaranteed to always complete execution or produce a useful output.
Optionals are used to represent the absence of a value,
but when an operation fails,
it's often useful to understand what caused the failure,
so that your code can respond accordingly.
As an example, consider the task of reading and processing data from a file on disk.
There are a number of ways this task can fail, including
the file not existing at the specified path,
the file not having read permissions, or
the file not being encoded in a compatible format.
Distinguishing among these different situations

allows a program to resolve some errors
and to communicate to the user any errors it can't resolve.
> Note: Error handling in Swift interoperates with error handling patterns
> that use the `NSError` class in Cocoa and Objective-C.
> For more information about this class,
> see Handling Cocoa Errors in Swift.

[swift_126] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Representing and Throwing Errors

Sumber: ht

Representing and Throwing Errors
In Swift, errors are represented by
values of types that conform to the `Error` protocol.
This empty protocol indicates that a type
can be used for error handling.
Swift enumerations are particularly well suited to modeling
a group of related error conditions,
with associated values allowing for additional information
about the nature of an error to be communicated.
For example, here's how you might represent the error conditions
of operating a vending machine inside a game:
Throwing an error lets you indicate that something unexpected happened
and the normal flow of execution can't continue.
You use a `throw` statement to throw an error.
For example,
the following code throws an error to indicate
that five additional coins are needed by the vending machine:

Contoh Kode / Perintah CLI:

```
enum VendingMachineError: Error {
    case invalidSelection
    case insufficientFunds(coinsNeeded: Int)
    case outOfStock
}

// ----------------------------------------

-->

Throwing an error lets you indicate that something unexpected happened
and the normal flow of execution can't continue.
You use a `throw` statement to throw an error.
For example,
the following code throws an error to indicate
that five additional coins are needed by the vending machine:

// ----------------------------------------

<!--
  - test: `throw-enum-error`
```

[swift_127] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Handling Errors

Sumber: ht

Handling Errors
When an error is thrown,
some surrounding piece of code must be responsible
for handling the error ---
for example, by correcting the problem,
trying an alternative approach,
or informing the user of the failure.
There are four ways to handle errors in Swift.
You can propagate the error from a function to the code that calls that function,
handle the error using a `do`-`catch` statement,
handle the error as an optional value,
or assert that the error will not occur.
Each approach is described in a section below.
When a function throws an error,
it changes the flow of your program,
so it's important that you can quickly identify places in your code that can throw errors.
To identify these places in your code, write the `try` keyword ---
or the `try?` or `try!` variation ---
before a piece of code that calls a function, method, or initializer that can throw an error.
These keywords are described in the sections below.
> Note: Error handling in Swift resembles exception handling in other languages,
> with the use of the `try`, `catch` and `throw` keywords.
> Unlike exception handling in many languages ---
> including Objective-C ---
> error handling in Swift doesn't involve unwinding the call stack,
> a process that can be computationally expensive.
> As such, the performance characteristics
> of a `throw` statement
> are comparable to those of a `return` statement.

[swift_128] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Propagating Errors Using Throwing Functions

Sumber: ht

Propagating Errors Using Throwing Functions
To indicate that a function, method, or initializer can throw an error,
you write the `throws` keyword in the function's declaration
after its parameters.
A function marked with `throws` is called a *throwing function*.
If the function specifies a return type,
you write the `throws` keyword before the return arrow (`->`).
A throwing function propagates errors that are thrown inside of it
to the scope from which it's called.
> Note: Only throwing functions can propagate errors.
> Any errors thrown inside a nonthrowing function
> must be handled inside the function.
In the example below,
the `VendingMachine` class has a `vend(itemNamed:)` method

that throws an appropriate `VendingMachineError`
if the requested item isn't available,
is out of stock,
or has a cost that exceeds the current deposited amount:
The implementation of the `vend(itemNamed:)` method
uses `guard` statements to exit the method early and throw appropriate errors
if any of the requirements for purchasing a snack aren't met.
Because a `throw` statement immediately transfers program control,
an item will be vended only if all of these requirements are met.
Because the `vend(itemNamed:)` method propagates any errors it throws,
any code that calls this method must either handle the errors ---
using a `do`-`catch` statement, `try?`, or `try!` ---
or continue to propagate them.
For example,
the `buyFavoriteSnack(person:vendingMachine:)` in the example below
is also a throwing function,
and any errors that the `vend(itemNamed:)` method throws will
propagate up to the point where the `buyFavoriteSnack(person:vendingMachine:)` function is called.
In this example,
the `buyFavoriteSnack(person: vendingMachine:)` function looks up a given person's favorite snack
and t

Contoh Kode / Perintah CLI:

```
func canThrowErrors() throws -> String

func cannotThrowErrors() -> String

// ----------------------------------------

-->

<!--
  - test: `throwing-function-cant-overload-nonthrowing`

// ----------------------------------------

-->

<!--
  - test: `throwing-parameter-can-overload-nonthrowing`
```

[swift_129] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Handling Errors Using Do-Catch

Sumber: ht

Handling Errors Using Do-Catch
You use a `do`-`catch` statement to handle errors
by running a block of code.
If an error is thrown by the code in the `do` clause,
it's matched against the `catch` clauses
to determine which one of them can handle the error.
Here is the general form of a `do`-`catch` statement:
You write a pattern after `catch` to indicate what errors
that clause can handle.

If a `catch` clause doesn't have a pattern,
the clause matches any error
and binds the error to a local constant named `error`.
For more information about pattern matching,
see .
For example, the following code matches against all three cases
of the `VendingMachineError` enumeration.
In the above example,
the `buyFavoriteSnack(person:vendingMachine:)` function is called in a `try` expression,
because it can throw an error.
If an error is thrown,
execution immediately transfers to the `catch` clauses,
which decide whether to allow propagation to continue.
If no pattern is matched, the error gets caught by the final `catch`
clause and is bound to a local `error` constant.
If no error is thrown,
the remaining statements in the `do` statement are executed.
The `catch` clauses don't have to handle every possible error
that the code in the `do` clause can throw.
If none of the `catch` clauses handle the error,
the error propagates to the surrounding scope.
However, the propagated error
must be handled by *some* surrounding scope.
In a nonthrowing function,
an enclosing `do`-`catch` statement
must handle the error.
In a throwing function,
either an enclosing `do`-`catch` statement
or the caller
must handle the error.
If the error propagates to the top-level scope
without being handled,
you'll get a runtime error.
For example, the above example can be written so any
error that isn't a `Ve

Contoh Kode / Perintah CLI:

```
do {
    try <#expression#>
    <#statements#>
} catch <#pattern 1#> {
    <#statements#>
} catch <#pattern 2#> where <#condition#> {
    <#statements#>
} catch <#pattern 3#>, <#pattern 4#> where <#condition#> {
    <#statements#>
} catch {
    <#statements#>
}

// ----------------------------------------

var vendingMachine = VendingMachine()
vendingMachine.coinsDeposited = 8
do {
    try buyFavoriteSnack(person: "Alice", vendingMachine: vendingMachine)
```

```
    print("Success! Yum.")
} catch VendingMachineError.invalidSelection {
    print("Invalid Selection.")
} catch VendingMachineError.outOfStock {
    print("Out of Stock.")
} catch VendingMachineError.insufficientFunds(let coinsNeeded) {
    print("Insufficient funds. Please insert an additional \(coinsNeeded) coins.")
} catch {
    print("Unexpected error: \(error).")
}
// Prints "Insufficient funds. Please insert an additional 2 coins."

// ----------------------------------------

-->

In the above example,
the `buyFavoriteSnack(person:vendingMachine:)` function is called in a `try` expression,
because it can throw an error.
If an error is thrown,
execution immediately transfers to the `catch` clauses,
which decide whether to allow propagation to continue.
If no pattern is matched, the error gets caught by the final `catch`
clause and is bound to a local `error` constant.
If no error is thrown,
the remaining statements in the `do` statement are executed.

The `catch` clauses don't have to handle every possible error
that the code in the `do` clause can throw.
If none of the `catch` clauses handle the error,
the error propagates to the surrounding scope.
However, the propagated error
must be handled by *some* surrounding scope.
In a nonthrowing function,
an enclosing `do`-`catch` statement
must handle the error.
In a throwing function,
either an enclosing `do`-`catch` statement
or the caller
must handle the error.
If the error propagates to the top-level scope
without being handled,
you'll get a runtime error.

For example, the above example can be written so any
error that isn't a `VendingMachineError` is instead
caught by the calling function:
```

[swift_130] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Converting Errors to Optional Values

Sumber: ht

Converting Errors to Optional Values

You use `try?` to handle an error by converting it to an optional value.
If an error is thrown while evaluating the `try?` expression,
the value of the expression is `nil`.
For example,
in the following code `x` and `y` have the same value and behavior:

If `someThrowingFunction()` throws an error,
the value of `x` and `y` is `nil`.
Otherwise, the value of `x` and `y` is the value that the function returned.
Note that `x` and `y` are an optional of whatever type `someThrowingFunction()` returns.
Here the function returns an integer, so `x` and `y` are optional integers.
Using `try?` lets you write concise error handling code
when you want to handle all errors in the same way.
For example,
the following code
uses several approaches to fetch data,
or returns `nil` if all of the approaches fail.

Contoh Kode / Perintah CLI:

```
func someThrowingFunction() throws -> Int {
    // ...
}

let x = try? someThrowingFunction()

let y: Int?
do {
    y = try someThrowingFunction()
} catch {
    y = nil
}

// ----------------------------------------

-->

If `someThrowingFunction()` throws an error,
the value of `x` and `y` is `nil`.
Otherwise, the value of `x` and `y` is the value that the function returned.
Note that `x` and `y` are an optional of whatever type `someThrowingFunction()` returns.
Here the function returns an integer, so `x` and `y` are optional integers.

Using `try?` lets you write concise error handling code
when you want to handle all errors in the same way.
For example,
the following code
uses several approaches to fetch data,
or returns `nil` if all of the approaches fail.

// ----------------------------------------

<!--
  - test: `optional-try-cached-data`
```

[swift_131] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Disabling Error Propagation

Sumber: ht

Disabling Error Propagation
Sometimes you know a throwing function or method
won't, in fact, throw an error at runtime.
On those occasions,
you can write `try!` before the expression to disable error propagation

and wrap the call in a runtime assertion that no error will be thrown.
If an error actually is thrown, you'll get a runtime error.
For example, the following code uses a `loadImage(atPath:)` function,
which loads the image resource at a given path
or throws an error if the image can't be loaded.
In this case, because the image is shipped with the application,
no error will be thrown at runtime,
so it's appropriate to disable error propagation.

Contoh Kode / Perintah CLI:

```
let photo = try! loadImage(atPath: "./Resources/John Appleseed.jpg")
```

[swift_132] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Specifying the Error Type

Sumber: ht

Specifying the Error Type
All of the examples above use the most common kind of error handling,
where the errors that your code throws
can be values of any type that conforms to the `Error` protocol.
This approach matches the reality that
you don't know ahead of time every error that could happen
while the code is running,
especially when propagating errors thrown somewhere else.
It also reflects the fact that errors can change over time.
New versions of a library ---
including libraries that your dependencies use ---
can throw new errors,
and the rich complexity of real-world user configurations
can expose failure modes that weren't visible during development or testing.
The error handling code in the examples above
always includes a default case to handle errors
that don't have a specific `catch` clause.
Most Swift code doesn't specify the type for the errors it throws.
However,
you might limit code to throwing errors of only one specific type
in the following special cases:
- When running code on an embedded system
that doesn't support dynamic allocation of memory.
Throwing an instance of `any Error` or another boxed protocol type
requires allocating memory at runtime to store the error.
In contrast,
throwing an error of a specific type
lets Swift avoid heap allocation for errors.
- When the errors are an implementation detail of some unit of code,
like a library,
and aren't part of the interface to that code.
Because the errors come from only the library,
and not from other dependencies or the library's clients,
you can make an exhaustive list of all possible failures.
And because these errors are an implementation detail of the library,
they're always handled within that library.

- In code that only propagates errors described by generic parameters,
like a funct

Contoh Kode / Perintah CLI:

```
enum StatisticsError: Error {
    case noRatings
    case invalidRating(Int)
}

// ----------------------------------------

func summarize(_ ratings: [Int]) throws(StatisticsError) {
    guard !ratings.isEmpty else { throw .noRatings }

    var counts = [1: 0, 2: 0, 3: 0]
    for rating in ratings {
        guard rating > 0 && rating <= 3 else { throw .invalidRating(rating) }
        counts[rating]! += 1
    }

    print("*", counts[1]!, "-- **", counts[2]!, "-- ***", counts[3]!)
}

// ----------------------------------------

func someThrowingFunction() throws {
    let ratings = [1, 2, 3, 2, 2, 1]
    try summarize(ratings)
}
```

[swift_133] Swift - Error Handling

Error Handling (Representing Errors, throws, do-catch, try?, try!, defer): Specifying Cleanup Actions

Sumber: ht

Specifying Cleanup Actions
You use a `defer` statement to execute a set of statements
just before code execution leaves the current block of code.
This statement lets you do any necessary cleanup
that should be performed regardless
of *how* execution leaves the current block of code ---
whether it leaves because an error was thrown
or because of a statement such as `return` or `break`.
For example, you can use a `defer` statement
to ensure that file descriptors are closed
and manually allocated memory is freed.
A `defer` statement defers execution until the current scope is exited.
This statement consists of the `defer` keyword and the statements to be executed later.
The deferred statements may not contain any code
that would transfer control out of the statements,
such as a `break` or a `return` statement,
or by throwing an error.
Deferred actions are executed in the reverse of
the order that they're written in your source code.
That is, the code in the first `defer` statement executes last,
the code in the second `defer` statement executes second to last,
and so on.

The last `defer` statement in source code order executes first.
The above example uses a `defer` statement
to ensure that the `open(_:)` function
has a corresponding call to `close(_:)`.
You can use a `defer` statement
even when no error handling code is involved.
For more information,
see .

Contoh Kode / Perintah CLI:

```
func processFile(filename: String) throws {
    if exists(filename) {
        let file = open(filename)
        defer {
            close(file)
        }
        while let line = try file.readline() {
            // Work with the file.
        }
        // close(file) is called here, at the end of the scope.
    }
}
```

[swift_134] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Overview

Sumber: ht

Concurrency
Perform asynchronous operations.
Swift has built-in support for writing asynchronous and parallel code
in a structured way.
Asynchronous code can be suspended and resumed later,
although only one piece of the program executes at a time.
Suspending and resuming code in your program
lets it continue to make progress
on short-term operations like updating its UI
while continuing to work on long-running operations
like fetching data over the network or parsing files.
Parallel code means multiple pieces of code run simultaneously ---
for example, a computer with a four-core processor
can run four pieces of code at the same time,
with each core carrying out one of the tasks.
A program that uses parallel and asynchronous code
carries out multiple operations at a time,
and it suspends operations that are waiting for an external system.
The rest of this chapter uses the term *concurrency*
to refer to this common combination of asynchronous and parallel code.
The additional scheduling flexibility from parallel or asynchronous code
also comes with a cost of increased complexity.
When you write concurrent code,
you don't know ahead of time what code will run at the same time,
and you might not always know the order that code will run.
A common problem in concurrent code happens

when multiple pieces of code try to access
some piece of shared mutable state ---
this is known as a *data race*.
When you use the language-level support for concurrency,
Swift detects and prevents data races,
and most data races produce a compile-time error.
Some data races can't be detected until your code is running;
these data races terminate code execution.
You use actors and isolation to protect against data races,
as described in this chapter.
> Note: If you've written concurrent cod

Contoh Kode / Perintah CLI:

```
listPhotos(inGallery: "Summer Vacation") { photoNames in
    let sortedNames = photoNames.sorted()
    let name = sortedNames[0]
    downloadPhoto(named: name) { photo in
        show(photo)
    }
}
```

[swift_135] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Defining and Calling Asynchronous Functions

Sumber: ht

Defining and Calling Asynchronous Functions
An *asynchronous function* or *asynchronous method*
is a special kind of function or method
that can be suspended while it's partway through execution.
This is in contrast to ordinary, synchronous functions and methods,
which either run to completion, throw an error, or never return.
An asynchronous function or method still does one of those three things,
but it can also pause in the middle when it's waiting for something.
Inside the body of an asynchronous function or method,
you mark each of these places where execution can be suspended.
To indicate that a function or method is asynchronous,
you write the `async` keyword in its declaration after its parameters,
similar to how you use `throws` to mark a throwing function.
If the function or method returns a value,
you write `async` before the return arrow (`->`).
For example,
here's how you might fetch the names of photos in a gallery:
For a function or method that's both asynchronous and throwing,
you write `async` before `throws`.
When calling an asynchronous method,
execution suspends until that method returns.
You write `await` in front of the call
to mark the possible suspension point.
This is like writing `try` when calling a throwing function,
to mark the possible change to the program's flow if there's an error.
Inside an asynchronous method,
the flow of execution can be suspended
only when you call another asynchronous method ---

suspension is never implicit or preemptive ---
which means every possible suspension point is marked with `await`.
Marking all of the possible suspension points in your code
helps make concurrent code easier to read and understand.
For example,
the code below fetches the names of all the pictures in a gallery
and then shows the first pi

Contoh Kode / Perintah CLI:

```
func listPhotos(inGallery name: String) async -> [String] {
    let result = // ... some asynchronous networking code ...
    return result
}

// -----------------------------------------

-->

For a function or method that's both asynchronous and throwing,
you write `async` before `throws`.

<!--
  - test: `async-comes-before-throws`

// -----------------------------------------

-->

When calling an asynchronous method,
execution suspends until that method returns.
You write `await` in front of the call
to mark the possible suspension point.
This is like writing `try` when calling a throwing function,
to mark the possible change to the program's flow if there's an error.
Inside an asynchronous method,
the flow of execution can be suspended
*only* when you call another asynchronous method ---
suspension is never implicit or preemptive ---
which means every possible suspension point is marked with `await`.
Marking all of the possible suspension points in your code
helps make concurrent code easier to read and understand.

For example,
the code below fetches the names of all the pictures in a gallery
and then shows the first picture:
```

[swift_136] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Asynchronous Sequences

Sumber: ht

Asynchronous Sequences
The `listPhotos(inGallery:)` function in the previous section
asynchronously returns the whole array at once,
after all of the array's elements are ready.
Another approach
is to wait for one element of the collection at a time
using an *asynchronous sequence*.

Here's what iterating over an asynchronous sequence looks like:
Instead of using an ordinary `for`-`in` loop,
the example above writes `for` with `await` after it.
Like when you call an asynchronous function or method,
writing `await` indicates a possible suspension point.
A `for`-`await`-`in` loop potentially suspends execution
at the beginning of each iteration,
when it's waiting for the next element to be available.
In the same way that you can use your own types in a `for`-`in` loop
by adding conformance to the [`Sequence`][] protocol,
you can use your own types in a `for`-`await`-`in` loop
by adding conformance to the [`AsyncSequence`] protocol.
[`Sequence`]: https://developer.apple.com/documentation/swift/sequence
[`AsyncSequence`]: https://developer.apple.com/documentation/swift/asyncsequence

Contoh Kode / Perintah CLI:

```
import Foundation

let handle = FileHandle.standardInput
for try await line in handle.bytes.lines {
    print(line)
}
```

[swift_137] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Calling Asynchronous Functions in Parallel

Sumber: ht

Calling Asynchronous Functions in Parallel
Calling an asynchronous function with `await`
runs only one piece of code at a time.
While the asynchronous code is running,
the caller waits for that code to finish
before moving on to run the next line of code.
For example,
to fetch the first three photos from a gallery,
you could await three calls to the `downloadPhoto(named:)` function
as follows:
This approach has an important drawback:
Although the download is asynchronous
and lets other work happen while it progresses,
only one call to `downloadPhoto(named:)` runs at a time.
Each photo downloads completely before the next one starts downloading.
However, there's no need for these operations to wait ---
each photo can download independently, or even at the same time.
To call an asynchronous function
and let it run in parallel with code around it,
write `async` in front of `let` when you define a constant,
and then write `await` each time you use the constant.
In this example,
all three calls to `downloadPhoto(named:)` start
without waiting for the previous one to complete.
If there are enough system resources available, they can run at the same time.

None of these function calls are marked with `await`
because the code doesn't suspend to wait for the function's result.
Instead, execution continues
until the line where `photos` is defined ---
at that point, the program needs the results from these asynchronous calls,
so you write `await` to pause execution
until all three photos finish downloading.
Here's how you can think about the differences between these two approaches:
- Call asynchronous functions with `await`
when the code on the following lines depends on that function's result.
This creates work that is carried out sequentially.
- Call asynchronous functions wi

Contoh Kode / Perintah CLI:

```
let firstPhoto = await downloadPhoto(named: photoNames[0])
let secondPhoto = await downloadPhoto(named: photoNames[1])
let thirdPhoto = await downloadPhoto(named: photoNames[2])

let photos = [firstPhoto, secondPhoto, thirdPhoto]
show(photos)

// ----------------------------------------

-->

This approach has an important drawback:
Although the download is asynchronous
and lets other work happen while it progresses,
only one call to `downloadPhoto(named:)` runs at a time.
Each photo downloads completely before the next one starts downloading.
However, there's no need for these operations to wait ---
each photo can download independently, or even at the same time.

To call an asynchronous function
and let it run in parallel with code around it,
write `async` in front of `let` when you define a constant,
and then write `await` each time you use the constant.

// ----------------------------------------

<!--
  - test: `calling-with-async-let`
```

[swift_138] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Tasks and Task Groups

Sumber: ht

Tasks and Task Groups
A *task* is a unit of work
that can be run asynchronously as part of your program.
All asynchronous code runs as part of some task.
A task itself does only one thing at a time,
but when you create multiple tasks,
Swift can schedule them to run simultaneously.
The `async`-`let` syntax described in the previous section
implicitly creates a child task ---

this syntax works well when you already know
what tasks your program needs to run.
You can also create a task group
(an instance of [`TaskGroup`][])
and explicitly add child tasks to that group,
which gives you more control over priority and cancellation,
and lets you create a dynamic number of tasks.
[`TaskGroup`]: https://developer.apple.com/documentation/swift/taskgroup
Tasks are arranged in a hierarchy.
Each task in a given task group has the same parent task,
and each task can have child tasks.
Because of the explicit relationship between tasks and task groups,
this approach is called *structured concurrency*.
The explicit parent-child relationship between tasks has several advantages:
- In a parent task,
you can't forget to wait for its child tasks to complete.
- When setting a higher priority on a child task,
the parent task's priority is automatically escalated.
- When a parent task is canceled,
each of its child tasks is also automatically canceled.
- Task-local values propagate to child tasks efficiently and automatically.
Here's another version of the code to download photos
that handles any number of photos:
The code above creates a new task group,
and then creates child tasks
to download each photo in the gallery.
Swift runs as many of these tasks concurrently as conditions allow.
As soon as a child task finishes downloading a photo,
that photo is displayed.
There's no guarantee about

Contoh Kode / Perintah CLI:

```
await withTaskGroup(of: Data.self) { group in
    let photoNames = await listPhotos(inGallery: "Summer Vacation")
    for name in photoNames {
        group.addTask {
            return await downloadPhoto(named: name)
        }
    }

    for await photo in group {
        show(photo)
    }
}

// ----------------------------------------

let photos = await withTaskGroup(of: Data.self) { group in
    let photoNames = await listPhotos(inGallery: "Summer Vacation")
    for name in photoNames {
        group.addTask {
            return await downloadPhoto(named: name)
        }
    }

    var results: [Data] = []
    for await photo in group {
```

```
        results.append(photo)
    }

    return results
}
```

[swift_139] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Task Cancellation

Sumber: ht

Task Cancellation
Swift concurrency uses a cooperative cancellation model.
Each task checks whether it has been canceled
at the appropriate points in its execution,
and responds to cancellation appropriately.
Depending on what work the task is doing,
responding to cancellation usually means one of the following:
- Throwing an error like `CancellationError`
- Returning `nil` or an empty collection
- Returning the partially completed work
Downloading pictures could take a long time
if the pictures are large or the network is slow.
To let the user stop this work,
without waiting for all of the tasks to complete,
the tasks need to check for cancellation and stop running if they are canceled.
There are two ways a task can do this:
by calling the [`Task.checkCancellation()`][] type method,
or by reading the [`Task.isCancelled`][`Task.isCancelled` type] type property.
Calling `checkCancellation()` throws an error if the task is canceled;
a throwing task can propagate the error out of the task,
stopping all of the task's work.
This has the advantage of being simple to implement and understand.
For more flexibility, use the `isCancelled` property,
which lets you perform clean-up work as part of stopping the task,
like closing network connections and deleting temporary files.
[`Task.checkCancellation()`]: https://developer.apple.com/documentation/swift/task/checkcancellation()
[`Task.isCancelled` type]: https://developer.apple.com/documentation/swift/task/iscancelled-swift.type.property
The code above makes several changes from the previous version:
- Each task is added using the
[`TaskGroup.addTaskUnlessCancelled(priority:operation:)`][] method,
to avoid starting new work after cancellation.
- After each call to `addTaskUnlessCancelled(priority:operation:)`,
the code confirm

Contoh Kode / Perintah CLI:

```
let photos = await withTaskGroup { group in
    let photoNames = await listPhotos(inGallery: "Summer Vacation")
    for name in photoNames {
        let added = group.addTaskUnlessCancelled {
            Task.isCancelled ? nil : await downloadPhoto(named: name)
        }
        guard added else { break }
    }
```

```
    var results: [Data] = []
    for await photo in group {
        if let photo { results.append(photo) }
    }
    return results
}

// ----------------------------------------

let task = Task {
    await withTaskCancellationHandler {
        // ...
    } onCancel: {
        print("Canceled!")
    }
}

// ... some time later...
task.cancel()  // Prints "Canceled!"

// ----------------------------------------

func generateSlideshow(forGallery gallery: String) async {
    let photos = await listPhotos(inGallery: gallery)
    for photo in photos {
        // ... render a few seconds of video for this photo ...
        await Task.yield()
    }
}
```

[swift_140] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Unstructured Concurrency

Sumber: ht

Unstructured Concurrency
In addition to the structured approaches to concurrency
described in the previous sections,
Swift also supports unstructured concurrency.
Unlike tasks that are part of a task group,
an *unstructured task* doesn't have a parent task.
You have complete flexibility to manage unstructured tasks
in whatever way your program needs,
but you're also completely responsible for their correctness.
To create an unstructured task
that runs similarly to the surrounding code,
call the [`Task.init(name:priority:operation:)`][] initializer.
The new task defaults to running with
the same actor isolation, priority, and task-local state as the current task.
To create an unstructured task
that's more independent from the surrounding code,
known more specifically as a *detached task*,
call the [`Task.detached(name:priority:operation:)`][] static method.
The new task defaults to running without any actor isolation
and doesn't inherit the current task's priority or task-local state.
Both of these operations return a task that you can interact with ---

for example, to wait for its result or to cancel it.
For more information about managing detached tasks,
see `Task`.
[`Task.init(name:priority:operation:)`]:
https://developer.apple.com/documentation/swift/task/init(name:priority:operation:)-43wmk
[`Task.detached(name:priority:operation:)`]:
https://developer.apple.com/documentation/swift/task/detached(name:priority:operation:)-795w1

Contoh Kode / Perintah CLI:

```
let newPhoto = // ... some photo data ...
let handle = Task {
    return await add(newPhoto, toGalleryNamed: "Spring Adventures")
}
let result = await handle.value
```

[swift_141] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Isolation

Sumber: ht

Isolation
The previous sections discuss approaches for splitting up concurrent work.
That work can involve changing shared data, such as an app's UI.
If different parts of your code can modify the same data at the same time,
that risks creating a data race.
Swift protects you from data races in your code:
Whenever you read or modify a piece of data,
Swift ensures that no other code is modifying it concurrently.
This guarantee is called *data isolation*.
There are three main ways to isolate data:
1. Immutable data is always isolated.
Because you can't modify a constant,
there's no risk of other code modifying a constant
at the same time you're reading it.
2. Data that's referenced by only the current task is always isolated.
A local variable is safe to read and write
because no code outside the task has a reference to that memory,
so no other code can modify that data.
In addition,
if you capture the variable in a closure,
Swift ensures that the closure isn't used concurrently.
3. Data that's protected by an actor is isolated
if the code accessing that data is also isolated to the actor.
If the current function is isolated to an actor,
it's safe to read and write data that's protected by that actor
because any other code that's isolated to that same actor
must wait its turn before running.

[swift_142] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): The Main Actor

Sumber: ht

The Main Actor
An actor is an object that protects access to mutable data
by forcing code to take turns accessing that data.
The most important actor in many programs is the *main actor*.
In an app,
the main actor protects all of the data that's used to show the UI.
The main actor takes turns rendering the UI,
handling UI events,
and running code that you write that needs to query or update the UI.
Before you start using concurrency in your code,
everything runs on the main actor.
As you identify long-running or resource-intensive code,
you can move this work off the main actor
in a way that's still safe and correct.
> Note:
> The main actor is closely related to the main thread,
> but they're not the same thing.
> The main actor has private mutable state,
> and the main thread serializes access to that state.
> When you run code on the main actor,
> Swift executes that code on the main thread.
> Because of this connection,
> you might see these two terms used interchangeably.
> Your code interacts with the main actor;
> the main thread is a lower-level implementation detail.
There are several ways to run work on the main actor.
To ensure a function always runs on the main actor,
mark it with the `@MainActor` attribute:
In the code above,
the `@MainActor` attribute on the `show(_:)` function
requires this function to run only on the main actor.
Within other code that's running on the main actor,
you can call `show(_:)` as a synchronous function.
However,
to call `show(_:)` from code that isn't running on the main actor,
you have to include `await` and call it as an asynchronous function
because switching to the main actor introduces a potential suspension point.
For example:
In the code above,
both the `downloadPhoto(named:)` and `show(_:)` functions
might suspend wh

Contoh Kode / Perintah CLI:

```
@MainActor
func show(_: Data) {
    // ... UI code to display the photo ...
}

// ----------------------------------------
```

```
func downloadAndShowPhoto(named name: String) async {
    let photo = await downloadPhoto(named: name)
    await show(photo)
}

// ----------------------------------------

let photo = await downloadPhoto(named: "Trees at Sunrise")
Task { @MainActor in
    show(photo)
}
```

[swift_143] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Actors

Sumber: ht

Actors
Swift provides the main actor for you ---
you can also define your own actors.
Actors let you safely share information between concurrent code.
Like classes, actors are reference types,
so the comparison of value types and reference types
in
applies to actors as well as classes.
Unlike classes,
actors allow only one task to access their mutable state at a time,
which makes it safe for code in multiple tasks
to interact with the same instance of an actor.
For example, here's an actor that records temperatures:
You introduce an actor with the `actor` keyword,
followed by its definition in a pair of braces.
The `TemperatureLogger` actor has properties
that other code outside the actor can access,
and restricts the `max` property so only code inside the actor
can update the maximum value.
You create an instance of an actor
using the same initializer syntax as structures and classes.
When you access a property or method of an actor,
you use `await` to mark the potential suspension point.
For example:
In this example,
accessing `logger.max` is a possible suspension point.
Because the actor allows only one task at a time to access its mutable state,
if code from another task is already interacting with the logger,
this code suspends while it waits to access the property.
In contrast,
code that's part of the actor doesn't write `await`
when accessing the actor's properties.
For example,
here's a method that updates a `TemperatureLogger` with a new temperature:
The `update(with:)` method is already running on the actor,
so it doesn't mark its access to properties like `max` with `await`.

This method also shows one of the reasons
why actors allow only one task at a time to interact with their mutable state:
Some updates to an actor's state temporarily break invariants.

Contoh Kode / Perintah CLI:

```
actor TemperatureLogger {
    let label: String
    var measurements: [Int]
    private(set) var max: Int

    init(label: String, measurement: Int) {
        self.label = label
        self.measurements = [measurement]
        self.max = measurement
    }
}

// ----------------------------------------

-->

You introduce an actor with the `actor` keyword,
followed by its definition in a pair of braces.
The `TemperatureLogger` actor has properties
that other code outside the actor can access,
and restricts the `max` property so only code inside the actor
can update the maximum value.

You create an instance of an actor
using the same initializer syntax as structures and classes.
When you access a property or method of an actor,
you use `await` to mark the potential suspension point.
For example:

// ----------------------------------------

In this example,
accessing `logger.max` is a possible suspension point.
Because the actor allows only one task at a time to access its mutable state,
if code from another task is already interacting with the logger,
this code suspends while it waits to access the property.

In contrast,
code that's part of the actor doesn't write `await`
when accessing the actor's properties.
For example,
here's a method that updates a `TemperatureLogger` with a new temperature:
```

[swift_144] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Global Actors

Sumber: ht

Global Actors
The main actor is a global singleton instance of the [`MainActor`][] type.
An actor can normally have multiple instances,
each of which provides independent isolation.
This is why you declare all of an actor's isolated data

as instance properties of that actor.
However, because `MainActor` is a singleton ---
there is only ever a single instance of this type ---
the type alone is sufficient to identify the actor,
allowing you to mark main-actor isolation using just an attribute.
This approach gives you more flexibility to organize your code
in the way that works best for you.
[`MainActor`]: https://developer.apple.com/documentation/swift/mainactor
You can define your own singleton global actors
using the `@globalActor` attribute,
as described in .

[swift_145] Swift - Modern Concurrency (Swift 6 Strict)

Concurrency (async/await, Task and TaskGroup, Actors data isolation, Sendable types): Sendable Types

Sumber: ht

Sendable Types
Tasks and actors let you divide a program
into pieces that can safely run concurrently.
Inside of a task or an instance of an actor,
the part of a program that contains mutable state,
like variables and properties,
is called a *concurrency domain*.
Some kinds of data can't be shared between concurrency domains,
because that data contains mutable state,
but it doesn't protect against overlapping access.
A type that can be shared from one concurrency domain to another
is known as a *sendable* type.
For example, it can be passed as an argument when calling an actor method
or be returned as the result of a task.
The examples earlier in this chapter didn't discuss sendability
because those examples use simple value types
that are always safe to share
for the data being passed between concurrency domains.
In contrast,
some types aren't safe to pass across concurrency domains.
For example, a class that contains mutable properties
and doesn't serialize access to those properties
can produce unpredictable and incorrect results
when you pass instances of that class between different tasks.
You mark a type as being sendable
by declaring conformance to the `Sendable` protocol.
That protocol doesn't have any code requirements,
but it does have semantic requirements that Swift enforces.
In general, there are three ways for a type to be sendable:
- The type is a value type,
and its mutable state is made up of other sendable data ---
for example, a structure with stored properties that are sendable
or an enumeration with associated values that are sendable.
- The type doesn't have any mutable state,
and its immutable state is made up of other sendable data ---

for example, a structure or class that has only read-only properties.
- The type has code that ensures the sa

Contoh Kode / Perintah CLI:

```
struct TemperatureReading: Sendable {
    var measurement: Int
}

extension TemperatureLogger {
    func addReading(from reading: TemperatureReading) {
        measurements.append(reading.measurement)
    }
}

let logger = TemperatureLogger(label: "Tea kettle", measurement: 85)
let reading = TemperatureReading(measurement: 45)
await logger.addReading(from: reading)

// ----------------------------------------

-->

Because `TemperatureReading` is a structure that has only sendable properties,
and the structure isn't marked `public` or `@usableFromInline`,
it's implicitly sendable.
Here's a version of the structure
where conformance to the `Sendable` protocol is implied:

// ----------------------------------------

<!--
  - test: `actors-implicitly-sendable`
```

[swift_146] Swift - Modern Metaprogramming - Swift Macros

Macros (Freestanding Macros #, Attached Macros @, Compiler AST Transformation): Overview

Sumber: ht

Macros
Use macros to generate code at compile time.
Macros transform your source code when you compile it,
letting you avoid writing repetitive code by hand.
During compilation,
Swift expands any macros in your code before building your code as usual.
!A diagram showing an overview of macro expansion. On the left, a stylized representation of Swift code. On the right, the same code with several lines added by the macro.
Expanding a macro is always an additive operation:
Macros add new code,
but they never delete or modify existing code.
Both the input to a macro and the output of macro expansion
are checked to ensure they're syntactically valid Swift code.
Likewise, the values you pass to a macro
and the values in code generated by a macro
are checked to ensure they have the correct types.
In addition,
if the macro's implementation encounters an error when expanding that macro,
the compiler treats this as a compilation error.

These guarantees make it easier to reason about code that uses macros,
and they make it easier to identify issues
like using a macro incorrectly
or a macro implementation that has a bug.
Swift has two kinds of macros:
- *Freestanding macros* appear on their own,
without being attached to a declaration.
- *Attached macros* modify the declaration that they're attached to.
You call attached and freestanding macros slightly differently,
but they both follow the same model for macro expansion,
and you implement them both using the same approach.
The following sections describe both kinds of macros in more detail.

[swift_147] Swift - Modern Metaprogramming - Swift Macros

Macros (Freestanding Macros #, Attached Macros @, Compiler AST Transformation): Freestanding Macros

Sumber: ht

Freestanding Macros
To call a freestanding macro,
you write a number sign (`#`) before its name,
and you write any arguments to the macro in parentheses after its name.
For example:
In the first line,
`#function` calls the [`function()`][] macro from the Swift standard library.
When you compile this code,
Swift calls that macro's implementation,
which replaces `#function` with the name of the current function.
When you run this code and call `myFunction()`,
it prints "Currently running myFunction()".
In the second line,
`#warning` calls the [`warning(_:)`][] macro from the Swift standard library
to produce a custom compile-time warning.
[`function()`]: https://developer.apple.com/documentation/swift/function()
[`warning(_:)`]: https://developer.apple.com/documentation/swift/warning(_:)
Freestanding macros can produce a value, like `#function` does,
or they can perform an action at compile time, like `#warning` does.

Contoh Kode / Perintah CLI:

```
func myFunction() {
    print("Currently running \(#function)")
    #warning("Something's wrong")
}
```

[swift_148] Swift - Modern Metaprogramming - Swift Macros

Macros (Freestanding Macros #, Attached Macros @, Compiler AST Transformation): Attached Macros

Sumber: ht

Attached Macros
To call an attached macro,
you write an at sign (`@`) before its name,
and you write any arguments to the macro in parentheses after its name.

Attached macros modify the declaration that they're attached to.
They add code to that declaration,
like defining a new method or adding conformance to a protocol.
For example, consider the following code
that doesn't use macros:
In this code,
each of the options in the `SundaeToppings` option set
includes a call to the initializer,
which is repetitive and manual.
It would be easy to make a mistake when adding a new option,
like typing the wrong number at the end of the line.
Here's a version of this code that uses a macro instead:
This version of `SundaeToppings` calls an `@OptionSet` macro.
The macro reads the list of cases in the private enumeration,
generates the list of constants for each option,
and adds a conformance to the [`OptionSet`][] protocol.
[`OptionSet`]: https://developer.apple.com/documentation/swift/optionset
For comparison,
here's what the expanded version of the `@OptionSet` macro looks like.
You don't write this code,
and you would see it only if you specifically asked Swift
to show the macro's expansion.
All of the code after the private enumeration
comes from the `@OptionSet` macro.
The version of `SundaeToppings`
that uses a macro to generate all of the static variables
is easier to read and easier to maintain
than the manually coded version, earlier.

Contoh Kode / Perintah CLI:

```
struct SundaeToppings: OptionSet {
    let rawValue: Int
    static let nuts = SundaeToppings(rawValue: 1 << 0)
    static let cherry = SundaeToppings(rawValue: 1 << 1)
    static let fudge = SundaeToppings(rawValue: 1 << 2)
}

// ----------------------------------------

@OptionSet<Int>
struct SundaeToppings {
    private enum Options: Int {
        case nuts
        case cherry
        case fudge
    }
}

// ----------------------------------------

struct SundaeToppings {
    private enum Options: Int {
        case nuts
        case cherry
        case fudge
    }
```

```
    typealias RawValue = Int
    var rawValue: RawValue
    init() { self.rawValue = 0 }
    init(rawValue: RawValue) { self.rawValue = rawValue }
    static let nuts: Self = Self(rawValue: 1 << Options.nuts.rawValue)
    static let cherry: Self = Self(rawValue: 1 << Options.cherry.rawValue)
    static let fudge: Self = Self(rawValue: 1 << Options.fudge.rawValue)
}
extension SundaeToppings: OptionSet { }
```

[swift_149] Swift - Modern Metaprogramming - Swift Macros

Macros (Freestanding Macros #, Attached Macros @, Compiler AST Transformation): Macro Declarations

Sumber: ht

Macro Declarations
In most Swift code,
when you implement a symbol, like a function or type,
there's no separate declaration.
However, for macros, the declaration and implementation are separate.
A macro's declaration contains its name,
the parameters it takes,
where it can be used,
and what kind of code it generates.
A macro's implementation contains the code
that expands the macro by generating Swift code.
You introduce a macro declaration with the `macro` keyword.
For example,
here's part of the declaration for
the `@OptionSet` macro used in the previous example:
The first line
specifies the macro's name and its arguments ---
the name is `OptionSet`, and it doesn't take any arguments.
The second line
uses the [`externalMacro(module:type:)`][] macro from the Swift standard library
to tell Swift where the macro's implementation is located.
In this case,
the `SwiftMacros` module
contains a type named `OptionSetMacro`,
which implements the `@OptionSet` macro.
[`externalMacro(module:type:)`]: https://developer.apple.com/documentation/swift/externalmacro(module:type:)
Because `OptionSet` is an attached macro,
its name uses upper camel case,
like the names for structures and classes.
Freestanding macros have lower camel case names,
like the names for variables and functions.
> Note:
> Macros are always declared as `public`.
> Because the code that declares a macro
> is in a different module from code that uses that macro,
> there isn't anywhere you could apply a nonpublic macro.
A macro declaration defines the macro's *roles* ---
the places in source code where that macro can be called,

and the kinds of code the macro can generate.
Every macro has one or more roles,
which you write as part of the attributes
at the beginning of the macro declaration.
Here's a bit more of

Contoh Kode / Perintah CLI:

```
public macro OptionSet<RawType>() =
        #externalMacro(module: "SwiftMacros", type: "OptionSetMacro")

// ----------------------------------------

@attached(member)
@attached(extension, conformances: OptionSet)
public macro OptionSet<RawType>() =
        #externalMacro(module: "SwiftMacros", type: "OptionSetMacro")

// ----------------------------------------

@freestanding(expression)
public macro line<T: ExpressibleByIntegerLiteral>() -> T =
        /* ... location of the macro implementation... */
```

[swift_150] Swift - Modern Metaprogramming - Swift Macros

Macros (Freestanding Macros #, Attached Macros @, Compiler AST Transformation): Macro Expansion

Sumber: ht

Macro Expansion
When building Swift code that uses macros,
the compiler calls the macros' implementation to expand them.
!Diagram showing the four steps of expanding macros. The input is Swift source code. This becomes a tree, representing the code's structure. The macro implementation adds branches to the tree. The result is Swift source with additional code.
Specifically, Swift expands macros in the following way:
1. The compiler reads the code,
creating an in-memory representation of the syntax.
1. The compiler sends part of the in-memory representation
to the macro implementation,
which expands the macro.
1. The compiler replaces the macro call with its expanded form.
1. The compiler continues with compilation,
using the expanded source code.
To go through the specific steps, consider the following:
The `#fourCharacterCode` macro takes a string that's four characters long
and returns an unsigned 32-bit integer
that corresponds to the ASCII values in the string joined together.
Some file formats use integers like this to identify data
because they're compact but still readable in a debugger.
The section below
shows how to implement this macro.
To expand the macros in the code above,
the compiler reads the Swift file
and creates an in-memory representation of that code
known as an *abstract syntax tree*, or AST.

The AST makes the code's structure explicit,
which makes it easier to write code that interacts with that structure ---
like a compiler or a macro implementation.
Here's a representation of the AST for the code above,
slightly simplified by omitting some extra detail:
!A tree diagram, with a constant as the root element. The constant has a name, magic number, and a value. The constant's value is a macro call. The macro call has a name, fourCharacter

Contoh Kode / Perintah CLI:

```
let magicNumber = #fourCharacterCode("ABCD")

// ----------------------------------------

let magicNumber = 1145258561 as UInt32

// ----------------------------------------

let something = #someMacro {
      struct A { }
      @someMacro struct B { }
  }
```

[swift_151] Swift - Modern Metaprogramming - Swift Macros

Macros (Freestanding Macros #, Attached Macros @, Compiler AST Transformation): Implementing a Macro

Sumber: ht

Implementing a Macro
To implement a macro, you make two components:
A type that performs the macro expansion,
and a library that declares the macro to expose it as API.
These parts are built separately from code that uses the macro,
even if you're developing the macro and its clients together,
because the macro implementation runs
as part of building the macro's clients.
To create a new macro using Swift Package Manager,
run `swift package init --type macro` ---
this creates several files,
including a template for a macro implementation and declaration.
To add macros to an existing project,
edit the beginning of your `Package.swift` file as follows:
- Set a Swift tools version of 5.9 or later in the `swift-tools-version` comment.
- Import the `CompilerPluginSupport` module.
- Include macOS 10.15 as a minimum deployment target in the `platforms` list.
The code below shows the beginning of an example `Package.swift` file.
Next, add a target for the macro implementation
and a target for the macro library
to your existing `Package.swift` file.
For example,
you can add something like the following,
changing the names to match your project:
The code above defines two targets:
`MyProjectMacros` contains the implementation of the macros,
and `MyProject` makes those macros available.

The implementation of a macro
uses the [SwiftSyntax][] module to interact with Swift code
in a structured way, using an AST.
If you created a new macro package with Swift Package Manager,
the generated `Package.swift` file
automatically includes a dependency on SwiftSyntax.
If you're adding macros to an existing project,
add a dependency on SwiftSyntax in your `Package.swift` file:
[SwiftSyntax]: https://github.com/swiftlang/swift-syntax
Depending on your macro's role,
there's a corresponding pro

Contoh Kode / Perintah CLI:

```
// swift-tools-version: 5.9

import PackageDescription
import CompilerPluginSupport

let package = Package(
    name: "MyPackage",
    platforms: [ .iOS(.v17), .macOS(.v13)],
    // ...
)

// ---------------------------------------

targets: [
    // Macro implementation that performs the source transformations.
    .macro(
        name: "MyProjectMacros",
        dependencies: [
            .product(name: "SwiftSyntaxMacros", package: "swift-syntax"),
            .product(name: "SwiftCompilerPlugin", package: "swift-syntax")
        ]
    ),

    // Library that exposes a macro as part of its API.
    .target(name: "MyProject", dependencies: ["MyProjectMacros"]),
]

// ---------------------------------------

dependencies: [
    .package(url: "https://github.com/swiftlang/swift-syntax", from: "509.0.0")
],
```

[swift_152] Swift - Modern Metaprogramming - Swift Macros

Macros (Freestanding Macros #, Attached Macros @, Compiler AST Transformation): Developing and Debugging Macros

Sumber: ht

Developing and Debugging Macros
Macros are well suited to development using tests:
They transform one AST into another AST
without depending on any external state,
and without making changes to any external state.
In addition, you can create syntax nodes from a string literal,
which simplifies setting up the input for a test.

You can also read the `description` property of an AST
to get a string to compare against an expected value.
For example,
here's a test of the `#fourCharacterCode` macro from previous sections:
The example above tests the macro using a precondition,
but you could use a testing framework instead.

Contoh Kode / Perintah CLI:

```
let source: SourceFileSyntax =
    """
    let abcd = #fourCharacterCode("ABCD")
    """

let file = BasicMacroExpansionContext.KnownSourceFile(
    moduleName: "MyModule",
    fullFilePath: "test.swift"
)

let context = BasicMacroExpansionContext(sourceFiles: [source: file])

let transformedSF = source.expand(
    macros:["fourCharacterCode": FourCharacterCode.self],
    in: context
)

let expectedDescription =
    """
    let abcd = 1145258561 as UInt32
    """

precondition(transformedSF.description == expectedDescription)
```